

UNIVERSITÀ
DI TORINO

Università degli Studi di Torino

Laurea Magistrale in Astrofisica e Fisica Teorica

Effects of Dirichlet Boundary Conditions in Yang-Mills Theories

Relatore

Michele Caselle

Co-relatori

Lorenzo Verzichelli

Dario Panfalone

Candidato

Marco Nebbia

819691

Anno Accademico 2024/2025

Parleremo per immagini
per tenere gli occhi bene aperti su quanto non si vede.
Ritroveremo le cassette degli attrezzi
per riparare l'ideale,
e il nostro cinismo farà sbocciare nuovi sogni.
Innescheremo la macchina della volontà
per produrre domande che spostino cattedrali.
Affinchè tutto accada di nuovo
oltre che nulla vada perduto.
Saremo il fatale prevalere dell'azione.
Per sempre, tutto comincia ora.

Raein - Nirvana

Abstract

(2+1)-dimensional Yang-Mills Theory with SU(2) as gauge group is studied via lattice regularization. Dirichlet boundary conditions on time dimension are enforced to study differences arising with respect to usual periodic boundary conditions. Numerical simulations based on Monte Carlo algorithms have been performed.

Ringraziamenti

Ringrazio Michele Caselle che, con il suo corso "Teoria Statistica dei Campi", mi ha appassionato e stimolato, facendo nascere la curiosità che muove questa tesi. L'elaborato nasce da una sua idea e, una volta assunte le vesti di relatore, i nostri incontri sono sempre stati fonte di spunti essenziali a comprendere a fondo il problema.
Ringrazio Lorenzo Verzichelli e Dario Panfalone, la loro disponibilità nel seguirmi e guidarmi attraverso i problemi incontrati, la loro esperienza e profonda padronanza dell'argomento trattato. Il loro contributo è stato fondamentale, sia a livello pratico che a livello umano.
Ringrazio tutto il gruppo di ricerca, che mi ha accolto e coinvolto. Si è mostrato ricco di idee, progetti e sempre interessato all'evoluzione del mio percorso.

Dediche

Dedico questa tesi alla mia famiglia, che mi ha permesso di completare un percorso lungo e sicuramente non lineare, senza mai farmi mancare un sostegno senza il quale questa tesi non sarebbe, probabilmente, nemmeno iniziata. A mia madre, che mi ha fornito aiuto in qualsiasi modo fosse possibile, spesso anteponendo le mie richieste ai suoi desideri. A mio padre, che mi ha ispirato e fatto scoprire il divertimento nel risolvere problemi, senza mai prendersi troppo sul serio. A mio fratello, che ha trovato la sua dimensione nonostante la mia assenza durante gli anni della sua crescita, e da cui traggo esempio.
Non posso non dedicare questa tesi alle persone che mi sono state accanto.
Bianca, Linda e Matteo, avete ridefinito la mia quotidianità, il mio concetto di amicizia. Abbiamo condiviso emozioni, timori, sorrisi ed esperienze così diverse che fatico a concepire come sia possibile che sia successo tutto così velocemente. Quando penso che se mi fossi laureato secondo i canoni non vi avrei incontrato, sono contento di non aver superato un esame per due anni. A tutti i membri dell'Osservatorio, che hanno reso questa facoltà più di un luogo dove studiare: un luogo dove conoscere, sia persone che concetti nuovi.
A "420", la cui dolcezza nei miei confronti mi è parsa a tratti incomprensibile. Ricevere uno dei vostri tanti inviti è il modo migliore di rallegrare una giornata. Alla mia classe di liceo, che mi ha fatto scoprire quanto duraturo possa essere un'amicizia anche quando si prendono strade diverse, e che c'è sempre un modo per ritrovarsi. A Nina e Veronica, vi ringrazio di aver trascorso con me il vostro periodo a Torino e di avermi accolto a casa vostra nelle occasioni più disparate. A tutti i gruppi di amici lontani, con i quali mi sento libero di essere sincero e di parlare di qualsiasi argomento. In tutti questi anni, non mi sono mai sentito solo.
Ad Aurora, Hana, Micaela e Giulia, che conoscendomi a fondo hanno capito parti di me che non avevo, fino ad allora, mai riconosciuto. Camminare di fianco a voi è stato, per me, un regalo inaspettato e una fonte di gioia sempre nuova.
Un ultimo pensiero a chiunque abbia creato la bellezza che mi ha raggiunto: musica, libri, film, fotografie non sono solamente distrazioni, ma segnali che ci sia possibilità di sublimare

la dimensione umana e personale. Ogni opera pregiata ravviva il fuoco della curiosità.

Contents

1 Introduction **1**

1.1 Historical Background 1

1.2 Lattice Regularization 3

1.3 Yang-Mills Theories 5

2 Theoretical Framework **8**

2.1 Dirichlet Boundary Conditions 8

2.2 Casimir Effect and Principal Chiral Model 9

2.2.1 Casimir Effect 9

2.2.2 Principal Chiral Model 11

2.3 Lattice Observables 13

2.3.1 Plaquette 13

2.3.2 Wilson Loop 14

2.3.3 Polyakov Loop 16

3 Monte Carlo Methods **20**

3.1 Monte Carlo Basics 20

3.2 Markov Chains 22

3.3 Markov Chain Monte Carlo 24

3.4 MCMC Data Analysis 27

4 Results **31**

4.1 Lattice Setup 31

4.2 Quality Check 32

4.3 Measures 35

4.3.1 Plaquettes 36

4.3.2 Polyakov Loop 38

4.3.3 Polyakov Loop Correlator 41
4.3.4 Wilson Loop 43

5 Conclusion **48**

A Supplements **50**
A.1 Continuum Limit of Wilson Action 50
A.2 Periodicity of Markov Chains 52
A.3 Stochastic Matrix Properties 53

B Additional Figures **58**

C Simulation Code **65**
C.1 Lattice 65
C.2 Gauge Variables 67
C.3 Measures 75
C.4 Main 87

Chapter 1

Introduction

1.1 Historical Background

Quantum Field Theories (QFTs), since their appearance in the middle of XX century, have been the most powerful tool to describe interactions between subatomic particles. These theoretical frameworks are based on the path integral formulation of Quantum Mechanics, completed by Feynman [1], for which the evolution of a quantum system in time was obtained by considering every possible trajectory in phase space:

$$\langle q_2|q_1\rangle = \int \mathrm{e}^{\frac{\mathrm{i}}{\hbar}\mathscr{S}[q]}\,[\mathscr{D}q]\,, \tag{1.1}$$

where $|q_i\rangle \coloneqq |q\,(t_i)\rangle$ are the starting and ending states,

$$\hat{U}\,(t_2,t_1) = \mathrm{e}^{-\mathrm{i}\frac{t_2-t_1}{\hbar}\hat{\mathscr{H}}} \tag{1.2}$$

is the time-evolution operator defined from the Hamiltonian $\hat{\mathscr{H}}$ and $\mathscr{S}\,[q] = \int \mathscr{L}\,[q]\,\mathrm{d}^D x\,\mathrm{d}t$ is the action describing system dynamics that weighs each trajectory between the two states. The path integral measure $[\mathscr{D}q]$ is what accounts for every possible intermediate configuration of the system. It is formally defined as an infinite product of measures at every time instant [2]

$$[\mathscr{D}q] \coloneqq \prod_{t\in(t_1,t_2)} \mathrm{d}q\,(t)\,, \tag{1.3}$$

but to practically perform the computation, it is necessary to discretize the time difference $t_2 - t_1 = Na$ and integrate on a finite number of evolution points.

In Quantum Field Theory, physical information is encoded in Green functions, i.e. time-

ordered correlators of field operators $\hat{\phi}(\vec{x},t)$ that evolve in time by the Hamiltonian following Heisenberg representation [3]

$$\hat{\phi}(\vec{x},t) = \mathrm{e}^{\mathrm{i}\mathscr{H}t}\hat{\phi}(\vec{x},0)\,\mathrm{e}^{-\mathrm{i}\mathscr{H}t}. \tag{1.4}$$

To build a path integral representation of Green functions, it is convenient to perform the Wick rotation $t \mapsto -\mathrm{i}t$ that maps the Minkowski spacetime to the euclidean one. The outcome is that the complex oscillating exponential containing the action becomes a real decreasing one, thus it is possible to use it as a Boltzmann weight of a statistical distribution. This allows to write formally

$$\langle 0|\,\mathrm{T}\left[\hat{\phi}(x_1)\ldots\hat{\phi}(x_n)\right]|0\rangle = \frac{1}{Z}\int \phi(x_1)\ldots\phi(x_n)\,\mathrm{e}^{-\mathscr{S}_{\mathrm{E}}[\{\phi\}]}\,[\mathscr{D}\phi]\,, \tag{1.5}$$

where $\mathscr{S}_{\mathrm{E}}$ is the euclidean action, obtained after the Wick rotation, and the normalization factor Z is defined as

$$Z \coloneqq \int \mathrm{e}^{-\mathscr{S}_{\mathrm{E}}[\{\phi\}]}\,[\mathscr{D}\phi]\,. \tag{1.6}$$

The benefit of matching statistical mechanics formalism is to have a suitable way of studying the system at a specific temperature. When choosing boundary conditions, it is common to opt for periodic boundary conditions in all directions. On space dimensions it is straightforward to implement them, but on time dimension it means that the initial and final state of the system, if posed at the start and at the end of time dimension, have to match: for a dynamics that starts at t_{in} and ends at t_{fin} periodic boundary conditions on time are written as

$$\phi(\vec{x},t_{\mathrm{in}}) = \phi(\vec{x},t_{\mathrm{fin}})\,. \tag{1.7}$$

The time evolution of this field in euclidean formulation has the form

$$\langle\phi(\vec{x},t_{\mathrm{in}})|\,\mathrm{e}^{-\frac{t_{\mathrm{fin}}-t_{\mathrm{in}}}{\hbar}\mathscr{H}}\,|\phi(\vec{x},t_{\mathrm{fin}})\rangle$$

that, if (1.7) is enforced, it is precisely the term that construct the partition function in a canonical statistical ensemble

$$\mathrm{tr}\left(\mathrm{e}^{-\beta\mathscr{H}}\right) = \sum_{\{n\}}\langle n|\,\mathrm{e}^{-\beta\mathscr{H}}\,|n\rangle\,. \tag{1.8}$$

The canonical ensemble at temperature $T = \frac{1}{k_{\mathrm{B}}\beta}$ is, in fact, describing a statistical system at equilibrium point and the boundary conditions on time reflect this property. By the

identification between two formalisms, it is possible to define the temperature for a QFT: if $k_B = 1 = \hbar$ then $t_{fin} - t_{in} = \frac{1}{T}$. It is possible to study finite temperature systems using this relation, and to change its value by modifying the time size: shrinking vertically means raising the temperature and vice versa. For a large class of theories, there exists a critical value T_C for which it undergoes a phase transition between two regimes.

Looking at the most famous QFT, *Quantum Electrodynamics* (QED), the procedure to obtain astonishingly accurate theoretical predictions was perturbation theory. The interacting part of the exponential was expanded in series, producing an infinite number of terms, represented graphically by Feynman diagrams. Even if difficulty in solving such integrals made impossible to compute contributions after a certain order, what was achieved was enough to match extremely well the experimental data. It is important to point out that series expansion was allowed by the fact that QED coupling factor has a small value, $\alpha \ll 1$, and that approaching some of these diagrams in a straightforward way yields infinite results that have no physical meaning. This problem arises in diagrams where a loop, representing virtual particles, is formed; when considering scenarios of said particles having really high (UV divergences) or really small (IR divergences) momenta, infinities may appear. To extract physical information from these objects it is necessary to *renormalize* them, namely to reabsorb the infinities and to highlight the meaningful part of such terms. Many approaches have been followed to do so, one of which is lattice regularization.

The QFT that describes internal structure of hadrons, i.e. that sees them as bound states of quarks and gluons, is the *Quantum Chromodynamics* (QCD). It is a non-abelian gauge theory that is weakly interacting at short distance between quarks, a property called "asymptotic freedom" that allows perturbative approach in such regime. To show confinement of partons inside hadrons, however, perturbation theory isn't viable: at large distances between quarks, interaction is too big and the series expansion ceases to produce physically relevant results after a very small number of terms compared to what happens in QED. This barrier limits the number of observables that can be studied analytically in QCD when sticking to its continuum formulation.

1.2 Lattice Regularization

Lattice regularization provides a natural framework, especially for QCD, to treat these issues in a straightforward way:

- it discretizes not only time - as it is common to do with path integrals - but also space, reflecting the relativistic principle of QFTs of treating time as a coordinate;

- it is commonly defined as an euclidean geometry, so it is useful to study QFTs after Wick rotation, with all the statistical tools previously discussed;
- finite and non-zero lattice sizes $\left\{N^{(i)}\right\}$ and spacing a protect from both IR and UV divergences, since high momenta of virtual particles are linked to short sized loops and vice versa.

After Wilson proposed this regularization for QCD [4] a new class of macroscopic observables became accessible by numerical evaluation, thus the spark of interest on studying QFTs using this setup.

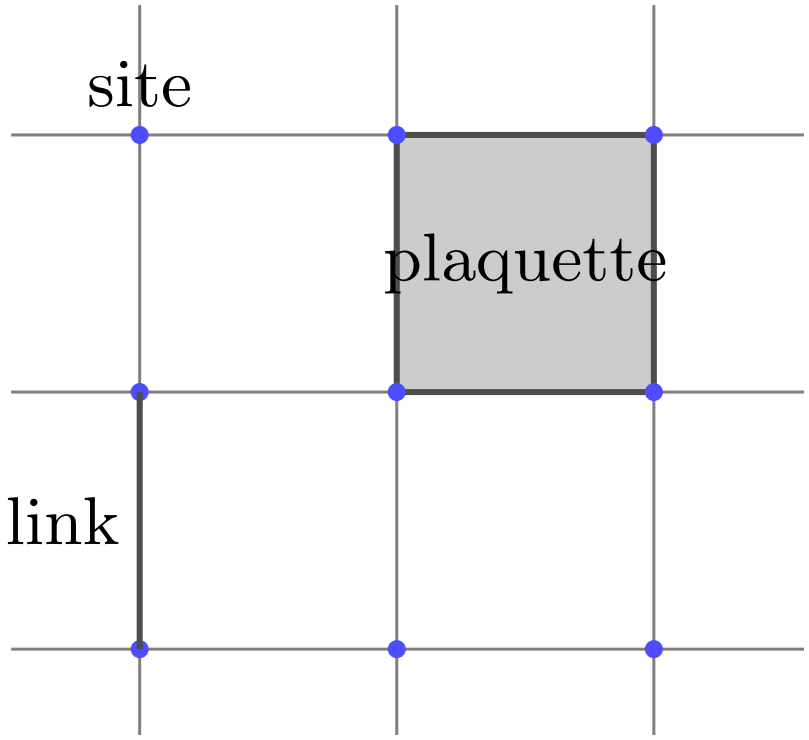


Figure 1.1: Lattice elements.

In practical terms, the usual D-dimensional continuum spacetime is replaced by a lattice (usually hyper-cubic) of uniform spacing a and fields can only exist on lattice elements. The one vertical dimension of the lattice is commonly identified with time direction and its size is $N_t \cdot a$, while all other $D-1$ "horizontal" directions represent spatial dimensions, sized $\{N_s^{(i)} \cdot a\}$. Integrals are replaced by sums on all sites $\{x\}$ and derivatives by finite differences

between next neighbours in chosen direction

$$\int \mathrm{d}^D x \longmapsto a^D \sum_{\{x\}}, \qquad \partial_\mu \phi(x) \longmapsto \nabla_\mu \phi(x) \coloneqq \frac{\phi(x + a\hat{\mu}) - \phi(x)}{a}, \tag{1.9}$$

where $\hat{\mu}$ is the unit vector in direction μ across spacetime dimensions and the hat on operators has been dropped to not be mistaken with unit vector notation. Bosonic fields of grade k lay on k-dimensional geometrical objects, so scalar fields are said "to live" on sites, while vectors on links between them.

These requests imply that continuous spacetime symmetries of the system, such as translational and rotational, become discrete and geometry dependent. For the usual hypercubic lattice, the translation invariance reduces to multiples of a and rotational to multiples of 90°. Periodic boundary conditions on time mean that $N_\mathrm{t} = \frac{1}{aT}$, in accordance with what previously discussed for the continuum case.

1.3 Yang-Mills Theories

Yang-Mills theories are QFTs where the gauge Lie group G is chosen to be non-abelian [5], with particular interest on SU (N) groups, $N \geq 2$. The field strength for the matrix-valued field $A^a_\mu(x)$, that now has an index a running on generators $\{\tau_a\}$ of Lie algebra $\mathfrak{g}$ associated to G, becomes

$$F^a_{\mu\nu}(x) = \partial_\mu A^a_\nu(x) - \partial_\nu A^a_\mu(x) + g f^a{}_{bc} A^b_\mu(x) A^c_\nu(x), \tag{1.10}$$

where $f^a{}_{bc}$ are the structure constants obtained by commutation relations of generators $[\tau_a, \tau_b] = \mathrm{i} f_{ab}{}^c$ and g is the coupling strength. The non-abelian nature of the theory is reflected in the interaction term between two bosonic fields. The action in a D-dimensional spacetime is

$$\mathscr{S}_{\mathrm{YM}} = -\frac{1}{4} \int F^a_{\mu\nu}(x) F^{\mu\nu}_a(x) \, \mathrm{d}^D x \,. \tag{1.11}$$

When regularizing the theory on lattice, the gauge fields $A_\mu(x)$ take values on links starting from site x, but they are not the most convenient quantity to work with when solving the theory: they take value in the Lie algebra of the gauge group, that is tangent to the Lie group and hence it is homomorph is only to the region close to identity element of G. Therefore it is useful to exponentiate them to obtain a group valued variable $U_\mu(x) \coloneqq \mathrm{e}^{\mathrm{i}cA_\mu(x)} \in G$ that also lives on links. A mental picture of its nature starts by visualizing every site x as equipped with a different internal vector space span by the basis $\{e_i(x)\}$. If so, $U_\mu(x)$ is

the matrix mapping two next neighbours bases:

$$e_i\left(x + a\hat{\mu}\right) = U_\mu\left(x\right) e_i\left(x\right). \tag{1.12}$$

From now on, it will be set $a = 1$ for the sake of brevity. With this interpretation in mind, it is easy to state that a gauge variable going "backwards" must be the inverse of the previous gauge variable going forward

$$U_{-\mu}\left(x + \hat{\mu}\right) = U_\mu^{-1}\left(x\right), \tag{1.13}$$

since after travelling back and forth on the same link, the ending site would be the starting one and so the basis. A local gauge transformation $V\left(x\right) \in G$ changes the basis of site x and affects the gauge variable such that

$$U_\mu\left(x\right) \longmapsto U'_\mu\left(x\right) = V\left(x\right) U_\mu\left(x\right) V^{-1}\left(x + \hat{\mu}\right). \tag{1.14}$$

The theory has to be invariant under said transformations, hence the building block of the regularized action must be so. A trace of a closed path on the lattice $\mathrm{tr}\left(\prod U_\mu\right)$ is indeed invariant, since every link transformation is compensated by the end of the previous and the start of the next one, and first and last elide themselves by cyclic property. The simplest closed path on lattice is the *plaquette*, that for a hyper-cubic structure takes the form

$$U_{\mu\nu}\left(x\right) \coloneqq U_\mu\left(x\right) U_\nu\left(x + \hat{\mu}\right) U_{-\mu}\left(x + \hat{\mu} + \hat{\nu}\right) U_{-\nu}\left(x + \hat{\nu}\right). \tag{1.15}$$

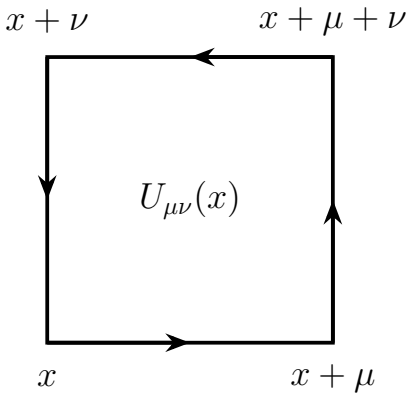


Figure 1.2: Oriented plaquette.

For SU (N) gauge group and by identity (1.13), plaquette can also be written as

$$U_{\mu\nu}(x) = U_\mu(x)\, U_\nu(x+\hat{\mu})\, U_\mu^\dagger(x+\hat{\nu})\, U_\nu^\dagger(x)\,. \tag{1.16}$$

On a lattice with N sites, the Wilson action for Yang-Mills theories is

$$\mathscr{S}_{\mathrm{w}} = \frac{\beta}{2N} \sum_{\{x\}} \sum_{\{\mu,\nu\}} \mathrm{Re\ tr}\left[U_{\mu\nu}(x)\right], \tag{1.17}$$

where only the real part of the complex trace is taken to have a well-defined Boltzmann weight and to choose the quantity that describes system dynamics. In fact, summing on all possible directions yields a plaquette covered clockwise but also counter-clockwise, so its adjoint: the real part is the common term of both results. However, for SU (2) gauge group, the trace of a gauge matrix is already real.

For this action to match the Yang-Mills (1.11) in the continuum limit, it is necessary to make an identification that implies $\beta(g) \propto g^{-2}$, so the β parameter is the inverse of the (square of) theory coupling constant. The derivation is shown in appendix A.1. By this relation, it is possible to control the interaction strength by tuning the β parameter.

Chapter 2

Theoretical Framework

2.1 Dirichlet Boundary Conditions

The purpose of this project is to explore the consequences of replacing periodic boundary conditions with Dirichlet ones on a single direction, namely, to fix the values of gauge fields on the boundary faces of the lattice. The setup for this work will have $G = \mathrm{SU}(2)$ as gauge group and $D = 2 + 1$, so a lattice with two spatial and one temporal dimensions. To enhance clarity, from now on these will be the assumptions to work with; however, the examination that will follow along this chapter can be generalized to an arbitrary number of dimensions or to a different gauge group.

The way to introduce Dirichlet boundary conditions is to define the border $B = B_1 \cup B_2$ as the union of the two opposite extreme faces of the lattice on a chosen direction and to require the gauge field to vanish on both of them $A_\mu(x) = 0$, $x \in B$. On time direction for a lattice with N_t vertical links and $N_t + 1$ vertical points this translates to a condition on links laying on these faces

$$U_\mu(\vec{x}, t = 0) = \mathbb{1} = U_\mu(\vec{x}, t = N_t) \qquad \forall \vec{x}, \forall \mu \neq 0. \tag{2.1}$$

This requirement breaks the discrete translation symmetry on vertical direction of the lattice, so every "time slice" is different from the other. However, it is easily expected that the change in quantities measured at different time slices must follow a symmetric trend with respect to the halfway slice. Of course, this choice should affect the system the more the two faces are close to each other, while in the limit $N_t \gg 1$ results should match the ones obtained with periodic boundary conditions on all directions.

It is interesting to study this scenario for various reasons. Firstly, periodic boundary

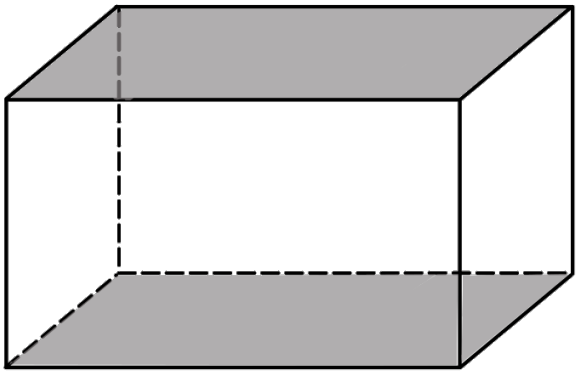


Figure 2.1: Dirichlet boundary conditions on a 2+1-dimensional lattice.

conditions on time were the reason temperature can be defined for the lattice system and, thus, observe a phase transition. If the fields on borders have to assume a fixed value, how will it impact the behaviour as temperature is raised? In second place, in a configuration where the two Dirichlet faces are much closer than their size $N_t \ll N_s$, the setup resembles two plates of a capacitor, with the lattice describing the vacuum between them. Given this analogy, the free energy of the gauge theory can be linked to the Casimir effect [6], which describes in fact the vacuum expectation value of the energy inside the capacitor. Moreover, if the lattice is so compressed that it reduces to a single vertical link between the two borders, all the horizontal links are fixed. Under these circumstances, the 3-dimensional gauge theory becomes a 2-dimensional spin theory, the Principal Chiral Model, since the vertical links can be viewed as spin variables in a horizontal plane.

2.2 Casimir Effect and Principal Chiral Model

2.2.1 Casimir Effect

Casimir effect is a phenomenon that describes the change in vacuum expectation energy of a quantum theory when macroscopic material objects are placed close to each other and the space in between them is examined, the first and most common example are two parallel perfectly conductive plates. In electrodynamics the effect arises from the impossibility for the electric field to penetrate into the material, i.e. the request for it to vanish on the surface of both plates, assuming the metal slabs have infinite electric conductivity. It can be thought as if this poses a limit on the frequencies the field can assume between the plates to satisfy this constraint. Thus, its vacuum expectation energy is lower than

vacuum outside the capacitor one's, resulting in a net force between the plates. On a four dimensional Minkowski spacetime equipped with the metric $\eta = \text{diag}\,(+1, -1, -1, -1)$, the relation between U (1) field strength and electric and magnetic fields is the well-known

$$F = \begin{pmatrix} 0 & E_x & E_y & E_z \\ -E_x & 0 & -B_z & B_y \\ -E_y & B_z & 0 & -B_x \\ -E_z & -B_y & B_x & 0 \end{pmatrix}. \tag{2.2}$$

Hence, temporal part of the tensor is related to electric field, spatial part to magnetic one. It can be generalized for non abelian gauge theories by remembering that $A_\mu = \left\{A_\mu^a\right\}$ is a collection of Lie algebra variables now, and so will be the "chromo-electric" E and "chromo-magnetic" B fields. The energy density is obtained by constructing the stress-energy tensor for the chromo-electromagnetic field

$$T_{\mu\nu} = -F_{\mu\alpha}F_\nu{}^\alpha + \frac{1}{4}\eta_{\mu\nu}F_{\alpha\beta}F^{\alpha\beta}, \tag{2.3}$$

and taking its temporal-temporal component. By noting that $\frac{1}{4}F_{ij}F^{ij} = \frac{1}{2}B_iB^i$, it follows that its form is

$$T_{00} = \frac{1}{2}\left(\vec{B}^2 + \vec{E}^2\right). \tag{2.4}$$

After applying Wick rotation to land into euclidean space, only the temporal components are affected and change their sign, becoming $E_i^2 \mapsto -E_i^2$, thus

$$T_{00}^{(\text{E})} = \frac{1}{2}\left(\vec{B}^2 - \vec{E}^2\right). \tag{2.5}$$

In the scenario where $D = 2 + 1$, field strength drops its last row and column

$$F = \begin{pmatrix} 0 & E_x & E_y \\ -E_x & 0 & -B \\ -E_y & B & 0 \end{pmatrix}, \tag{2.6}$$

maintaining the identification between chromo-electric field and temporal components, chromo-magnetic field and spatial. The setup of the problem is symmetric for rotations of $\theta = \frac{\pi}{2}$ on the xy plane, hence the expectation values of the chromo-electric fields have to

be the same $\langle E_x^2 \rangle = \langle E_y^2 \rangle$. In this framework, energy density (2.5) becomes

$$T_{00}^{(\mathrm{E})} = \frac{1}{2}\left(B^2 - 2E^2\right). \tag{2.7}$$

2.2.2 Principal Chiral Model

The Principal Chiral Model (PCM), also referred as Principal Chiral Field, is a QFT that has been extensively studied since its first introduction in 1960 [7], especially in its 2-dimensional formulation. It is a spin theory, hence its field $U(x)$ is a Minkowski scalar, taking value in a Lie group G. The fact that the field is defined on the whole group is the source of the "principal" epithet, since in topology and differential geometry the fibre bundle which fibres are isomorphic to the group G specifying the action on them is called *principal G-bundle*. It has been of such interest since it has a lot of features in common with QCD in a four-dimensional spacetime, such as non-abelian symmetry and asymptotic freedom. Indeed, this is not an isolated occurrence, it is widely acknowledged that a spin theory in D dimensions resembles a gauge theory in $2D$ dimensions [8]: in first ones, fields are defined on sites and action on links, while in the second ones fields take values on links and action is built from plaquettes; what is interpreted as the mass of a particle for a spin theory is viewed as tension of a string for the gauge theory, and the list of analogies goes on.

Given the similarity with QCD, which is the final model of interest, let's consider $G = \mathrm{SU}(N)$, $N \geq 2$, and start with the continuum formulation of PCM to better highlight its characteristics. The action is

$$\mathscr{S} = \frac{1}{g^2}\int \mathrm{Re\ tr}\left[\partial_\mu U(x)\,\partial^\mu U^\dagger(x)\right]\mathrm{d}^2x\,, \tag{2.8}$$

where g is the coupling constant. Action is invariant independently under two different global transformations $\mathrm{SU}(N)_{\mathrm{L}}$ and $\mathrm{SU}(N)_{\mathrm{R}}$ acting on field as

$$\begin{aligned} &V_{\mathrm{L}}:\ U(x) \longmapsto V_{\mathrm{L}} U(x)\,, && V_{\mathrm{L}} \in \mathrm{SU}(N)_{\mathrm{L}}\,; \\ &V_{\mathrm{R}}:\ U(x) \longmapsto U(x)\,V_{\mathrm{R}}^\dagger, && V_{\mathrm{R}} \in \mathrm{SU}(N)_{\mathrm{R}}\,; \end{aligned}$$

combining them together the whole symmetry of the system $\mathrm{SU}(N)_{\mathrm{L}} \times \mathrm{SU}(N)_{\mathrm{R}}$ is attained:

$$U(x) \longmapsto V_{\mathrm{L}} U(x)\,V_{\mathrm{R}}^\dagger, \qquad \mathscr{S} \longmapsto \mathscr{S}. \tag{2.9}$$

This is precisely the chiral symmetry structure of two-flavour QCD in the massless limit, which motivates much of the physical interest in the model.

Asymptotic freedom of PCM was first shown by Polyakov [9]: in contemporary Renormalization Group language, the Callan-Symanzik beta function for SU (N) has its first two terms universal and negative [8] in weak coupling regime

$$\frac{\beta(g)}{g} = -\frac{1}{16\pi} g^2 N - \left(\frac{1}{16\pi} g^2 N\right)^2 + \dots \ . \tag{2.10}$$

Beta function $\beta(g) \coloneqq \mu \frac{\partial g}{\partial \mu}$ accounts for change of coupling constant following the renormalization scale μ evolution. Meaning of (2.10), given the negative signs, is that the coupling constant decreases as the energy scale raises: the states are less interacting the closer they are, mirroring QCD beta function behaviour.

On the lattice, PCM action is regularized as

$$\mathscr{S} = \frac{\beta}{2N} \sum_{\{x\}} \sum_{\{\mu\}} \mathrm{Re}\ \mathrm{tr}\left[U(x)\, U^\dagger(x+\hat{\mu})\right], \tag{2.11}$$

where $\beta = \frac{2N}{g^2}$ as usual. The two-point correlation function between spins at distance $|x-y| = R$ defines the correlation length ξ via

$$G(R) = \langle \mathrm{Re}\ \mathrm{tr}\left[U(x)\, U^\dagger(y)\right]\rangle \sim \mathrm{e}^{-R/\xi}, \tag{2.12}$$

that inverted yields the mass gap $m = \frac{1}{\xi}$. The bound states mass spectrum for SU (N) is analytically known [10]:

$$m_r = m \frac{\sin\left(\frac{\pi}{N} r\right)}{\sin\left(\frac{\pi}{N}\right)}, \qquad r \in [1, N-1]\,. \tag{2.13}$$

As it is already been discussed, the theory exhibits two phases, one where coupling is weak and the other where it is strong. For SU (2) group, the system does not undergo a proper phase transition, but follows a rapid but smooth crossover between the two regimes, as it is represented by the fact that beta function computed in the two cases merges smoothly at the intersection of the phases [8]. Lastly, a perhaps unnecessary but clarifying remark on why referring the $D = 2$ PCM when the content of this exploration research is about a $D = 2+1$ gauge theory: when $N_\mathrm{t} = 1$ and Dirichlet boundary conditions (2.1) are enforced on top and bottom faces, the plaquette (1.16) non-trivial components are only the vertical

links, yielding that Wilson action (1.17) and PCM one (2.11) coincide.

2.3 Lattice Observables

2.3.1 Plaquette

Plaquette (1.15) is the building block of the action, so it is related to the internal energy of the system. In fact, its average value across all lattice is obtained by

$$\langle \mathrm{tr}\,(U)\rangle = \frac{1}{K_{\mathrm{plaq.}}}\frac{\partial}{\partial\beta}\ln(Z) \tag{2.14}$$

given the trace of - products of - SU (2) matrices is always real and $K_{\mathrm{plaq.}}$ is the number of all different plaquettes constituting the lattice structure. However, since time direction is related to temperature, its lattice size is usually quite different from spatial ones: to perform studies at finite temperature it is necessary to force $N_{\mathrm{t}} \ll N_{\mathrm{s}}$. Thus, it is useful to consider separately the average "spatial" or horizontal plaquette, the one parallel to Dirichlet faces, and the "temporal" or vertical plaquette, the one perpendicular to them:

$$\langle \mathrm{tr}\,(U_{\mathrm{s}})\rangle = \frac{1}{K_{\mathrm{s}}}\sum_{\{x\}}\sum_{i<j=1}^{D}\mathrm{tr}\left[U_{ij}(x)\right], \tag{2.15}$$

$$\langle \mathrm{tr}\,(U_{\mathrm{t}})\rangle = \frac{1}{K_{\mathrm{t}}}\sum_{\{x\}}\sum_{i}^{D}\mathrm{tr}\left[U_{0i}(x)\right]. \tag{2.16}$$

In the considered setup, the normalization factors counting all the different spatial and temporal plaquettes are $K_{\mathrm{s}} = K$ and $K_{\mathrm{t}} = 2K$ respectively (K number of lattice sites). Applying Dirichlet boundary conditions clearly requires to make a further distinction between different time slices t, thus

$$\langle \mathrm{tr}\left[U_{\mathrm{s}}(t)\right]\rangle = \frac{1}{K_{\mathrm{s}}(t)}\sum_{\{\vec{x}\}}\sum_{i<j=1}^{D}\mathrm{tr}\left[U_{ij}(\vec{x},t)\right], \tag{2.17}$$

$$\langle \mathrm{tr}\left[U_{\mathrm{t}}(t)\right]\rangle = \frac{1}{K_{\mathrm{t}}(t)}\sum_{\{\vec{x}\}}\sum_{i=1}^{D}\mathrm{tr}\left[U_{0i}(\vec{x},t)\right]. \tag{2.18}$$

Note that now the sum is only on the "spatial" sites $\{\vec{x}\}$ on a given time slice t.

By identification (2.6) and the analogy between Wilson and Yang-Mills action (A.10), it is possible to connect the field strength elements to plaquettes:

$$(E_x)^2 = F_{01}F^{10} \leftrightarrow \operatorname{tr} U_{01}, \qquad (E_y)^2 = F_{02}F^{20} \leftrightarrow \operatorname{tr} U_{02}, \qquad (B)^2 = F_{12}F^{21} \leftrightarrow \operatorname{tr} U_{12}. \tag{2.19}$$

Energy density (2.7) would be expressed, in terms of lattice observables, by

$$T_{00} \propto \langle \operatorname{tr}(U_{\mathrm{s}}) \rangle - 2\langle \operatorname{tr}(U_{\mathrm{t}}) \rangle. \tag{2.20}$$

After the energy, a special interest is on the mass spectrum of the theory. To investigate it, a single observable is not enough: a correlator between two is needed. The mass of the fundamental glueball $m_{0^{++}}$ can be accessed by studying the average correlator of a spatial plaquette and the corresponding at a time distance t:

$$\langle \operatorname{tr}\left[U^{\dagger}(\vec{x}, t_0 + t)\right] \operatorname{tr}[U(\vec{x}, t_0)] \rangle \propto \mathrm{e}^{-m_{0^{++}} t}. \tag{2.21}$$

Excited states of glueballs are obtained by considering correlators of larger closed paths on lattice.

2.3.2 Wilson Loop

Plaquettes are local observables: their definition is such that their size is dependent on lattice spacing a, since they are squares with one link per side, thus they become point-like in continuum limit. It is possible to construct macroscopic observables which size is not affected by lattice spacing: the *Wilson loop*

$$W(R \times T) \coloneqq \langle \prod_{(x,\mu)\in\gamma} \operatorname{tr}[U_\mu(x)] \rangle, \tag{2.22}$$

where the notation $(x, \mu) \in \gamma$ means "for every $x \in \gamma$, select the direction μ such that also the link $U_\mu(x) \in \gamma$", being γ a rectangle on lattice of sides R and T. $R = R'/a$ and $T = T'/a$ are the length in spatial and time direction respectively, defined on the number of such links composing them R' and T': it is clear that this definition is not affected by continuum limit.

To study it, let's split γ into two vertical paths P and two horizontal paths M:

$$W(R \times T) = \langle \operatorname{tr}\left(M_{x_0 \to x_1} P_{x_1 \to x_1+T} M^{\dagger}_{x_0+T \to x_1+T} P^{\dagger}_{x_0 \to x_0+T} \right) \rangle. \tag{2.23}$$

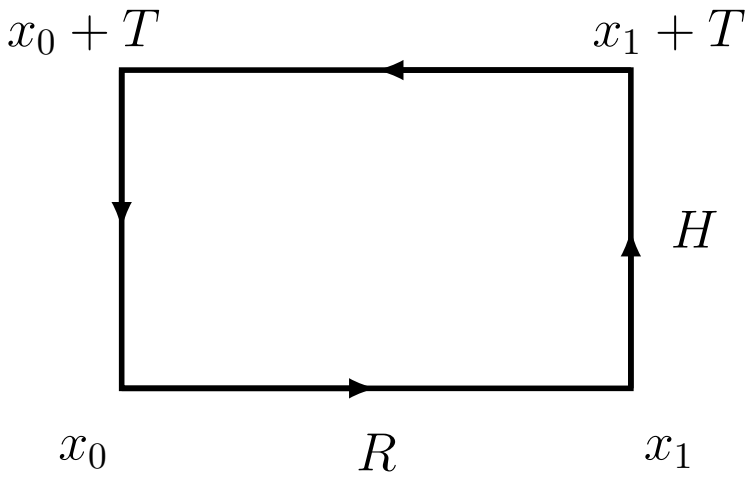


Figure 2.2: Wilson loop $R \times H$.

Vertical links composing P can be gauge transformed by $V(x) \in G$ such that

$$\begin{aligned} U_0(x) \longmapsto U_0'(x) &= V(x)\, U_0(x)\, V^{-1}\left(x + \hat{0}\right), \\ &\begin{cases} V(x) = \mathbb{1}, \\ V^{-1}\left(x + \hat{0}\right) = U_0^{-1}\left(x + \hat{0}\right). \end{cases} \end{aligned} \tag{2.24}$$

This fixes the left and right side of Wilson loop to a constant $P = \mathbb{1}$ and it can be seen as a correlator between two macroscopic objects — *Wilson lines* — in time, giving insight to its mass spectrum

$$W(R \times T) = \langle \mathrm{tr}\left(M_{x_0 \to x_1} M^{\dagger}_{x_0+T \to x_1+T}\right) \rangle \propto \mathrm{e}^{-mT}. \tag{2.25}$$

In Yang-Mills theory, Wilson loops represent the flux tube between two quenched quarks, i.e. two static quarks which mass is considered infinite, at $T = 0$. The trace on gauge index means that every possible color for quarks and gluons is considered, hence the object being studied is the sum of all possible colors for fixed quarks and their inter-potential: an approximation for a meson. However, for certain mesons, their energy at rest (their mass) is largely due just to the flux tube potential, but that is not true in general. The result depends on the distance between quarks $R = |x_1 - x_0|$, to compare it to the physical meson it is necessary to set R to its correct value, obtained from solving Schrödinger equation

at searching for minimum energy distance. This dependence permits also to study how the potential behaves when quarks are moved further from each other. The confinement hypothesis implies that it should have a linear structure:

$$V_{q\bar{q}}(R) = \sigma R + \dots \ , \tag{2.26}$$

where σ is the *string tension*, that would follow from the fact that Wilson loop value would be given by two contributions, one from the area and one from the perimeter

$$W(R \times T) = \mathrm{e}^{-\sigma RT - P(R+T) - c}. \tag{2.27}$$

This relation arises from strong coupling expansion, the weighted sum of all possible surfaces possible to construct with plaquettes that connect the perimeter of Wilson loop, but this technique is not able to describe every region in the confined phase [11]. The flux tube is affected by quantum fluctuations, that are initially accounted for by higher order terms in strong coupling expansion of the Wilson loop. At a critical value, the surface undergoes the so-called *roughening transition* [12, 13], at which the strong coupling series ceases to converge and the correction it gives to area law do not describe flux tube configurations properly any more. In the rough regime, the inter-quark potential is well explained by an effective string theory, which accounts for the more elaborate Wilson loop surface arrangements. A proper treatment of this matter is beyond the scope of this work, but it is worth mentioning the difference between such two regimes to prevent misunderstandings.

2.3.3 Polyakov Loop

When working in a finite temperature setup on the lattice with compact euclidean time, a new symmetry arises: taking a certain time coordinate t_0 and considering all vertical upward links $\{U_0(\vec{x}, t_0)\}$, Wilson action is invariant under the transformation

$$U_0(\vec{x}, t_0) \longmapsto U_0'(\vec{x}, t_0) = Z U_0(\vec{x}, t_0), \tag{2.28}$$

where Z is an element of the centre of the gauge group G. The centre of any group is the subgroup which elements commute with any element of the original group G. Center group of $\mathrm{SU}(N)$ is $\mathbb{Z}_N$, so for the case of interest $\mathrm{SU}(2)$, its centre is $\mathbb{Z}_2$. It is easy to see that plaquette expression (1.15), cornerstone of Wilson action, does not change after performing (2.28), since Z and Z^{-1} commute with all links, so they cancel each other. This geometry

also allows to build a new non local observable, the *Polyakov loop*

$$P(\vec{x}) = \mathrm{tr}\left[\prod_{t=0}^{N_t} U_0(\vec{x}, t)\right], \tag{2.29}$$

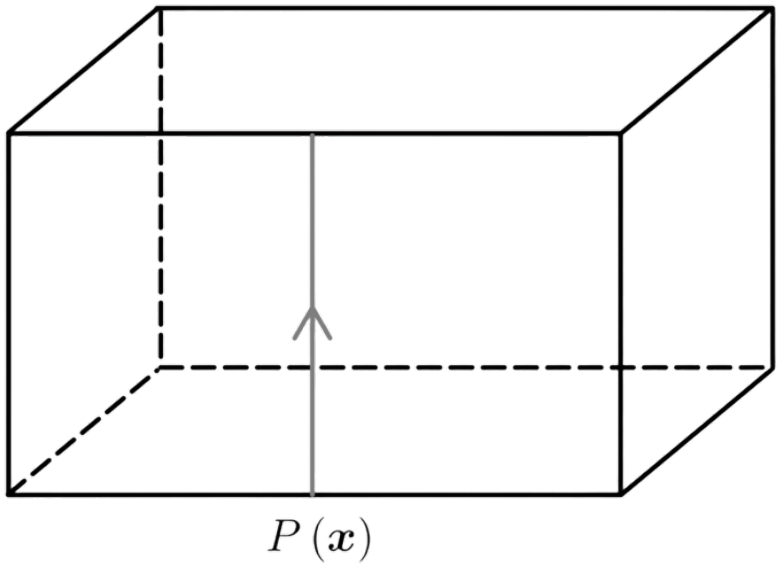


Figure 2.3: Polyakov loop.

that is a closed vertical straight path across all lattice and is the order parameter for the breaking of previously discussed symmetry. In fact, Polyakov loop is closed but has no adjoint part to compensate for the centre element $P(\vec{x}) \mapsto ZP(\vec{x})$, so it is not invariant under this transformation. If anything, its value marks whether the symmetry is broken or not: when the Polyakov loop vanishes, the symmetry is preserved, while when its expectation value is finite, it signals the transition to a new phase where the symmetry is spontaneously broken. It is possible to associate the free energy of a static quark to the observable

$$\langle P(\vec{x}) \rangle = \mathrm{e}^{-N_t F_q}, \tag{2.30}$$

to see the connection with deconfinement transition of quarks inside hadrons: when the expected value of Polyakov loop is zero, the free energy is infinite and the quark is confined inside the hadron, while when it becomes non-zero, the free energy has a finite value and the quark exists outside the particle. By shortening N_t, the temperature raises and it causes the phase transition, hence it exists a critical value $N_t^{(\mathrm{C})}$ that separates the two phases. It

is common to investigate it with the opposite procedure: fixing N_t and varying slowly β to find the law for $\beta_c(N_t)$, then reversing it to get $N_t^{(c)}(\beta)$. In this way, also the critical temperature is determined by

$$T_c(\beta) = \frac{1}{N_t^{(c)}(\beta)}. \tag{2.31}$$

If a single Polyakov loop is not invariant, considering a correlator of two Polyakov loops at distance R in opposite directions

$$G(R) \coloneqq \langle P(\vec{x}) P^\dagger(\vec{x} + R\hat{i}) \rangle \tag{2.32}$$

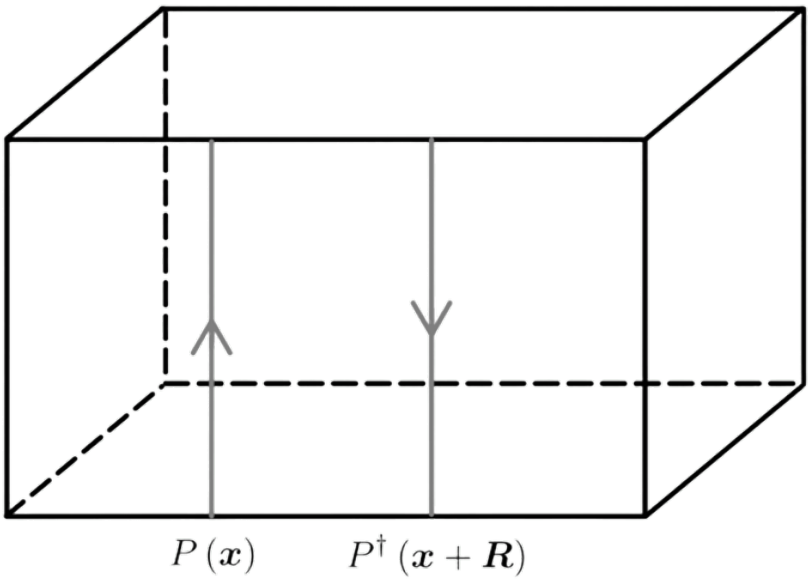


Figure 2.4: Polyakov loop correlator.

yields an object which obeys $\mathbb{Z}_N$ symmetry, since $ZZ^\dagger = \mathbb{1}$. Of course, since a single Polyakov loop gives information about a single quark status, the correlator of two opposite loops for SU(2) is describing the potential between them in a meson, so it is again useful to associate the free energy to it:

$$G(R) = \mathrm{e}^{-N_t F_{q\bar{q}}}. \tag{2.33}$$

In fact, the correlator of two Polyakov loops can be easily compared to a Wilson loop of spatial size R spanning all lattice height: $G(R) = W(R \times N_t)$. Thus, in the strong

coupling regime it is possible to recover the structure for $F_{q\bar{q}}$ from area law, resulting in

$$G(R) = \mathrm{e}^{-\sigma R N_t + \cdots}. \tag{2.34}$$

This quantity is the main tool to study string tension σ. Effective string corrections provide a dependency on temperature $\sigma(T)$, hence the string value obtained via the area law relation is said to be zero-temperature string tension $\sigma = \sigma(T = 0)$.

Chapter 3

Monte Carlo Methods

3.1 Monte Carlo Basics

To perform this research, numerical simulations using Monte Carlo algorithms have been performed. The reason is quite simple: to extract observables' expectation value by means of deterministic numerical integration, the number of operations a computer should complete would be astonishingly large and would also grow exponentially with lattice volume. Thus it is convenient to exploit statistical methods, such as Monte Carlo ones, to investigate lattice systems in a reasonable amount of time.

First of all, a Monte Carlo algorithm goal is to estimate the expectation value of a variable,

$$\langle O \rangle_p \coloneqq \int_a^b O(x)\, p(x)\, \mathrm{d}x\,, \tag{3.1}$$

following a probability density function (pdf) $p(x)$ on the set $[a, b]$, such that

$$\int_a^b p(x)\, \mathrm{d}x = 1, \qquad p(x) \geq 0 \quad \forall x \in [a, b]\,. \tag{3.2}$$

To do so, Monte Carlo algorithms generate a *sample* ω from the probability space Ω (in this case $[a, b]$) and evaluate $O(\omega)$. Producing more and more samples yields better accuracy on the estimated value, as the variance of the mean of n independent and identically distributed samples relates to the variance of a single one as $\sigma_n^2 = \sigma_1^2/n$.

However, sampling naively, i.e. uniformly, $x = \omega$ values on all domain $[a, b]$ via a pseudo-random number generator and feeding it to the pdf is often very inefficient, since the error scales with the square root of n. In physical systems, such as statistical mechanics

descriptions of many bodies and QFTs, $p(x)$ is rather peaked in the region around the preferred configuration and extremely small otherwise, so considering all domain uniformly would yield for the most configurations with no physical interest. $p(x)$ in physical cases is the Boltzmann weight coming from the exponential of the action $\mathrm{e}^{-\mathscr{S}}$ [14]. Hence the introduction of *importance sampling*, where points are sampled following the very pdf it is being considered to highlight relevance of physical behaviour and to reduce simulation time. Usually, the solution is to rewrite the desired quantity as follows:

$$\langle O \rangle_p = \int_a^b O(x) \frac{p(x)}{q(x)} q(x) \, \mathrm{d}x \,, \tag{3.3}$$

ensuring that everywhere $q(x) = 0$ also $f(x) p(x)$ is null, or alternatively

$$\langle O \rangle_p = \langle O \frac{p}{q} \rangle_q. \tag{3.4}$$

In this context, $p(x)$ is the *target distribution*, $q(x)$ the *importance distribution* and $\frac{p(x)}{q(x)}$ the *likelihood*. The goal is to maximize the likelihood, thus a good choice for the importance distribution is it being proportional to $|O(x)|p(x)$ in such a way that $q(x)$ overlaps $p(x)$ as much as possible. By doing so, variance of the estimate sampling on $q(x)$ is reduced with respect to sampling uniformly.

The simplest way to conduce importance sampling is to perform a change of variable, defining

$$y \coloneqq \int_a^x p(z) \, \mathrm{d}z \tag{3.5}$$

and inserting it into the expectation value (3.1), that becomes

$$\langle O \rangle = \int_0^1 O(y) \, \mathrm{d}y \,. \tag{3.6}$$

Indeed, when $y = 1$, normalization of $p(x)$ enforces that $x = b$. $O(y)$ is distributed uniformly in the interval $[0, 1]$ for y hence it is easy to sample, the problem is moved to connect it with the original pdf.

Another way is the accept/reject method by von Neumann [15], where the importance distribution is $c \cdot q(x) \geq p(x)$, $c \geq 1$. A sample ω is drawn according to $q(x)$ and is

accepted with probability $\frac{p(\omega)}{cq(\omega)}$. The average acceptance probability is

$$\langle P_{\text{acc}} \rangle_q = \int q(\omega) \frac{p(\omega)}{cq(\omega)} \, \mathrm{d}\omega = \frac{1}{c}, \tag{3.7}$$

that confirms that the accepted values follow the target distribution:

$$\frac{P_{\text{acc}}(x) q(x)}{\langle P_{\text{acc}} \rangle_q} = \frac{\frac{p(x)}{cq(x)} q(x)}{\frac{1}{c}} = p(x). \tag{3.8}$$

Since the average acceptance probability is $1/c$, the difficulty of this method lies in finding a value of c big enough to ensure that $c \cdot q(x) \geq p(x)$, but such that the acceptance rate is not too small, unless the algorithm would need too many iterations to accept a sample.

A more efficient way of simulating a physical system using Monte Carlo algorithms is by combining it with stochastic processes, more precisely Markov chains, which are more suited to model evolution from an initial configuration to an equilibrium one.

3.2 Markov Chains

A stochastic process is a collection of random variables $\{x_t\}$ with some index t that is often interpreted as time that are elements of the state space Ω, which in this case is assumed finite. The pdf is replaced by a set of conditional probabilities $\{p_n(x_n \mid x_1, x_2, \ldots x_{n-1})\}$, so the total pdf can be written as

$$p(\{x_t\}) = \prod_{t=1}^{n} p_t(x_t \mid x_1, x_2, \ldots x_{t-1}). \tag{3.9}$$

In this context $\{x_t\}$ are not independent random variables, on the contrary they are related to each other via the "history" of the process. They are, nevertheless, identically distributed, a property that makes a stochastic process *stationary*; that is safe enough to assume, as it would not make sense for the physics of the lattice under study to be dependent on a computational parameter.

Markov chains are stochastic processes where the probability of passing from the state $x_t = a$ to $x_{t+1} = b$ depends only on a, b and t, i.e.

$$p_{t+1}(x_{t+1} \mid x_1, x_2, \ldots x_t) = p_{t+1}(x_{t+1} \mid x_t), \qquad \forall\ t. \tag{3.10}$$

For a stationary Markov chain the probability of transitioning from state a to state b is labelled $W_{ba} = p\left(x_{t+1} = b \mid x_t = a\right)$ and is an element of the stochastic matrix W, namely a matrix defined by two properties:

1. $W_{ba} \geq 0$ for any $a, b \in \Omega$;
2. $\sum_{\{a \in \Omega\}} W_{ba} = 1$ for any $b \in \Omega$.

Last property means that every step $x_t = a$ will land in another state of Ω at x_{t+1}. The probability of passing from state a to state b in k Markov chain steps is labelled $\left(W^k\right)_{ba}$ and it is also a stochastic matrix.

A Markov chain is said *irreducible* if there exists a finite probability of reaching a final state b starting from any initial state a after a finite number of steps k, i.e. $\left(W^k\right)_{ba} > 0$ for any $a, b \in \Omega$. When represented by a graph, a Markov chain has states in vertices and two vertices are connected by an oriented edge from a to b if $W_{ba} > 0$. The Markov chain is irreducible if, starting from any given vertex of the representing graph, it is possible to reach any vertex (included the starting one) by travelling along the graph following the oriented edges, as depicted in Figure 3.1. If $W_{ba} > 0$ for any $a, b \in \Omega$, then of course the Markov chain is irreducible.

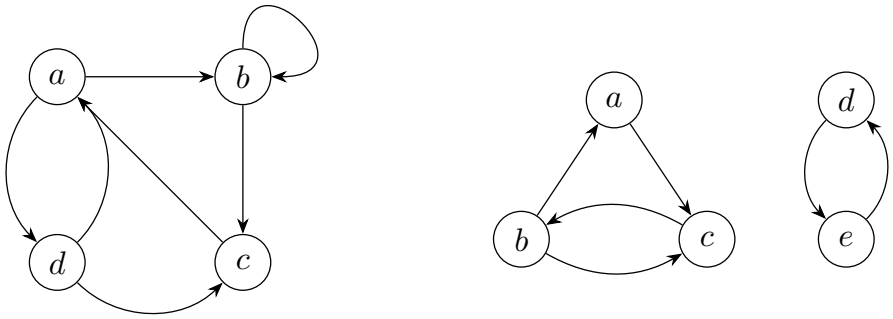


Figure 3.1: Example of an irreducible Markov chain (left) and reducible Markov chain (right).

A useful quantity to define for Markov chains towards the upcoming discussion for Monte Carlo simulations is the *period* of their states. A more detailed treatment is available in section A.2, but for the goal of this work it is sufficient to view the period T_a of any state $a \in \Omega$ as the smallest value of k for which it is possible that, if $x_t = a$, then $x_{t+k} = a$. If all the states have period $T = 1$, the Markov chain is said to be *aperiodic* and it is sufficient

that $W_{aa} > 0$ for any possible $a \in \Omega$ to ensure it, i.e. every vertex of the graph has an outwards edge that loops and return to the same vertex.

Studying a Markov chain, it is interesting to ask what is the probability it will be in a chosen state b at step t. Since the evolution of a Markov chain has no memory, namely the final state b depends only on the previous one a and the stochastic matrix W as stated in (3.10), then it summarizes in writing the evolution equation

$$p\left(x_t = b\right) = \sum_{\{a \in \Omega\}} W_{ba} p\left(x_{t-1} = a\right). \tag{3.11}$$

Considering Markov chains with a great number of steps, it is useful to determine what happens in the $t \to \infty$ limit, more precisely if it exists a pdf π_a to which the probability of being in state a tends towards in said limit

$$\pi_a = \lim_{t \to \infty} p\left(x_t = a\right). \tag{3.12}$$

If so, it is called *stationary* distribution of the Markov chain, and the reason is easy to see: taking the infinite time limit of (3.11) it yields

$$\pi_b = \sum_{\{a \in \Omega\}} W_{ba} \pi_a, \tag{3.13}$$

so π_b is an eigenvector of W with eigenvalue $\lambda_b = 1$.

It is possible to show that for an irreducible and aperiodic Markov chain there exists a single stationary distribution $\pi = \{\pi_a\}$, and that $\pi_a > 0$ for any $a \in \Omega$. If the stochastic matrix W is also diagonalizable, the probability for the Markov chain to be in state a at time t, i.e. $\left(W^t p\right)_a$, converges exponentially to the corresponding element of invariant distribution

$$\sum_{\{a \in \Omega\}} \left| \left(W^t p\right)_a - \pi_a \right| \leq A \mathrm{e}^{-\frac{t}{\tau}}, \tag{3.14}$$

where τ is called *autocorrelation time*. This fact has to be kept in mind when building a sampling algorithm based on Markov chains.

3.3 Markov Chain Monte Carlo

Given what has been discussed in the previous section, starting from the initial state b (sampled from a deterministic pdf) of an irreducible and aperiodic Markov chain, it will

evolve following W and, asymptotically, states in Ω will be visited following the invariant distribution π. Thus the goal is to construct a Markov chain algorithm that will converge to the desired π, the distribution describing the physical evolution of the system being considered, and sampling data after when it is reasonable to assume the Markov chain has reached it, since they will follow π. This technique is called Markov Chain Monte Carlo (MCMC). Needless to say, this means that samples from the Markov chain are not independent, rather they are related through the stochastic matrix W, which is what has to be determined to obtain the stationary distribution.

Starting from the stationary property (3.13) and using normalization of stochastic matrix on the left hand side, it becomes

$$\sum_{\{a\in\Omega\}} W_{ab}\pi_b = \sum_{\{a\in\Omega\}} W_{ba}\pi_a, \tag{3.15}$$

then by subtracting $W_{bb}\pi_b$ on both sides the balance equation becomes

$$\sum_{\{a\neq b\}} W_{ab}\pi_b = \sum_{\{a\neq b\}} W_{ba}\pi_a. \tag{3.16}$$

The left hand side represents the asymptotic probability of leaving b, while the right hand side the probability of landing onto b from another state and they are equal, hence the "balance" label. It is the necessary condition for π to satisfy for being the asymptotic distribution associated to the Markov chain encoded in W. Since a $N \times N$ stochastic matrix W has to be normalized along all its columns, it contains $N^2 - N$ independent elements, but the balance equation adds only N more constraints. It is not enough to completely determine W without ambiguity, hence the purpose of a stricter requirement: the *detailed balance* equation

$$W_{ab}\pi_b = W_{ba}\pi_a, \qquad \text{for all } a, b \in \Omega. \tag{3.17}$$

This condition also ensures that W is diagonalizable, given that it is also irreducible.

There are a lot of different MCMC algorithm, such as Metropolis-Hasting algorithm, Langevin algorithm, Hybrid Monte Carlo, etc. For the scope of this work, it is enough to discuss the *heat-bath* algorithm, also called *Gibbs sampler*, that can be used when states in Ω are sets of independent numbers that describe the target system, for instance positions and momenta of classical objects. Labelling the state of the system at any time with the i-th number a and the set α, the set of all other numbers required to specify the state,

$x_t = (a, \alpha)$, the conditional probability for the i-th number to be exactly a when the rest of the system is denoted by α is, of course,

$$p\left(a \mid \alpha\right) = \frac{\pi\left(a, \alpha\right)}{\sum\limits_{\{a'\}} \pi\left(a', \alpha\right)}. \tag{3.18}$$

A single step of the heat-bath algorithm consists in generating a new state by modifying a single number at a time, so the stochastic matrix between two consecutive states can be written as

$$W_{(b,\beta)(a,\alpha)} = p\left(b \mid a\right) \delta_{\beta\alpha}, \tag{3.19}$$

so the rest of the state α is considered fixed at each update, thus the "heat-bath" naming. It is easy to see that this form for the stochastic matrix satisfies the detailed balance equation. Since the update has to take place on all numbers of the state, what is commonly referred to as "update step" is the complete update of all numbers in the set. This algorithm does not have a accept/reject part, like the von Neumann method has, so the update takes place at every step.

Of course, the states considered when studying a lattice QFT are the gauge fields configurations $\{U_\mu(x)\}$ and the stationary distribution of interest in this work is the one of normalized Boltzmann weights of lattice action $\pi[U] = Z^{-1} \exp\left(\mathscr{S}_{-\mathrm{w}}[U]\right)$. To explore the configuration space more quickly, it is opportune to implement also a method called *overrelaxation*. It is suited for problems in which the probability distribution can be represented with Boltzmann weights and the component of the action can be interpreted as elements of a vector space. If so, the energy associated to one of such elements $\vec{U}$ can be written as $E\left(\vec{U}\right) = -J\vec{U} \cdot \vec{S}$, where $\vec{S}$ is the vector that comprehends all other elements of the configuration that are interacting with $\vec{U}$ and, thus, are the ones that influence the value of the update $\vec{U}'$ through heat-bath procedure. By decomposing the element into its parallel and orthogonal components with respect to $\vec{S}$, i.e.

$$\vec{U}_\parallel = \frac{\vec{U} \cdot \vec{S}}{|\vec{S}|^2} \vec{S}, \qquad \vec{U}_\perp = \vec{U} - \frac{\vec{U} \cdot \vec{S}}{|\vec{S}|^2} \vec{S}, \tag{3.20}$$

it is easier to see how an update $\vec{U}'$ should be to explore further regions of the configuration space: heat-bath tends to minimize the angle between $\vec{U}$ and $\vec{S}$, while overrelaxation reflects $\vec{U}$ with respect to $\vec{S}$ by changing the sign of the transverse component $\vec{U}_\perp \mapsto \vec{U}'_\perp = -\vec{U}_\perp$ with a certain probability. After this kind of transformation, energy of the system is left

unchanged but the update has a bigger probability to visit different configurations that are equivalent to the previous one.

3.4 MCMC Data Analysis

If $O : \Omega \to \mathbb{R}$ is an observable of the physical system which value is finite, its expected value can be estimated through Monte Carlo simulations by rephrasing (3.1) as

$$\langle O \rangle_\pi = \sum_{\{a \in \Omega\}} O_a \pi_a. \tag{3.21}$$

The way to do so is to build the Markov chain which invariant distribution is π and let it update states for a certain period, then taking the average value of such observable on the states $\{x_i\}$ visited by the MCMC during its evolution [3]

$$\overline{O} = \frac{1}{n} \sum_{i=1}^{n} O\left(x_i\right), \tag{3.22}$$

since, in the $n \gg \tau$ regime, it would mean to sample from the stationary distribution.

To see this method is suitable for reproducing desired dynamics, it is convenient to define the average of the possible samplings of a Markov chain

$$\langle O \rangle_s \coloneqq \sum_{\{a \in \Omega\}} O_a \left(Wp\right)_a. \tag{3.23}$$

The difference between the average of the outcomes of n samplings $\langle \overline{O} \rangle_s$ and the expected value $\langle O \rangle_\pi$ is

$$\left| \langle \overline{O} \rangle_s - \langle O \rangle_\pi \right| = \left| \frac{1}{n} \sum_{i=1}^{n} \sum_{\{a \in \Omega\}} O_a \left(W^i p\right)_a - \sum_{\{a \in \Omega\}} O_a \pi_a \right|, \tag{3.24}$$

but decomposing the probability of reaching state a after i updates as the difference between asymptotic probability of being in such state and the π_a itself $\left(W^i p\right)_a = R_a^{(i)} + \pi_a$, it becomes

$$\left| \langle \overline{O} \rangle_s - \langle O \rangle_\pi \right| = \left| \frac{1}{n} \sum_{i=1}^{n} \sum_{\{a \in \Omega\}} O_a R_a^{(i)} \right|. \tag{3.25}$$

By triangle inequality it is possible to write the relation

$$\left|\langle\overline{O}\rangle_s - \langle O\rangle_\pi\right| \leq \frac{1}{n}\sum_{i=1}^{n}\sum_{\{a\in\Omega\}} |O_a|\left|R_a^{(i)}\right|, \tag{3.26}$$

then defining the greatest magnitude that the observable can reach $M \coloneqq \max_{\{a\in\Omega\}} |O_a|$ and exploiting (3.14) the inequality becomes

$$\left|\langle\overline{O}\rangle_s - \langle O\rangle_\pi\right| \leq A\frac{M}{n}\sum_{i=1}^{n} \mathrm{e}^{-\frac{i}{\tau}}. \tag{3.27}$$

It may be unnecessary to specify, but the i in the exponential does not represent the imaginary unit[1], but the step index of the sum. Finally, the sum of a finite number of non-negative terms is always less or equal to the analogous infinite sum

$$\left|\langle\overline{O}\rangle_s - \langle O\rangle_\pi\right| \leq A\frac{M}{n}\sum_{i=1}^{\infty} \mathrm{e}^{-\frac{i}{\tau}} = A\frac{M}{n}\frac{\mathrm{e}^{-\frac{1}{\tau}}}{1-\mathrm{e}^{-\frac{1}{\tau}}}, \tag{3.28}$$

therefore the average outcome of a MCMC algorithm differs from the physical expected value by a term proportional to $\frac{1}{n}$. In $n \to \infty$ limit it is negligible with respect to the standard Monte Carlo error $\sim \frac{1}{\sqrt{n}}$, however for practical applications it is useful to implement expedients to minimize it. The most straightforward is to discard the initial steps of the Markov chain until a certain threshold called *thermalization* n_{therm} is reached, in analogy with the language referred to heat-bath algorithm. It is the number of steps after which the updates bring the system from its initial configuration to the one following the physically relevant problem, so it is a way to sample states that are closer to the stationary distribution. By doing so, (3.28) becomes

$$\left|\langle\overline{O}\rangle_s - \langle O\rangle_\pi\right| \leq \frac{AM}{n-n_{\mathrm{th}}}\sum_{i=n_{\mathrm{th}}+1}^{\infty} \mathrm{e}^{-\frac{i}{\tau}} = \frac{AM}{n-n_{\mathrm{th}}}\frac{\mathrm{e}^{-\frac{n_{\mathrm{th}}+1}{\tau}}}{1-\mathrm{e}^{-\frac{1}{\tau}}}, \tag{3.29}$$

hence it decreases by a factor $\frac{n}{n-n_{\mathrm{th}}}\mathrm{e}^{-\frac{n_{\mathrm{th}}}{\tau}}$.

As previously stated, samples coming from different steps of the Markov chain are related to each other via the stochastic matrix W. Thus their values are correlated to each other and the standard deviation of the average measure on sampled states (3.22) is an

[1]Imaginary unit "i" and Euler's number "e" are typed with Roman font style in this work.

underestimation of the actual error. Calling $\delta O(x_i) \coloneqq O(x_i) - \langle O \rangle_\pi$ the difference between each value on single sample and the expected value, the autocorrelation function between samples is

$$A(x_i, x_j) = \frac{\langle \delta O(x_i)\, \delta O(x_j) \rangle_s}{\sigma^2(O)}. \tag{3.30}$$

From the definition of variance of the averaged observable on the samples, it is possible to show that it is related to the "true" variance of the observable via the relation

$$\sigma^2\left(\overline{O}\right) = \frac{\sigma^2(O)}{N^2} \sum_{i=1}^{n} A(x_i, x_j). \tag{3.31}$$

For samples x_i that are in the stationary region, it is reasonable to assume that the autocorrelation function is bounded, $A(x_i, x_j) \in [-1, +1]$, therefore it is clear from the previous equation that $\sigma^2(O) \geq \sigma^2\left(\overline{O}\right)$.

The practical method to compensate for this systematic error is a procedure called *binning* or *blocking*. It starts by constructing sets of k samples $\{\hat{x}_i^{(k)}\}$, where now i is an index running on the possibles $\lfloor n/k \rfloor$ groups of samples (if k does not divide n, the first $n - k\lfloor n/k \rfloor$ are left out from the procedure). Then the value of the observable on the sets is defined as the mean of the values on individual samples

$$O(\hat{x}_i^{(k)}) = \frac{1}{k} \sum_{j=1}^{k} O(x_{i+j}). \tag{3.32}$$

The average of observables on sets is equivalent to the average on single samples, since

$$\frac{k}{n} \sum_{i=1}^{N/k} O(\hat{x}_i^{(k)}) = \frac{k}{n} \sum_{i=1}^{N/k} \frac{1}{k} \sum_{j=1}^{k} O(x_{i+j}) = \frac{1}{n} \sum_{i=1}^{n} O(x_i), \tag{3.33}$$

but the point is to find the value $\tilde{k}$ for which $O(\hat{x}_i^{(\tilde{k})})$ and $O(\hat{x}_{i+1}^{(\tilde{k})})$ are not autocorrelated and estimate the variance on these blocks. To do so, a blocking algorithm increases k and computes the variance $\sigma^2[O(\hat{x}^{(k)})]$ until a plateau is reached and its value is taken as the variance of the observable average. It is a method that needs $k \ll n$ to yield results that are not affected by statistical fluctuations.

Another effect to consider is that, often, observables are correlated: for example, a Wilson loop of spatial size R shares a portion of its links with another Wilson loop of size R' with the starting vertex in the same lattice site as the first one. Hence a correct error

estimation should comprehend the correlation between m observables, so by writing

$$\Sigma(O^{(i)}, O^{(j)}) = \frac{\langle \delta O^{(i)} \delta O^{(j)} \rangle_s}{\sigma(O^{(i)})\sigma(O^{(j)})}, \qquad i, j \in \{1, 2, \dots m\}, \tag{3.34}$$

a normalized covariance matrix is obtained. It is useful to compute the normalized version since the magnitude of the error has to be determined through blocking procedure, that if repeated for every realization of the observable produces a vector

$$\vec{\epsilon} = \left(\sigma^2(O^{(1)}), \sigma^2(O^{(2)}), \dots \sigma^2(O^{(m)})\right). \tag{3.35}$$

Combining the two techniques yields the covariance matrix with proper error estimation

$$C = (\vec{\epsilon})^{\mathrm{T}} \Sigma \vec{\epsilon}. \tag{3.36}$$

Chapter 4

Results

4.1 Lattice Setup

To perform numerical simulations of the system under consideration, the starting point was Claudio Bonati's C code for Yang-Mills theories (`https://github.com/claudio-bonati/yang-mills`). It is a Monte Carlo Markov Chain algorithm with heat-bath update procedure. The program computes the *staples* $S_\mu(x)$ departing from every link $U_\mu(x)$, defined as

$$S_\mu(x) \coloneqq \sum_{\nu \neq \mu} U_\nu(x+\hat{\mu})\, U_\mu^\dagger(x+\hat{\nu})\, U_\nu^\dagger(x)\,, \tag{4.1}$$

and uses its value to update the link, sampling from the probability distribution [16]

$$p(U_\mu) = \frac{1}{\sqrt{\det(U_\mu)}} \exp\left[-\frac{1}{N}\operatorname{tr}(\beta U_\mu S_\mu)\right]. \tag{4.2}$$

The code distinguishes between even and odd sites, i.e. sites which sum of "cartesian coordinates" (site number in each spacetime direction) is even or odd, and updates all even sites before odd ones. The reason is to prevent an update of a site using a "backward" staple that has already been updated and a "forward" staple that has not. After the update, overrelaxation is performed to limit autocorrelation with the same procedure of working on all even sites before odd ones.

The code was modified to implement Dirichlet boundary conditions (2.1), since the default code architecture implies periodic boundary conditions. This would make the D-dimensional lattice a D-torus, with equal number of lattice sites and links in both spatial (N_s) and time direction (N_t). Spatial directions has been left unchanged, while it was

chosen to fix values on extremal time faces. Since periodic boundary conditions mean that $t = 0$ and $t = N_\mathrm{t}$ planes coincide, Dirichlet boundary conditions have been enforced as

$$U_\mu(\vec{x}, t = 0) = \mathbb{1} = U_\mu(\vec{x}, t = N_\mathrm{t} - 1), \qquad \mu \in \{1, 2\}, \tag{4.3}$$

such that on top of every Dirichlet lattice lives a PCM with sizes identical to lattice spatial sizes, in the slice between the two Dirichlet faces. Links laying on those faces are excluded from the update. From now on, when referring to the lattice with Dirichlet boundary conditions on time, N_t will mean the number of vertical links between such two faces: by simulating a lattice with four vertical points, $N_\mathrm{t} = 3$ and the last vertical link will be a spin of PCM. The code is available at `https://github.com/marco-nebbia/yang-mills-leonardo`.

Numerical simulations have been performed on CINECA's EuroHPC supercomputer LEONARDO. Every simulation was repeated in parallel 16 times or "runs" with a different random seed for the initial gauge configuration, each with n_sample Monte Carlo steps. For each simulation, data was analysed discarding the first n_th samples to analyse only thermalized lattice, i.e. values after the Markov chain brought the system from the initial pseudo-random configuration to physically interesting ones. Unless differently specified, $n_\mathrm{th} = 500$. To extract measures and errors from statistical fluctuations around equilibrium values, blocking procedure and correlation matrix are used. The depiction of the process showed in Quality Check subsection will be exemplary to all other data analysis cases.

goal of simulation	n_runs	β	n_sample	N_t	$N_\mathrm{s1} \times N_\mathrm{s2}$
quality check	16	12.15266	16000	160	9×160
varying N_t			8000	2-12	64×64
Polyakov loop check			8000	2	64×64
Polyakov correlator study			80000	8	64×64

Table 4.1: Simulation parameters.

4.2 Quality Check

As previously stated, Dirichlet boundary conditions effects should be indistinguishable from periodic ones for observables measured in the bulk of the lattice when $N_\mathrm{t} \to \infty$. Thus, it is expected that results in the bulk with this setup would match ones obtained with periodic boundary conditions for a sufficiently large N_t. To check it, the value of the Polyakov loop correlator $G(R)$ with $R \in [6, 47]$ obtained in reference [17] for a lattice with $N_\mathrm{t} = 9$,

$N_\mathrm{s} = 160$, $\beta = 12.15266$ and periodic boundary conditions was taken as a benchmark. Since the goal is to have a great separation between the Dirichlet faces, the "original" lattice has been rotated by 90°, such that $N_\mathrm{t} = 160$, $N_{\mathrm{s}1} = 9$, $N_{\mathrm{s}2} = 160$. The equivalent to original Polyakov loop correlator $G(R)$ is the correlator between two horizontal Polyakov loops defined on direction x_1 (the one that has the same size of time size in the original lattice)

$$P_1(x_2, t) \coloneqq \mathrm{tr}\left[\prod_{x_1=0}^{N_{\mathrm{s}1}} U_1(x_1, x_2, t)\right], \tag{4.4}$$

in the bulk of the lattice and far from Dirichlet faces, namely

$$G^{(\mathrm{h,b})}(R) \coloneqq G_{\mathrm{horiz.}}(R)|_{\mathrm{bulk}} = \langle P_1(x_2, t = l)\, P_1^\dagger(x_2, t = l + R)\rangle, \tag{4.5}$$

where $l = \frac{N_\mathrm{t} - R}{2}$ and the average is on all x_2 values. To preserve symmetry between correlator and both top and bottom Dirichlet faces, only even values of R have been considered.

The remaining part of Monte Carlo histories after thermalization from all runs are appended as if they were sections of the same history, to analyse them together. During Markov Chain evolution, the system fluctuates around the equilibrium value, which is computed as the mean of $G^{(\mathrm{h,b})}(R)$ across all Monte Carlo steps and initially the error is taken as the standard deviation of this set of values. This would be a great underestimation for the error value, so blocking procedure with blocks of size $k = 10, 20, 30, \ldots 16000$ is performed on histories for every R value to minimize self-correlation between step values, some examples are in Figure 4.1. Note that appending all runs into one "history" allows a greater maximum value for k. Error value is chosen to be the average value of blocking errors between $k = 4000$ and $k = 6000$ and errors for all R values are stored in a column vector $\vec{\epsilon}$.

To take into account cross-correlation between different values of R, normalized covariance matrix $\Sigma\left(G^{(\mathrm{h,b})}\right)$ is built, which entries are the Pearson correlation coefficients

$$\rho(R, R') = \frac{\mathrm{Cov}\left(G^{(\mathrm{h,b})}(R), G^{(\mathrm{h,b})}(R')\right)}{\sigma_R \sigma_{R'}}, \tag{4.6}$$

where σ_R is a shorthand for the standard deviation of all measures for the horizontal Polyakov loop correlator at distance R. It is computed also for each run history, to be able to compare them to the average $\Sigma\left(G^{(\mathrm{h,b})}\right)$ obtained considering all runs together. Two examples of covariance matrices for single runs are shown in Figure 4.2 and compared to the final matrix. It is clear that matrices for both runs are slightly different, but really

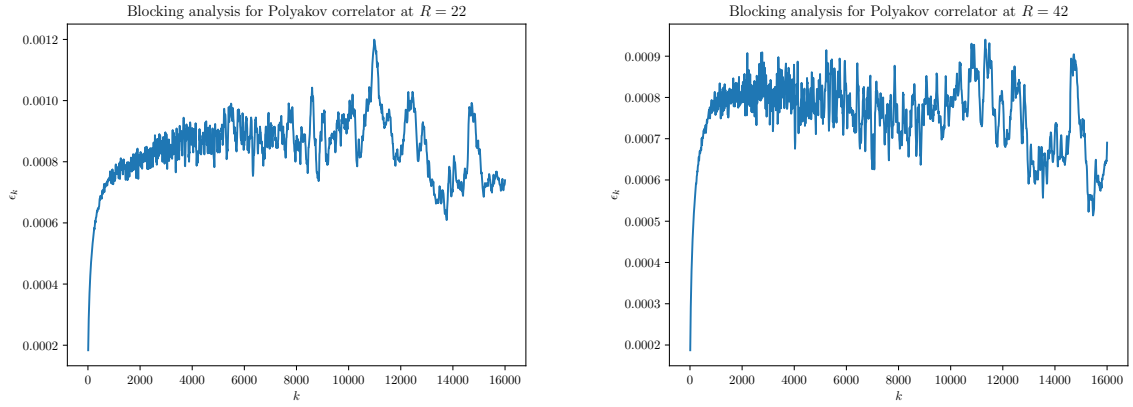


Figure 4.1: Error value evolution during blocking for $R = 22$ (left) and $R = 42$ (right).

similar to each other and to the average covariance matrix.

To combine both procedures it is sufficient to define the covariance matrix after blocking as

$$C = (\vec{\epsilon})^{\mathrm{T}} \, \Sigma \, \vec{\epsilon}, \tag{4.7}$$

such that the diagonal of C is the array of variances obtained after blocking. A fit of $G^{(\mathrm{h,b})}(R)$ values has been performed with function

$$G^{(\mathrm{h,b})}(R) = k \left[K_0 \left(\frac{R}{\xi} \right) + K_0 \left(\frac{N_{\mathrm{t}} - R}{\xi} \right) \right], \tag{4.8}$$

to consider symmetric effects, with $K_0(x)$ modified Bessel function of the second kind that admits the expansion

$$K_0(x) \simeq \sqrt{\frac{\pi}{2x}} \mathrm{e}^{-x} \left[1 + \mathcal{O}\left(\frac{1}{x} \right) \right]. \tag{4.9}$$

Fit results are presented in Table 4.2, in agreement with ones found in the reference, and it is plotted on the data points in Figure 4.3.

R_{min}	R_{max}	k	ξ	χ^2_{red}
6	48	0.041(1)	22.54(87)	0.166

Table 4.2: Fit results of $G^{(\mathrm{h,b})}(R)$.

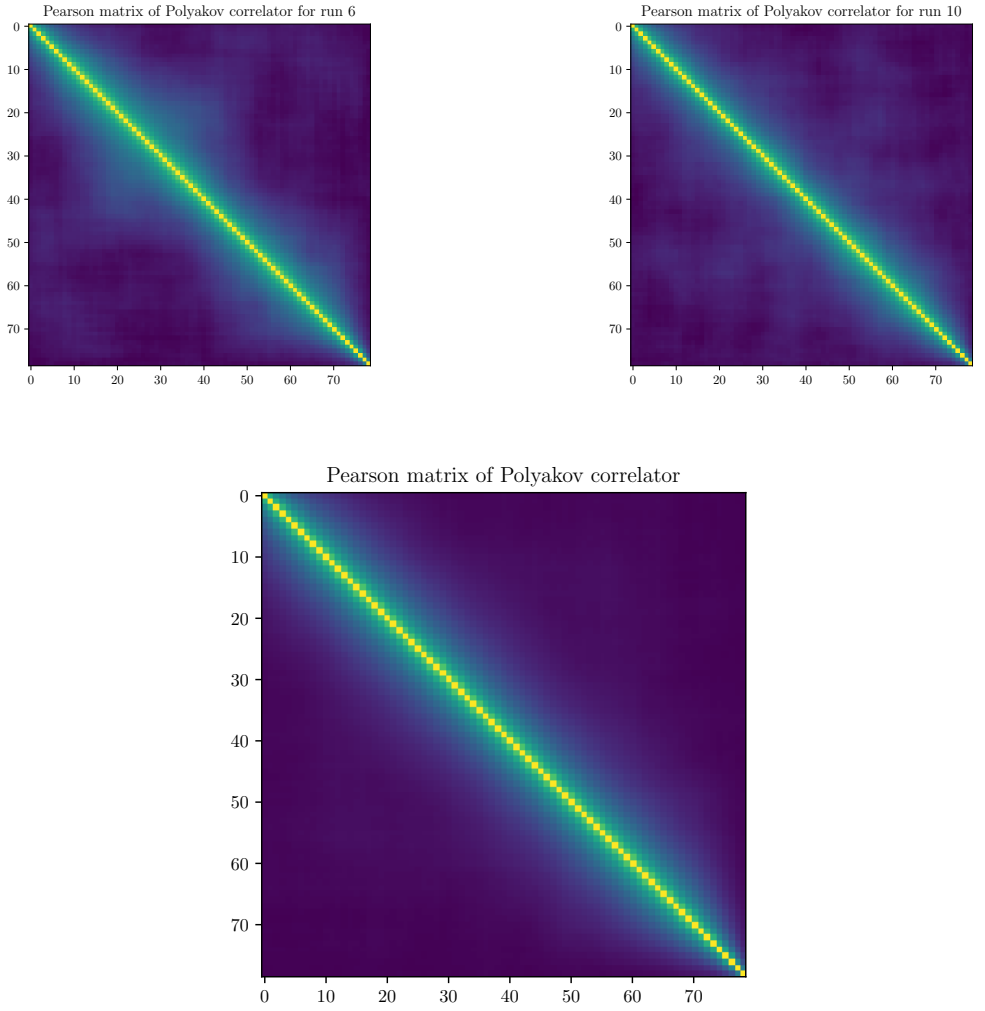


Figure 4.2: Normalized covariance matrix for run 6 (top left), run 10 (top right) and all runs (bottom).

4.3 Measures

After the quality check, further simulations have been performed in the usual setup $N_\mathrm{t} < N_\mathrm{s}$, as showed in rows 2-4 of Table 4.1. N_t is left to vary, studying how the lattice behaves at different time size values. For an analogous lattice with periodic boundary conditions, deconfinement phase transition happens at [18]

$$\beta_\mathrm{c}\left(N_\mathrm{t}\right) = 1.5028(21)N_\mathrm{t} + 0.705(21) - 0.718(49)\frac{1}{N_\mathrm{t}}. \tag{4.10}$$

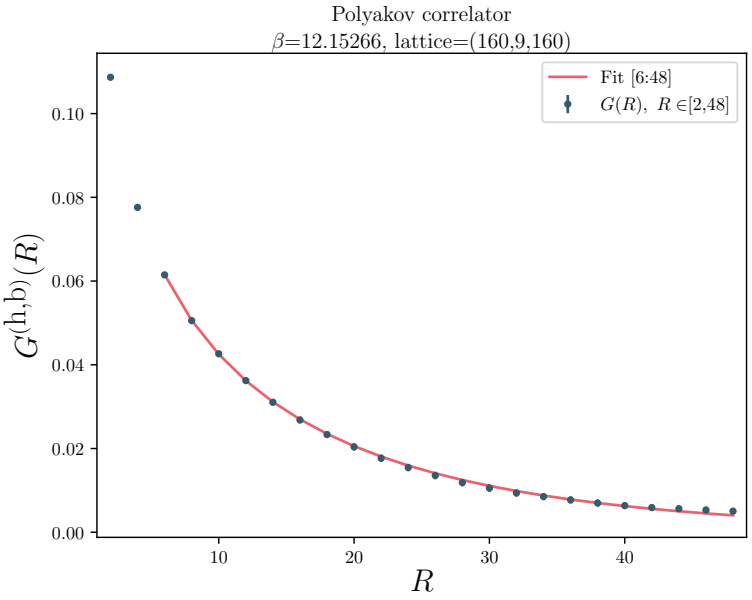


Figure 4.3: Fit and data of horizontal Polyakov loop correlator in the bulk of lattice.

Inverting this relation for chosen value of β yields $N_{\mathrm{t}}^{(\mathrm{C})} = 7.68$. Initial gauge configuration was set for all links outside Dirichlet faces to $U_{\mu}(x) = \mathbb{1} + 0.001M$, where M is a 2×2 matrix which entries are random numbers in $[-1, +1]$ sampled from uniform distribution and then normalized such that $M \in \mathrm{SU}(2)$.

4.3.1 Plaquettes

As previously stated, a distinction between spatial and temporal plaquettes has to be made. The average value on all spatial sites is performed for both the traces of spatial and temporal plaquettes at each time slice t. Trivial spatial ones that lay on Dirichlet faces are ignored, while temporal plaquettes with one link on a Dirichlet face are analysed. The symmetric trend of both observables with respect to $N_{\mathrm{t}}/2$ is clear in all lattices simulated, as it is shown in Figure 4.4 and further plots are available in Figure B.1.

After checking that the plaquettes behave as expected, it is possible to analyse Casimir effect comparing the plaquettes for all values of N_{t} simulated. The mean on all time slices is performed for the two kinds of plaquettes, to obtain a single value for the average spatial and for the temporal on every lattice. The observable extracted by using relation (2.20) is

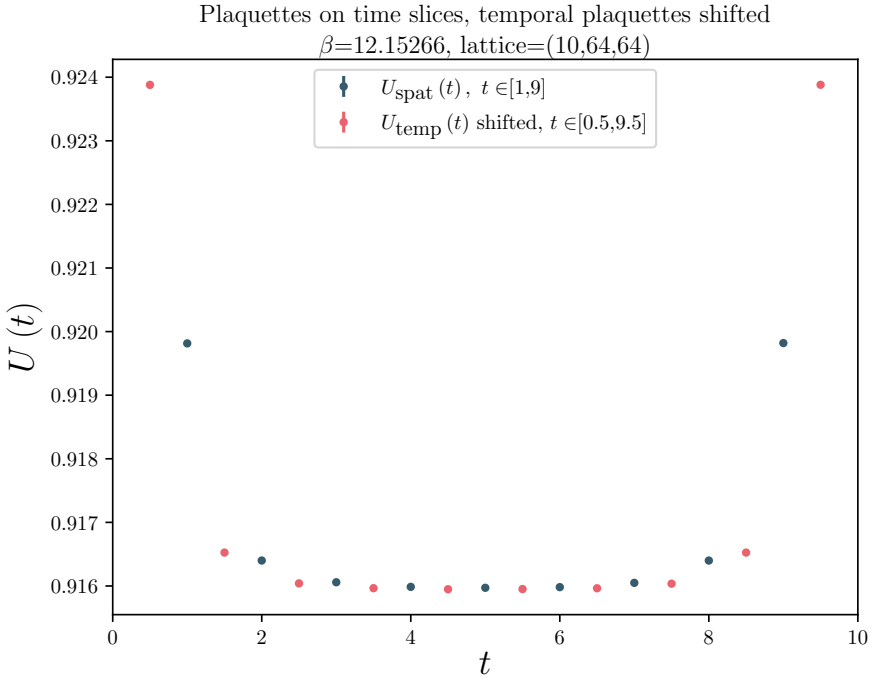


Figure 4.4: Plaquette on different time slices t for $N_\mathrm{t} = 10$. Temporal plaquettes are shifted to appear halfway between the two spatial plaquettes connected to them.

then compared to result presented in reference [19]

$$T^{00} = -\frac{3}{4} N_\mathrm{s} N_\mathrm{t} \left(\frac{m}{\pi N_\mathrm{t}} \right)^{3/2} \sum_{n=1}^{\infty} \frac{K_{3/2}\left(2nmN_\mathrm{t}\right)}{n^{3/2}} + c, \tag{4.11}$$

describing precisely a 2+1-dimensional lattice, where m and c are free parameters. The constant term c accounts for the fact that, given the current setup symmetries, the temporal plaquette is multiplied by 2 to get the energy density, producing a shift in the asymptotic ($N_\mathrm{t} \to \infty$) value of the observable. The trend of the data in the first place resembles a decreasing exponential with a negative coefficient, thus it has been plotted its difference with respect to the asymptotic value. The result is available in Figure B.2, the logarithmic scale shows that the decrease is not purely an exponential, therefore the fitting function has been expressed as

$$K_{3/2}(x) \simeq \sqrt{\frac{\pi}{2x}} \mathrm{e}^{-x}. \tag{4.12}$$

It has been found that a single term of the sum in (4.11) matches the tendency of data, as

shown in Figure 4.5.

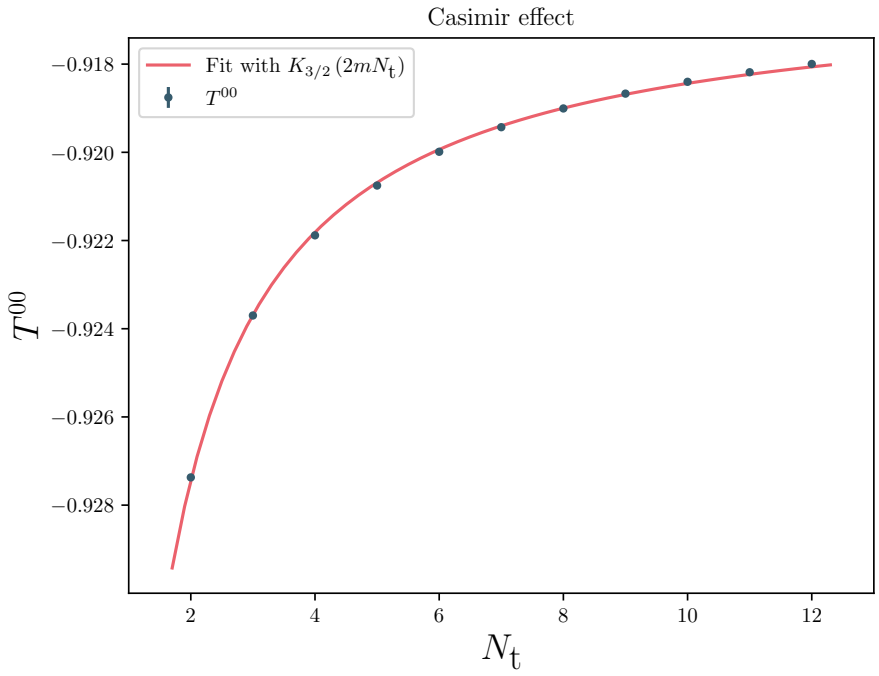


Figure 4.5: Casimir effect data plotted against fit results from (4.11), $n = 1$.

The fit outcome is shown in Table 4.3. It has been noticed that, when more terms of the infinite sum on n modified Bessel functions are taken into account, χ^2 remains almost constant. This is due to a very small value of the error on the plaquette measure, which has low autocorrelation. Including the $\mathcal{O}\left(\frac{1}{x}\right)$ term in $K_{3/2}(x)$ is also discarded, as for the fit values obtained, $2mN_t$ is not in the territory where this correction would be subleading.

m	c	χ^2_{red}
0.00295(1)	-0.91631(1)	9.45

Table 4.3: Fit results of energy density with (4.11), $n = 1$.

4.3.2 Polyakov Loop

The average Polyakov loop $\langle P(\vec{x})\rangle$ has been measured, results are shown in Figure 4.6 and Table 4.4.

r	$\langle P(\vec{x})\rangle$
2	0.537(20)
3	0.323(31)
4	0.218(24)
5	0.108(25)
6	0.076(23)
7	0.036(19)

r	$\langle P(\vec{x})\rangle$
8	0.046(15)
9	0.012(11)
10	0.010(7)
11	0.006(6)
12	-0.001(4)

Table 4.4: Polyakov loop at various N_t values.

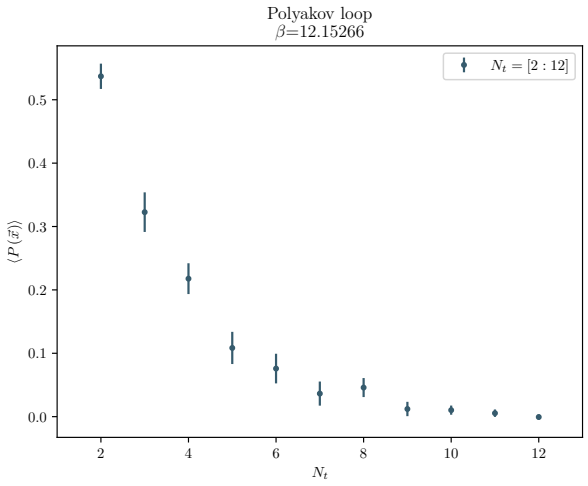


Figure 4.6: Data points of average Polyakov loop for lattices with varying N_t.

Results are unexpected: it seems that the Polyakov loop begins to acquire a non-null value around $N_\mathrm{t} = 8$. This should not happen, since a lattice with Dirichlet boundary conditions possesses a continuous symmetry $\mathrm{SU}(2)_\mathrm{L} \times \mathrm{SU}(2)_\mathrm{R}$: a vertical plaquette with one link laying on bottom plane is

$$U_{0\mu}(\vec{x}, t=0) = \mathbb{1} \cdot U_0(\vec{x}+\hat{\mu}, t=0)\, U_\mu^\dagger(\vec{x}, t=1)\, U_0^\dagger(\vec{x}, t=0), \qquad \mu \in \{1,2\}. \tag{4.13}$$

When performing a transformation $U_0(\vec{x}, t=0) \mapsto V\, U_0(\vec{x}, t=0)$, $V \in \mathrm{SU}(2)$, the cyclicity of trace in action (1.17) and the fact that vertical plaquette has now only three non trivial components make the action invariant under this transformation. The same happens for top Dirichlet face, thus the symmetry $\mathrm{SU}(2)_\mathrm{L} \times \mathrm{SU}(2)_\mathrm{R}$. Polyakov loop is the order parameter of this symmetry as in periodic boundary conditions scenario, missing the adjoint part to compensate for the transformation. This group is bigger than the centre $\mathbb{Z}_2$ of gauge group and it is not discrete, thus it would be impossible for a 2+1-dimensional system at finite temperature to break such symmetry [20, 21, 22].

These anomalous values may depend on sample size and starting configuration of the numerical simulation. To check, the Monte Carlo history of Polyakov loop is considered for $N_\mathrm{t} = 2$, the time size that yields the higher result for the observable. The values are plotted in a histogram on the left of Figure 4.7, it shows a high peak around 0.7, but also that the measure drifts slowly towards zero and that crosses it continuously. That is not the behaviour to be expected from a broken symmetry, furthermore from Table

4.4 it is evident that the result is closer to $\langle P(\vec{x})\rangle = 1$ the closer Dirichlet faces get. Considering also that the initial value of every link is itself very close to the identity, it may signify that simulation has not been initialized properly and that the proximity between the Dirichlet planes affects the rate at which Markov chain updates land the system into physically meaningful configurations. It is also to report that the blocking procedure failed to highlight an easily recognizable plateau for the error on the Polyakov loop, as it is shown in Figure B.5. This could mean that the sample size chosen is not sufficient to study the Polyakov loop observable properly.

A simulation with hot starting configuration (every link initial value is a random SU (2) matrix) has been performed with all other parameters left unchanged and the histogram on the right of Figure 4.7 depicts a very different dynamics: the Polyakov loop takes values both on positive and negative part of the real axis. Its average value in this scenario was significantly smaller than the previous value and with a much greater relative error, with the two measures not matching. This shows that the cold initialization affected largely the measure outcome when considering this order of sample sizes.

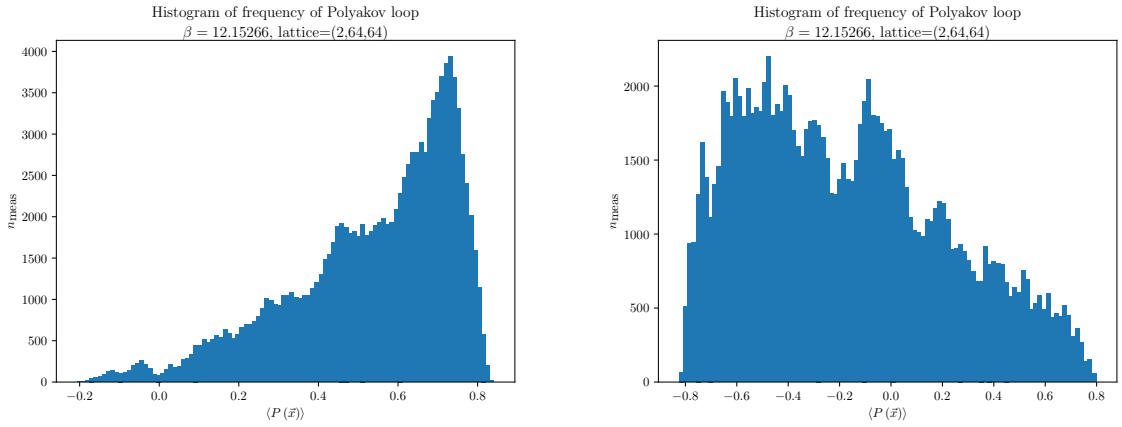


Figure 4.7: Histogram of Polyakov loop values during Monte Carlo simulation with cold (left) and (hot) starting configurations.

To check further, one could ask how the thermalization choice affects the measure. Different thermalization values have been explored for the data from simulation with hot initial configuration, with results presented in Figure 4.8. The trend towards zero is clear, however it is quite slow compared to the sample size of 8000 Monte Carlo steps. Choosing a higher value for thermalization would mean discarding a great portion of samples, thus a more reliable evaluation of the average Polyakov loop has to be performed on a bigger sample size.

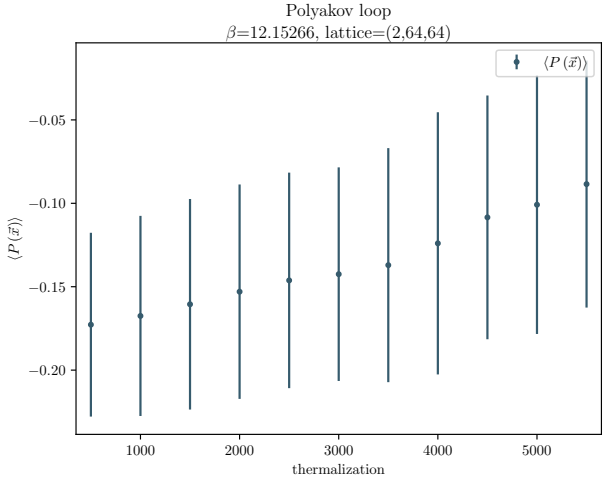


Figure 4.8: Polyakov loop values from random initialization with respect to thermalization choice.

4.3.3 Polyakov Loop Correlator

The Polyakov loop correlator $G(R)$ and correlator between spins of PCM $s(R)$ are computed for spatial distances $R \in [1, 64]$. It is clear the resemblance between the two: Polyakov loop correlator becomes a spin correlator when $N_t = 1$. The trend of both observables is symmetric with respect to $N_s/2$, as shown for a single lattice in Figure 4.9, an effect called "lattice echo". This result is not surprising: periodic boundary conditions on space allow the two loop to relate to each other crossing both distances R and $N_s - R$.

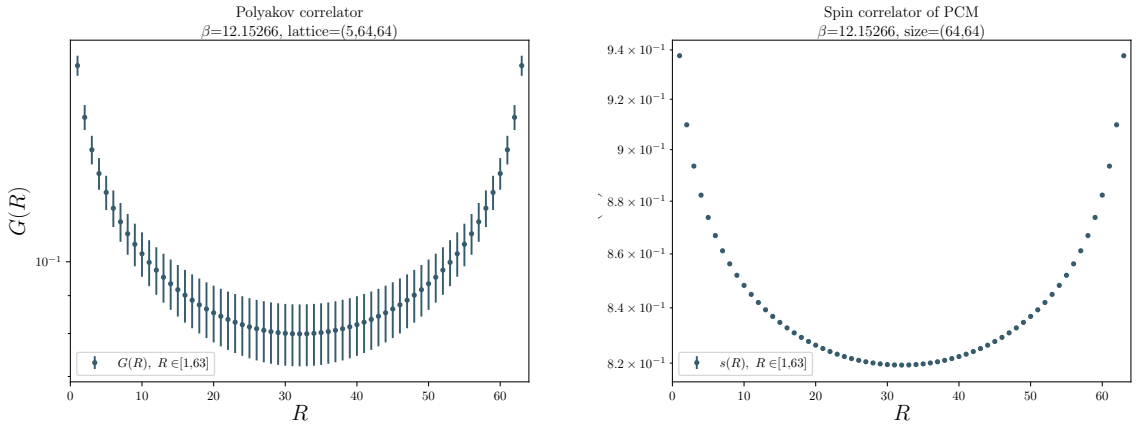


Figure 4.9: Symmetric trend of $G(R)$ and $s(R)$.

However, the uncertainty on Polyakov loop correlator for this sample size is too great to be able to model this trend. For this reason an additional simulation with a much greater sample size has been conducted to further study $G(R)$. It is compared to an analogous of the function used during the quality check, i.e.

$$G(R) = k\left[K_0(mR) + K_0(m(N_\mathrm{s} - R))\right], \tag{4.14}$$

where k and m are free parameters. A naive fit combining blocking procedure and covariance matrix on all data points would not work. Given the symmetry of the observable, $G(R)$ and $G(N_\mathrm{s} - R)$ are perfectly correlated, as it is shown in Figure B.3: they describe the same quantity. Hence the covariance matrix is not guaranteed to be positive definite and invertible, creating issues when trying to interpolate all data points. An answer to that would be to consider only the first half of the values and to try to fit them with (4.14), as it is represented in Figure B.4, discarding progressively the first r distances under the assumption that said values are more affected by statistical noise. Results are presented in Table 4.5 and the χ^2_red seems to point out a good data description from $r = 4$. Nevertheless, the plot in Figure B.3 with said r value for the fit to start shows that function (4.14) with this choice of parameters deviates from data when R is approaching $N_\mathrm{s}/2$. In that region errors are bigger due to larger fluctuations of the measure during Monte Carlo history, but also covariance matrix shows states that data from adjacent measures has a high correlation, these two circumstances influence fit results to describe badly $G(R)$ for such values.

N_t	k	m	χ^2_red
3	0.0408(2)	0.0436(7)	3.165
4	0.0397(3)	0.0416(7)	1.498
5	0.0389(3)	0.0404(8)	0.808
6	0.0384(4)	0.0397(8)	0.477
7	0.0379(4)	0.0391(8)	0.297
8	0.0376(5)	0.0387(9)	0.192

Table 4.5: Fit results using (4.14) considering covariance matrix.

One could ask which of the two effects causes the most discrepancy between data values and fit. To answer, two alternative remedies have been compared: removing the last four distances from the fit domain, thus losing the points where echo effect is stronger, but also with a greater uncertainty; or trying to include all data values and ignoring the correlation matrix, neglecting the relations it encodes. The one of these two treatments that improves data description the most will point out which was the main cause of the gap, whether

if error magnitude (first case) or correlation (second one). The two cases are juxtaposed in Figure 4.10, where it is evident that bypassing correlation analysis provides a better agreement between data and interpolated function. Fit values of said configuration is in Table 4.6.

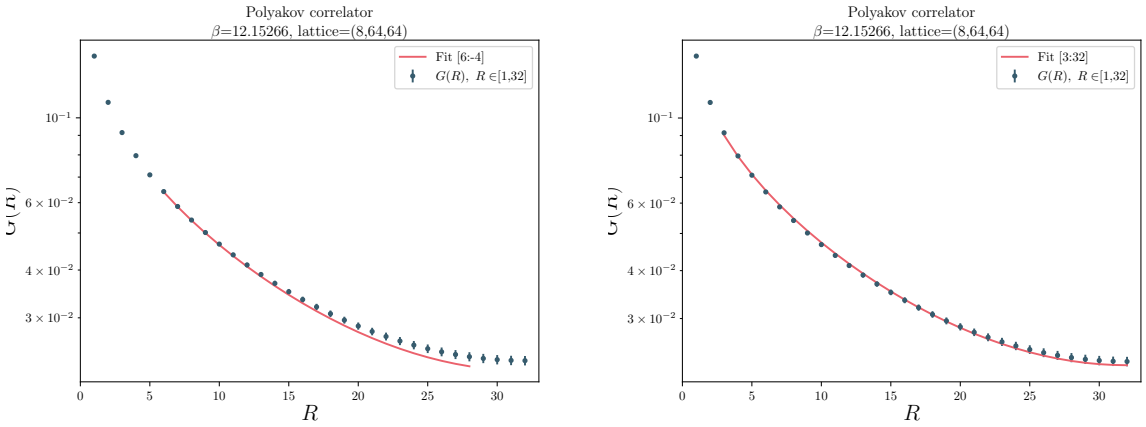


Figure 4.10: Two different treatments to improve fit of (4.14) with data: removing the last points with $r = 4$ (left) or ignoring covariance matrix with $r = 2$ (right).

N_t	k	m	χ^2_red
1	0.0398(2)	0.0404(2)	3.063
2	0.0387(2)	0.0394(3)	0.908
3	0.0380(3)	0.0389(3)	0.365
4	0.0376(3)	0.0385(3)	0.174
5	0.0373(4)	0.0382(4)	0.091

Table 4.6: Fit results at different r for (4.14) without covariance matrix.

4.3.4 Wilson Loop

To further examine the transition between the gauge theory and the Principal Chiral Model, another observable related to spin correlator $s\,(R)$ is taken into account, the Wilson loop of vertical size $T = 1$ emerging from a Dirichlet face. Implying that Wilson loop is directed towards positive direction of t when computed on a specific time slice, its border value is defined as

$$W^{(\mathrm{b})}\,(R \times 1) \coloneqq W\,(R \times 1)|_{t=0} = W\,(R \times 1)|_{t=N_\mathrm{t}-1}\,. \tag{4.15}$$

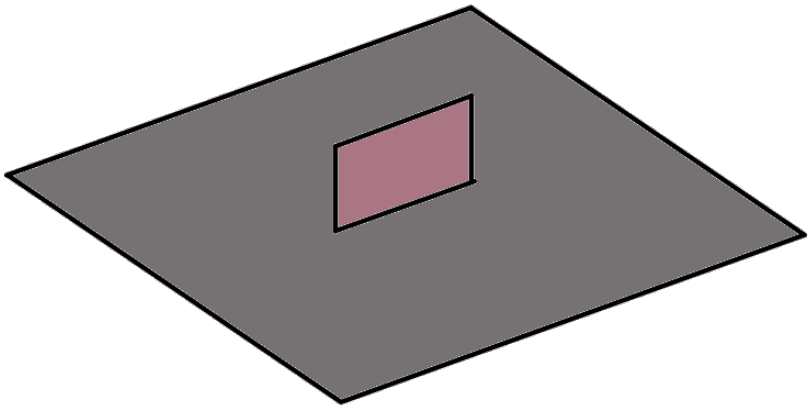


Figure 4.11: Wilson loop on border $W^{(\mathrm{b})}(R \times 1)$.

This quantity is related to $s(R)$ from another perspective with respect to the Polyakov loop correlator already discussed: it does not span all lattice height, thus not connecting the two Dirichlet faces as $G(R)$ does, rather its time size stays fixed to what is interpreted as the spin for PCM on all lattice heights $\{N_{\mathrm{t}}\}$. Data from different lattices can be found in Figure B.6, where can already be seen a general and interesting result compared to other observables previously discussed: the trend is not symmetric around $N_{\mathrm{s}}/2$. This is due to the fact that one of the horizontal paths between the two vertical links is not trivial, the one closer to the bulk of the lattice, while this is not true for $s(R)$ and $G(R)$.

To verify this claim, horizontal Polyakov loops on every different time slice $P_{\mathrm{h}}(t)$ have been measured, where

$$P_{\mathrm{h}}(t) = \sum_{\nu \neq \mu} \langle P_{\mu}(x_{\nu}, t) \rangle_{x_{\nu}}, \qquad \mu, \nu \in \{1, 2\}, \tag{4.16}$$

where $P_{\mu}(x_{\nu}, t)$ is the analogous of the usual (vertical) Polyakov loop, but in direction $\hat{\mu}$. This quantity is obviously symmetric around $t = N_{\mathrm{t}}/2$, as expected, and it is shown for various lattices in Figure B.9, discarding trivial loops on Dirichlet faces. What is important to analyse is the value of $P_{\mathrm{h}}(t = 1)$, since when considering a Wilson loop on border of size $R = N_{\mathrm{s}}$, the two vertical links coincide and the triviality of the inferior horizontal path

realizes:

$$
\begin{aligned}
W^{(\mathrm{b})}\left(N_{\mathrm{s}} \times 1\right) &= \langle \mathrm{tr}\left[\mathbb{1} \cdot U_0\left(\vec{x}, t=0\right) \prod_{x_\mu=0}^{N_{\mathrm{s}}-1} U_\mu^\dagger\left(x_\mu, t=1\right) \cdot U_0^\dagger\left(\vec{x}+R\hat{\mu}, t=0\right)\right] \rangle = \\
&= \langle \mathrm{tr}\left[U_0\left(\vec{x}, t=0\right) \prod_{x_\mu=0}^{N_{\mathrm{s}}-1} U_\mu^\dagger\left(x_\mu, t=1\right) \cdot U_0^\dagger\left(\vec{x}, t=0\right)\right] \rangle = \\
&= \langle \mathrm{tr}\left[\prod_{x_\mu=0}^{N_{\mathrm{s}}-1} U_\mu^\dagger\left(x_\mu, t=1\right)\right] \rangle = P_{\mathrm{h}}\left(t=1\right).
\end{aligned}
\tag{4.17}
$$

Simulation data confirms this relation: $W^{(\mathrm{b})}\left(N_{\mathrm{s}} \times 1\right)$ matches with $P_{\mathrm{h}}\left(t=1\right)$ for any lattice, as some examples show in Table 4.7.

N_{t}	$W^{(\mathrm{b})}\left(N_{\mathrm{s}} \times 1\right)$	$P_{\mathrm{h}}\left(t=1\right)$
3	0.0535(2)	0.0535(1)
6	0.0416(2)	0.0416(1)
9	0.0396(2)	0.0396(2)
12	0.0389(2)	0.0389(1)

Table 4.7: Comparison between Wilson loop on border across all spatial dimension and horizontal Polyakov loop on first non-trivial time slice for various N_{t}.

Given this behaviour, an hypothesis has been made on how to model the trend of Wilson loop on border: since the lattice echo is suppressed and the Wilson loop at great distances is related to the horizontal Polyakov loop, it was supposed that it follows a law of the type

$$
W^{(\mathrm{b})}\left(N_{\mathrm{s}} \times 1\right) = k_1 \mathrm{e}^{-mR} + k_2 \mathrm{e}^{-m\left(N_{\mathrm{s}}-R\right)}, \tag{4.18}
$$

where k_1, k_2 and m are free parameters and k_2 should be compatible with the value of the horizontal Polyakov loop. However, this assumption was proven wrong by performing fits on the data, as the value of k_2 obtained was at least two orders of magnitude smaller than the corresponding $P_{\mathrm{h}}\left(t=1\right)$. Thus, it is better to follow another route to extract some quantitative information from the Wilson loop. The linear trend in logarithmic scale still suggests to model the observable after an exponential function, so by writing

$$
W^{(\mathrm{b})}\left(N_{\mathrm{s}} \times 1\right) = k \cdot \mathrm{e}^{-m_{\mathrm{eff}} R} \tag{4.19}
$$

for R values that are not extremal it is possible to analyse the observable without accounting for extremely short- and long-range effects that cannot be neglected at those distances. Again, k is the amplitude parameter and m_{eff} is the mass parameter that is the inverse of the effective correlation length ξ_{eff}. Some plots are available in Figure B.7.

With this choice, it is possible to construct another quantity to have a better understanding of Wilson loop's dependence on distance:

$$F(R) \coloneqq \ln\left[W^{(\mathrm{b})}(R \times 1)\right] - \ln\left[W^{(\mathrm{b})}((R+1) \times 1)\right]. \tag{4.20}$$

. In facts it is easy to see that it represents effective mass evolution with R:

$$\begin{aligned} F(R) &= \ln\left[W^{(\mathrm{b})}(R \times 1)\right] - \ln\left[W^{(\mathrm{b})}((R+1) \times 1)\right] = \ln\left[\frac{k \cdot \mathrm{e}^{-m_{\text{eff}} R}}{k \cdot \mathrm{e}^{-m_{\text{eff}}(R+1)}}\right] = \\ &= \ln\left(\frac{1}{\mathrm{e}^{-m_{\text{eff}}}}\right) = m_{\text{eff}}. \end{aligned} \tag{4.21}$$

It is expected that this quantity should be constant with R in the range of interest, since m_{eff} does not depend on Wilson loop size. Plots for $F(R)$ for two scenarios depending on N_{t} are attached in Figure 4.12 and at different values of N_{t} in Figure B.8.

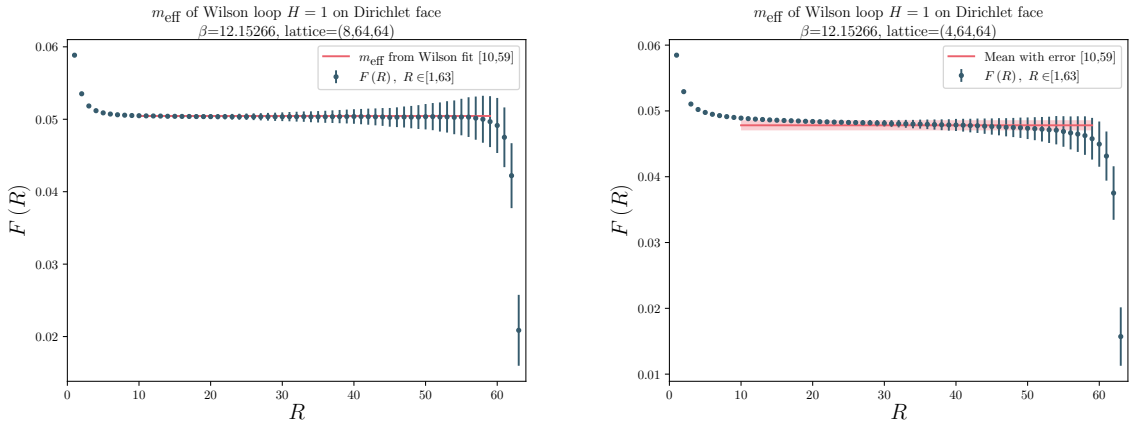


Figure 4.12: Estimates of effective mass of Wilson loop on border for $N_{\mathrm{t}} = 4, 8$. For $N_{\mathrm{t}} = 4$ it is plotted the mean of selected data points, while for $N_{\mathrm{t}} = 8$ the free parameter from Wilson data fitted with (4.19).

It was found that the model is compatible with data for lattices with $N_{\mathrm{t}} > 5$ and the m_{eff} value extracted from fitting Wilson loop with functional form (4.19) lays on $F(R)$ plateau. For $N_{\mathrm{t}} \leq 5$, on the other hand, $F(R)$ has a noticeable downward tendency also

for medium sized loops, so it would not be reasonable to compare a m_{eff} value obtained via fit procedure. To estimate its value without downsizing the error, it was chosen to compute the mean and standard deviation of $F(R)$ points in the chosen range. Finally, results of all effective masses is shown in Figure 4.13.

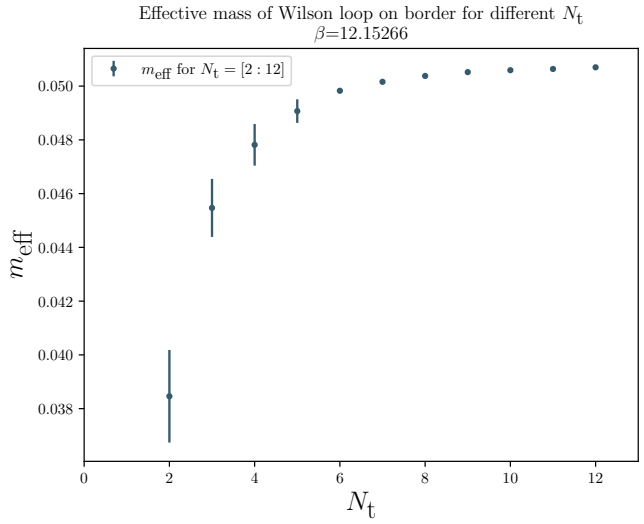


Figure 4.13: Effective masses extracted from $W^{(b)}(R \times 1)$ for all lattice heights in range $R \in [10, 59]$.

Chapter 5

Conclusion

This work investigates the implications of Dirichlet boundary conditions enforced as a constraint on a gauge theory, regularized on a lattice. Since the SU (2) × SU (2) Principal Chiral Model in $D = 2$ emerges as the $N_\mathrm{t} = 1$ limit of the $(2 + 1)$-dimensional gauge theory, it was selected to analyse how the physics in the bulk of the lattice is affected by the increasing closeness of Dirichlet faces and to relate the behaviour with the known facts about the PCM. However, with the current choice of parameters, observables such as the mass gap of the PCM are not accessible: the correlation length between spins exceeds the spatial size of the lattice $\xi \gg N_\mathrm{s}$. This is one of the factors limiting a comprehensive account of the spin-to-gauge theory transition. There is also an additional subtlety for that, which emerges by considering the change of dynamics when the $D = 2$ system is upgraded with an additional dimension, it is not trivial to yield a conclusive depiction of the consequences. In facts, the change of dimensionality allows supplementary degrees of freedom to emerge, that are nevertheless influenced by the boundary conditions enforced on the theory. Establishing a correspondence and comparison between the two models provides a starting point for further research regarding this underlying relation and offers a insight on the configurations better suited for a rigorous analysis.

Moreover, given the exploratory setting of this thesis, further evaluations have been performed. The expected symmetry with respect to the half point of the lattice time dimension was confirmed for all observables defined on time slices that have been measured. By considering the difference of spatial and temporal plaquettes, the energy density has confirmed Casimir effect predictions in $2 + 1$ dimensions. The setup in which the result is obtained slightly differs from the common arrangement when studying this quantity: vacuum energy difference is often computed by placing perfectly conductive parallel wires, while the current investigation regarded boundary surfaces as two-dimensional plates. The

Polyakov loop value analysis suggests than the value that has been found initially is highly dependent on the initial configuration of the simulation variables and time scale of their evolution towards equilibrium distribution.

Perhaps the most interesting result unveiled is the behaviour of $H = 1$ Wilson loops which stand on the Dirichlet faces with respect to R, that is considerably different from the symmetric trend of the other macroscopic observables determined by lattice echo. This pattern was explained as the result of the non-triviality of horizontal links on only one side of the loop, that form an horizontal Polyakov loop when $R = N_\mathrm{s}$. Numerical simulation outcomes confirmed this hypothesis. By modelling $W^{(\mathrm{b})}(R \times 1)$ with an exponential decay law in R, a picture of effective mass evolution with lattice time size N_t was provided. It is interesting to notice, even if it is too early to give a conclusive explanation, that the asymptotic value $N_\mathrm{t} \to \infty$ of said effective mass in Figure 4.13 should take value just over 0.05. This is considerably greater than the zero-temperature string tension value obtained with equal β and periodic boundary conditions on all dimensions $\sigma = 0.0137$. The analogy between the two quantities is encoded in the fact that they both are the coefficient of the $R \times 1$ area of the Wilson loop in its leading term of the expansion. This suggests that the spin theory on Dirichlet borders has an impact on this observable's behaviour and that further contributions should be taken into account when looking at the Wilson loop in these two different setups. A possible advice would be to continue studying how the Wilson loop is affected by also changing its temporal size H while maintaining its base anchored on the border of the lattice, that should eventually coincide with the already considered Polyakov loop correlator when $H = N_\mathrm{t}$. This should provide more informations on the new dynamics arising from the boundary constraints as these loops could extend from the extremal part of the time size, where the Dirichlet effects are stronger, to the bulk of the lattice.

Appendix A

Supplements

A.1 Continuum Limit of Wilson Action

In this derivation, lattice spacing a will be mentioned explicitly to highlight its role. To link $U_\mu(x)$ matrix to its algebra element, it is convenient to define $B_\mu(x) \coloneqq agA^b_\mu(x)\tau_b$ and exponentiate it

$$U_\mu(x) = \mathrm{e}^{\mathrm{i}B_\mu(x)}. \tag{A.1}$$

Taking the continuum limit means to get closer to the identity point of the group, a region where algebra elements describe the system dynamics. Using the finite difference on lattice defined in (1.9) it is possible to write every site algebra variable with respect to the previous site one:

$$B_\nu(x+\hat{\mu}) = B_\nu(x) + a\nabla_\mu B_\nu(x)\,. \tag{A.2}$$

In continuum limit $a \to 0$ derivative is recovered. Inserting (A.1) and (A.2) inside the plaquette definition (1.16) yields

$$U_{\mu\nu}(x) = \mathrm{e}^{\mathrm{i}B_\mu(x)}\mathrm{e}^{\mathrm{i}[B_\nu(x)+a\nabla_\mu B_\nu(x)]}\mathrm{e}^{-\mathrm{i}[B_\mu(x)+a\nabla_\nu B_\mu(x)]}\mathrm{e}^{-\mathrm{i}B_\nu(x)}. \tag{A.3}$$

Using Baker-Campbell-Hausdorf formula to first order $\mathrm{e}^x\mathrm{e}^y \simeq \mathrm{e}^{x+y+\frac{1}{2}[x,y]}$ it is possible to write

$$U_{\mu\nu}(x) \simeq \mathrm{e}^{\mathrm{i}[B_\mu(x)+B_\nu(x)+a\nabla_\mu B_\nu(x)]+\mathrm{i}^2\frac{1}{2}[B_\mu(x),B_\nu(x)]}\mathrm{e}^{-\mathrm{i}[B_\mu(x)+a\nabla_\nu B_\mu(x)+B_\nu(x)]+(-\mathrm{i})^2\frac{1}{2}[B_\mu(x),B_\nu(x)]}, \tag{A.4}$$

where a^3 terms are neglected since they are suppressed in continuum limit. Repeating the same procedure it is finally obtained

$$U_{\mu\nu}(x) \simeq \mathrm{e}^{\mathrm{i}[a\nabla_\mu B_\nu(x) - a\nabla_\nu B_\mu(x)] - [B_\mu(x), B_\nu(x)]}. \tag{A.5}$$

Defining the analogous of the field strength for the lattice fields

$$F_{\mu\nu}(x) \coloneqq \nabla_\mu A_\nu(x) - \nabla_\nu A_\mu(x) + \mathrm{i}g\left[A_\mu(x), A_\nu(x)\right], \tag{A.6}$$

the plaquette can be written as

$$U_{\mu\nu}(x) \simeq \mathrm{e}^{\mathrm{i}a^2 g F_{\mu\nu}(x)}. \tag{A.7}$$

By series expansion the first terms are

$$U_{\mu\nu}(x) \simeq \mathbb{1} + \mathrm{i}a^2 g F^b_{\mu\nu}(x)\,\tau_b - \frac{1}{2}a^4 g^2 F^b_{\mu\nu}(x)\,F^{\mu\nu|c}(x)\,\tau_b\tau_c, \tag{A.8}$$

on which the (real) trace should be performed. Trace of identity matrix is constant, so it is reabsorbed into normalization factor; for $\mathrm{SU}(N)$ groups in fundamental representation it is common to choose traceless generators $\mathrm{tr}(\tau_b) = 0$, normalized as $\mathrm{tr}(\tau_b\tau_c) = \frac{1}{2}\delta_{bc}$, where δ_{bc} is the Kronecker delta. Wilson action (1.17) thus becomes

$$\mathscr{S}_\mathrm{w} \simeq \frac{\beta}{2N}\sum_{\{x\}} \mathrm{Re}\left[-\frac{1}{4}a^4 g^2 F^b_{\mu\nu}(x)\,F^{\mu\nu}_b(x)\right] \tag{A.9}$$

Taking the continuum limit by reversing the first relation of (1.9) it becomes

$$\mathscr{S}_\mathrm{w} \simeq -\frac{\beta g^2}{8N}a^{4-D}\int F^b_{\mu\nu}(x)\,F^{\mu\nu}_b(x)\,\mathrm{d}^D x \tag{A.10}$$

which is equivalent to Yang-Mills action (1.11) if $\frac{\beta g^2}{8N}a^{4-D} = \frac{1}{4}$, giving the dependence $\beta(g)$

$$\beta(g) = \frac{2N}{g^2}a^{4-D}. \tag{A.11}$$

Note that in $D = 4$ the lattice spacing is completely absorbed and a dimensionless coupling strength g produces a dimensionless parameter β, while for $D \neq 4$ the coupling factor has to be dimensional to produce a dimensionless β, compensating a.

A.2 Periodicity of Markov Chains

For any initial state $a \in \Omega$ of a Markov chain, the set of *recurrence times* is defined as any number of steps k for which it is possible that the Markov chain returns in a, so

$$R_a \coloneqq \left\{ k > 0 \mid \left(W^k\right)_{aa} > 0 \right\}, \tag{A.12}$$

and the *period* T_a of a state as the greatest common divisor of R_a. Given this definition, if k is not divisible by T_a, then it is certain that the probability of the Markov chain of returning to a after k steps is zero, i.e. $\left(W^k\right)_{aa} = 0$. If every state of a Markov chain has period $T_a = 1$, then the chain is named *aperiodic*, as there is no typical time after which a state is visited again. With these definitions it is possible to explore some properties of irreducible Markov chains.

Theorem A.1. *All states of an irreducible Markov chain have the same period.*

Proof. Consider two states $a, b \in \Omega$ and their periods T_a, T_b. Since the Markov chain is irreducible, all states are accessible from any other, thus there exists positive steps k_1 and k_2 such that $\left(W^{k_1}\right)_{ba} > 0$ and $\left(W^{k_2}\right)_{ab} > 0$. Concatenating the two evolutions it is easy to see that for $k = k_1 + k_2$ it is true that $\left(W^k\right) > 0$, hence $k \in R_a$ and it is divisible by T_a.

With the same reasoning, there is also a number of steps l for which $\left(W^l\right)_{bb} > 0$, hence the Markov chain could take k_1 steps to move from a to b, then l steps and return in b, then other k_2 steps to finally return to a. The total number of steps would be $k + l$ and it must be divisible by T_a, but k is itself a multiple of T_a, hence also l should be divisible by T_a. l is a recurrence time of b, so it is by definition divisible by T_b, thus T_b must be divisible by T_a.

By switching a and b also T_a should be divisible by T_b, so it follows that $T_a = T_b$. □

Theorem A.2. *In an irreducible Markov chain of period T the state space can be decomposed as $\Omega = A_0 \cup A_1 \cup \dots A_{T-1}$, with $A_i \cap A_j = \{\emptyset\}$ for all $i \neq j$ and, if $a \in A_i$ and $W_{ab} > 0$, then $b \in A_{(i+1) \pmod T}$.*

Proof. The sets $\{A_i\}$ are defined as

$$A_i \coloneqq \left\{ a \in \Omega \mid \exists k \equiv i \pmod T \text{ such that } \left(W^k\right)_{a1} > 0 \right\}, \qquad i \in \{1, 2, \dots T-1\}, \tag{A.13}$$

i.e. all states that can be reached by state 1 after a number of steps that, subtracted from i, would make it divisible by T. Since the Markov chain is irreducible, it must contain all

these sets, and their intersection is the empty set, since if it were not the case, then a state a could be reached from state 1 with two different paths k_1, k_2 that have different result under modulo T, so

$$\left(W^{k_1}\right)_{a1} > 0, \quad \left(W^{k_2}\right)_{a1} > 0, \qquad k_1 \equiv k_2 \pmod{T}. \tag{A.14}$$

Irreducibility also ensures that there exists a value l for which $\left(W^l\right)_{1a} > 0$, so combining it with one of the considered paths yields $\left(W^{k_1+l}\right)_{11} > 0$, hence $k_1+l \in R_1$ and it is divisible by T. The same can be done for k_2, so also k_2+l is divisible by T and $k_1+l-(k_2+l) = k_1-k_2$ is divisible by T: they must be equivalent under modulo T.

Considering $a \in A_i$ and another state b for which $W_{ba} > 0$, then for A_i definition there is a k such that $k \equiv i \pmod{T}$ and $\left(W^k\right)_{a1} > 0$. Expanding the matrix element corresponding to $k+1$ steps starting from 1 to b it is strictly positive:

$$\left(W^{k+1}\right)_{b1} = \sum_{\{c\in\Omega\}} W_{bc}\left(W^k\right)_{c1} \geq W_{ba}\left(W^k\right)_{a1} > 0. \tag{A.15}$$

Thus $b \in A_{(n+1) \pmod{T}}$ since $k+1 \equiv i+1 \pmod{T}$. □

Corollary A.2.1. *If an irreducible Markov chain has period $T > 1$ and is defined by stochastic matrix W, then the Markov chain defined by W^T is reducible.*

Proof. Decomposing the sample space into disjoint sets $\Omega = A_1 \cup A_2 \cup \ldots A_{T-1}$, an element $a \in A_i$ has a k for which $\left(W^k\right)_{a1} > 0$ and $k-i$ is divisible by T, but of course $k+T \equiv i \pmod{T}$ and sets are disjointed, so it is impossible to not get an element of A_i. □

Using theorems about greatest common divisors, it is also possible to prove the following theorem.

Theorem A.3. *For an irreducible and aperiodic Markov chain, there exists a value k_0 such that $\left(W^k\right)_{ba} > 0$ if $k > k_0$ for any $a, b \in \Omega$.*

A.3 Stochastic Matrix Properties

In this section it is assumed that the stochastic matrix W under consideration is diagonalizable. For it, following theorems are valid.

Theorem A.4. *A stochastic matrix W has $\lambda = 1$ as one of its eigenvalues.*

Proof. Since the columns of a stochastic matrix are normalized, it is possible to write

$$\sum_{\{b\in\Omega\}} (W_{ba} - \delta_{ba}) = 0, \tag{A.16}$$

for every $a \in \Omega$, hence the rows of the matrix $W - \mathbb{1}$ are dependent and its determinant is zero, so $\lambda = 1$ is an eigenvalue of W. □

Theorem A.5. *Every eigenvalue of a stochastic matrix W is $|\lambda| \leq 1$.*

Proof. If v_b is the eigenvalue corresponding to λ, then

$$\sum_{\{a\in\Omega\}} W_{ba} v_a = \lambda v_b. \tag{A.17}$$

Studying the magnitude of eigenvalue it is true that

$$|\lambda||v_b| = |\lambda v_b| = |\sum_{\{a\in\Omega\}} W_{ba} v_a| \leq \sum_{\{a\in\Omega\}} |W_{ba} v_a| = \sum_{\{a\in\Omega\}} W_{ba}|v_a|, \tag{A.18}$$

since W_{ba} is always non-negative. Now considering

$$|\lambda| \sum_{\{b\in\Omega\}} |v_b| \leq \sum_{\{a\in\Omega\}} \overbrace{\sum_{\{b\in\Omega\}} W_{ba}}^{1} |v_a| = \sum_{\{a\in\Omega\}} |v_a|, \tag{A.19}$$

and comparing the initial and final sides, it follows that $|\lambda| \leq 1$. □

Theorem A.6. *If v_b is an eigenvector of a stochastic matrix with eigenvalue $\lambda \neq 1$, then the sum of the eigenvector on all states is zero.*

Proof. From stochastic matrix and eigenvector decomposition it is possible to write

$$\lambda \sum_{\{b\in\Omega\}} v_b = \sum_{\{a\in\Omega\}} \overbrace{\sum_{\{b\in\Omega\}} W_{ba}}^{1} v_a = \sum_{\{a\in\Omega\}} v_a, \tag{A.20}$$

thus, if $\lambda \neq 1$, it follows that $\sum v_b = 0$. □

Another theorem holds true, however the proof is slightly more technical, so it will not be presented.

Theorem A.7. *In an irreducible Markov chain defined by a stochastic matrix W, if an eigenvector v_a of the matrix has eigenvalue $\lambda = 1$, then all components of v_a have the same sign.*

This is particularly useful to prove the following property.

Theorem A.8. *In an irreducible Markov chain, the eigenvalue $\lambda = 1$ of its stochastic matrix is non-degenerate.*

Proof. Suppose that v and v' are two different eigenvectors with eigenvalue $\lambda = 1$, then all of their components are concordant because of the Theorem A.7. Assuming $v_a > 0$ and ${v'}_a > 0$ for any $a \in \Omega$, it is possible to normalize both vectors such that

$$\sum_{\{a\in\Omega\}} v_a = 1 = \sum_{\{a\in\Omega\}} v'_a. \tag{A.21}$$

Defining $w_a \coloneqq v_a - v'_a$ for all states, it is a linear combination of eigenvectors with the same eigenvalue, thus it is still an eigenvector of W with eigenvalue 1. If so, Theorem A.7 enforces again all components to have the same sign, but

$$\sum_{\{a\in\Omega\}} w_a = \sum_{\{a\in\Omega\}} v_a - \sum_{\{a\in\Omega\}} {v'}_a = 0 \tag{A.22}$$

and w cannot be the null eigenvector, thus $v_a = {v'}_a$. □

By means of the last two theorems it is possible to prove the following theorem, which will be central in the upcoming discussion of the invariant distribution of the Markov chain.

Theorem A.9. *An irreducible and aperiodic Markov chain defined by the stochastic matrix W, all eigenvalues $\{\lambda_a \neq 1\}$ satisfy $|\lambda_a| < 1$.*

It has been stated that, for an aperiodic and irreducible Markov chain, $\lambda = 1$ is a non-degenerate eigenvalue of the stochastic matrix W and its eigenvector has all components strictly positive, with an appropriate choice of sign. Furthermore, all other eigenvalues different from 1 have magnitude strictly inferior to 1. These facts mean that the Markov chain has a unique invariant pdf π, with $\pi_a > 0$ for any $a \in \Omega$, that is corresponding to the eigenvalue $\lambda = 1$. Expanding the quantity $\left(W^k p\right)_a$, where p is another pdf on Ω,

$$p_a = c_1 \pi_a + \sum_{\{j>1\}} c_j v_a^{(j)}, \tag{A.23}$$

where $v_a^{(j)}$ is the component on a of j-th eigenvector associated with an absolute eigenvalue smaller than one. From Theorem A.6 and normalization of pdfs it follows that

$$\sum_{\{a\in\Omega\}} v_a^{(j)} = 0, \qquad \sum_{\{a\in\Omega\}} p_a = 1 = \sum_{\{a\in\Omega\}} \pi_a, \tag{A.24}$$

so it is possible to combine them to obtain

$$\sum_{\{a\in\Omega\}} p_a = c_1 \sum_{\{a\in\Omega\}} + \sum_{\{a\in\Omega\}} \sum_{\{j>1\}} c_j v_a^{(j)} = c_1 + \sum_{\{a\in\Omega\}} \sum_{\{j>1\}} c_j v_a^{(j)}. \tag{A.25}$$

A normalization can be found such that

$$\sum_{\{a\in\Omega\}} \left| v_a^{(j)} \right| = 1, \tag{A.26}$$

hence the two sums can be switched, yielding

$$\sum_{\{a\in\Omega\}} p_a = c_1 + \sum_{\{j>1\}} \left(\sum_{\{a\in\Omega\}} v_a^{(j)} \right) c_j = c_1 = 1. \tag{A.27}$$

Applying W^k to the p pdf one finds

$$\left(W^k p\right)_a = \left(W^k \pi\right)_a + \sum_{\{j>1\}} c_j \left(W^k v^{(j)}\right)_a = \pi_a + \sum_{\{j>1\}} c_j \lambda_j^k v_a^{(j)}, \tag{A.28}$$

where it is used the fact that π is the eigenvector of W with $\lambda = 1$. To estimate the convergence of $W^k p$ to the stationary pdf π, let's define

$$\Lambda \coloneqq \max_{\{j>1\}} |\lambda_j| \tag{A.29}$$

that is of course positive because of the absolute value property, moreover every eigenvalue different from one is strictly smaller than $\lambda = 1$, hence this quantity is bounded $0 \leq \Lambda < 1$.

Considering the absolute difference between the two pdfs it follows

$$\sum_{\{a\in\Omega\}} \left|\left(W^k p\right)_a - \pi_a\right| = \sum_{\{a\in\Omega\}} \left| \sum_{\{j>1\}} c_j \lambda_j^k v_a^{(j)} \right| \leq \sum_{\{a\in\Omega\}} \sum_{\{j>1\}} |\lambda_j|^k |c_j| |v_a^{(j)}| \leq$$
$$\leq \Lambda^k \sum_{\{j>1\}} |c_j| \underbrace{\sum_{\{a\in\Omega\}} |v_a^{(j)}|}_{1} = \Lambda^k \sum_{\{j>1\}} |c_j|, \tag{A.30}$$

finally defining the sum of coefficients' magnitudes as a real number A it is found

$$\sum_{\{a\in\Omega\}} \left|\left(W^k p\right)_a - \pi_a\right| \leq \Lambda^k A = A \mathrm{e}^{k \ln(\Lambda)}. \tag{A.31}$$

Introducing *autocorrelation time* τ as

$$\tau := -\frac{1}{\ln(\Lambda)}, \tag{A.32}$$

then (3.14) is obtained. The pdf $W^k p$ hence converges exponentially to π with k and the typical timescale is determined by the largest $|\lambda_j|$ excluding $\lambda = 1$.

Appendix B

Additional Figures

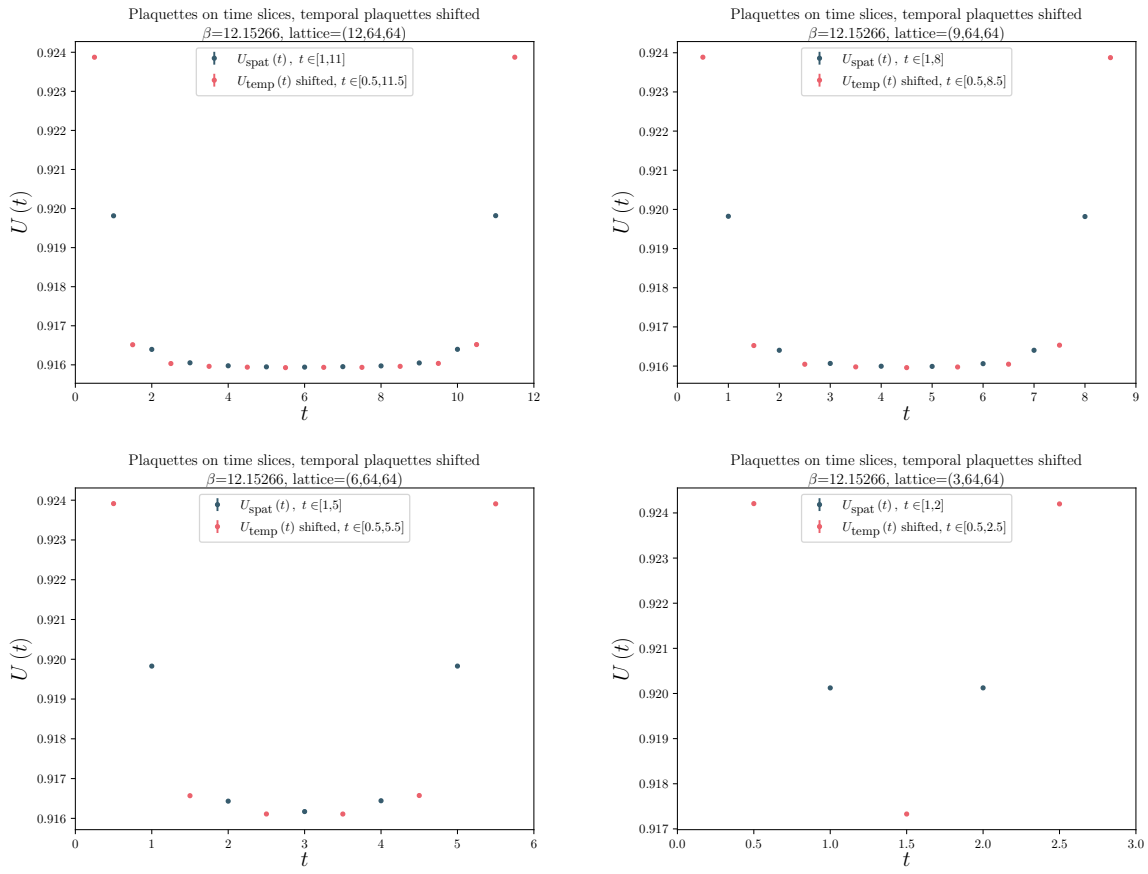


Figure B.1: Plaquette on different time slices t for $N_{\text{t}} = 3, 6, 9, 12$. Temporal plaquettes are shifted to appear halfway between the two spatial plaquettes connected to them.

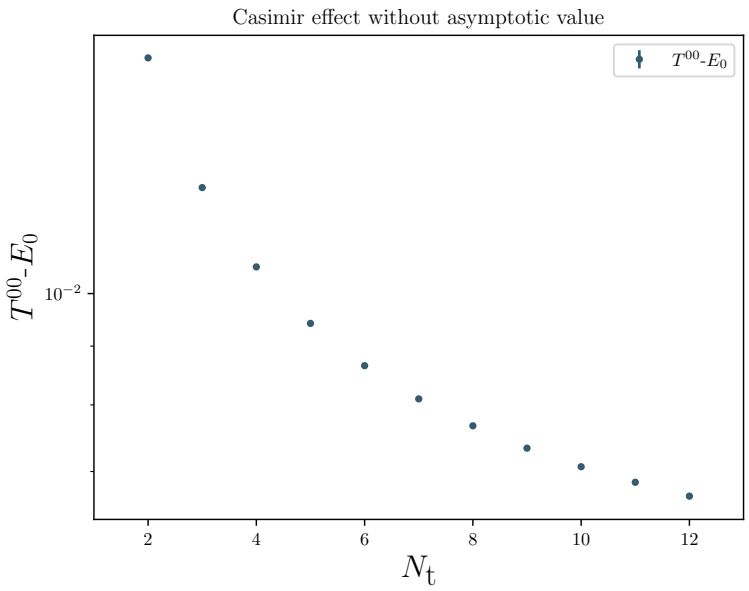


Figure B.2: Data of energy density for various N_t without the asymptotic constant value, logarithmic scale.

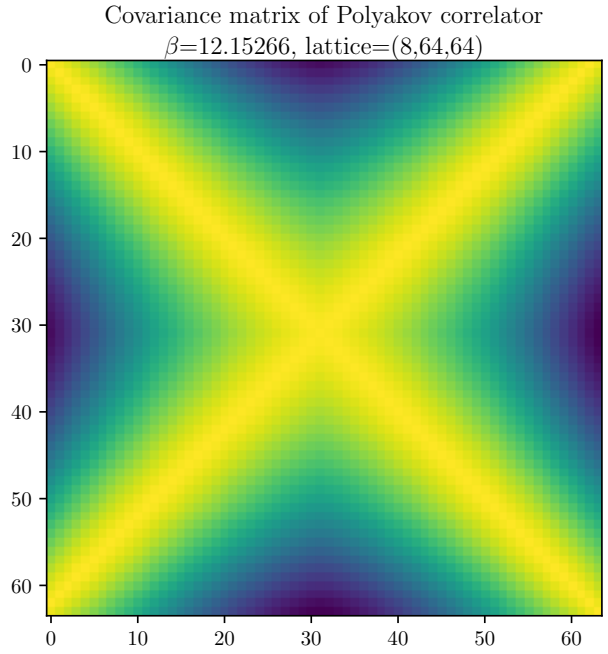


Figure B.3: Covariance matrix of Polyakov loop correlator for $N_t = 8$.

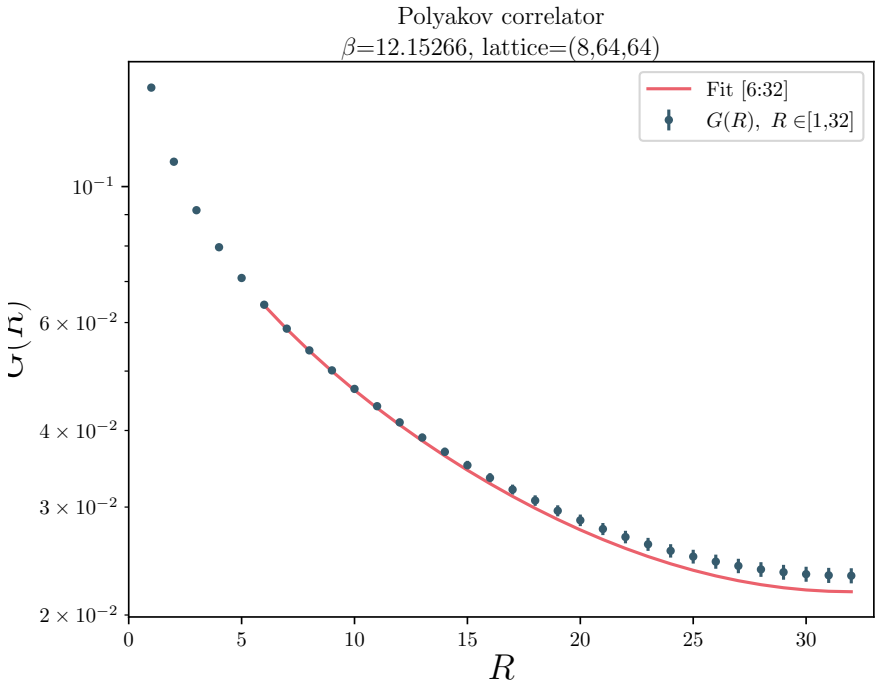


Figure B.4: Fit of $G\,(R)$ with equation (4.14) on one half of distance values, considering covariance matrix.

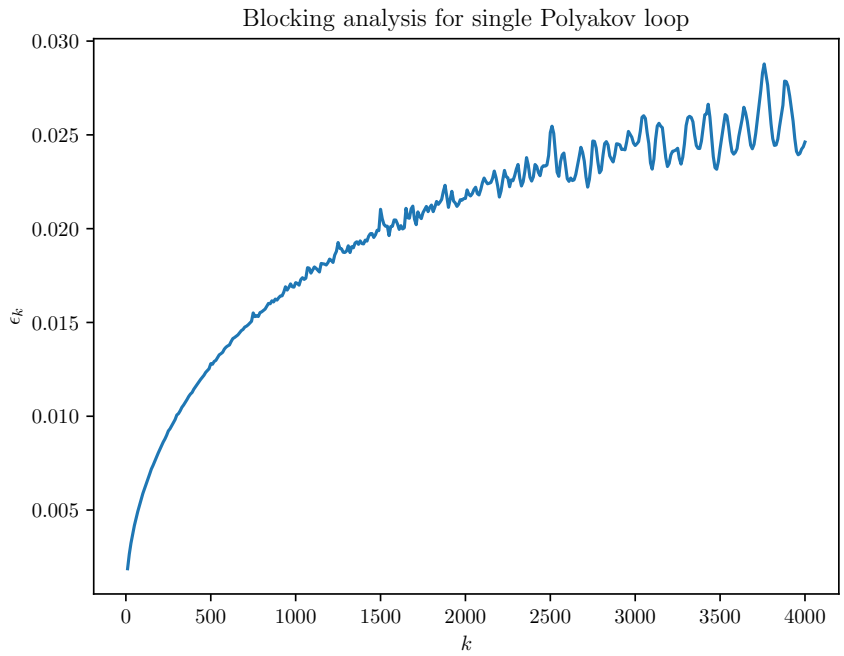


Figure B.5: Blocking error plot for Polyakov loop measures in cold initialization for $N_\mathrm{t} = 2$ with thermalization presumed after 500 Monte Carlo steps.

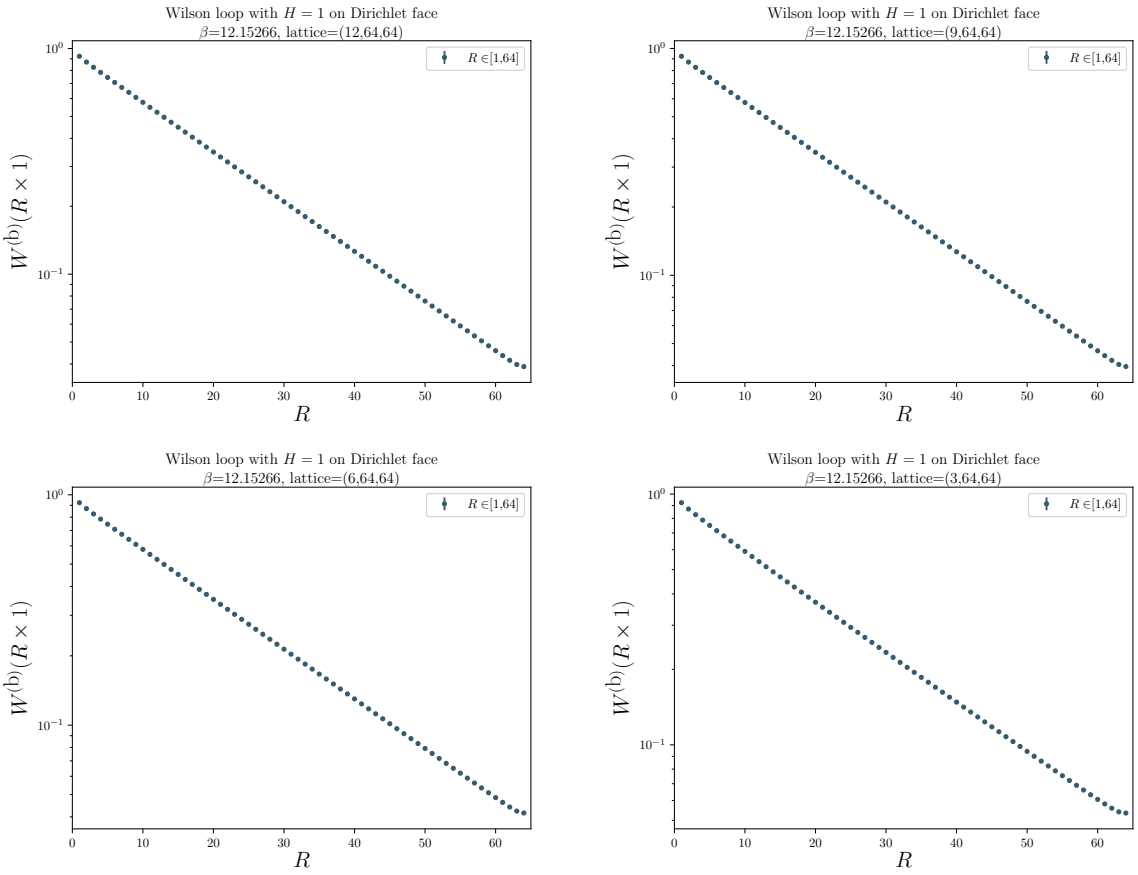


Figure B.6: Wilson on border data for $N_t = 3, 6, 9, 12$, logarithmic scale.

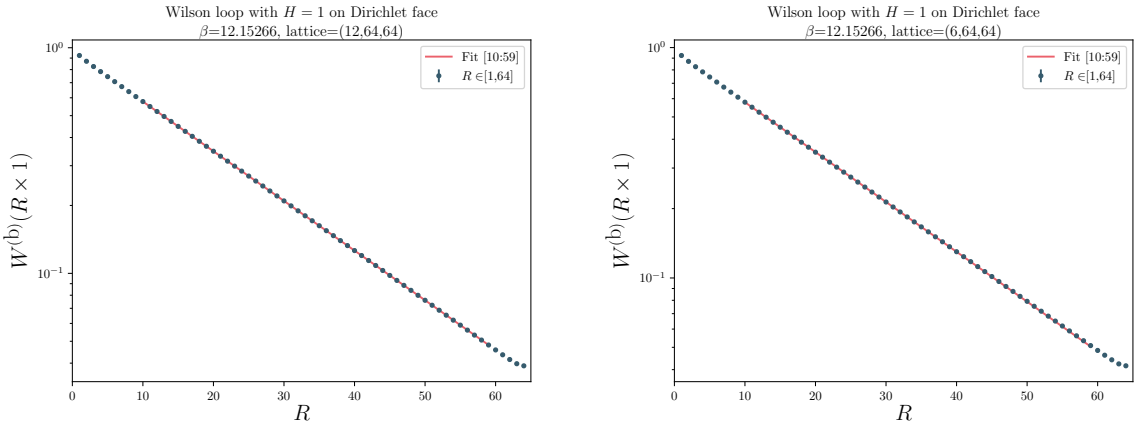


Figure B.7: Wilson data fitted with(4.19) for $N_t = 6, 9$.

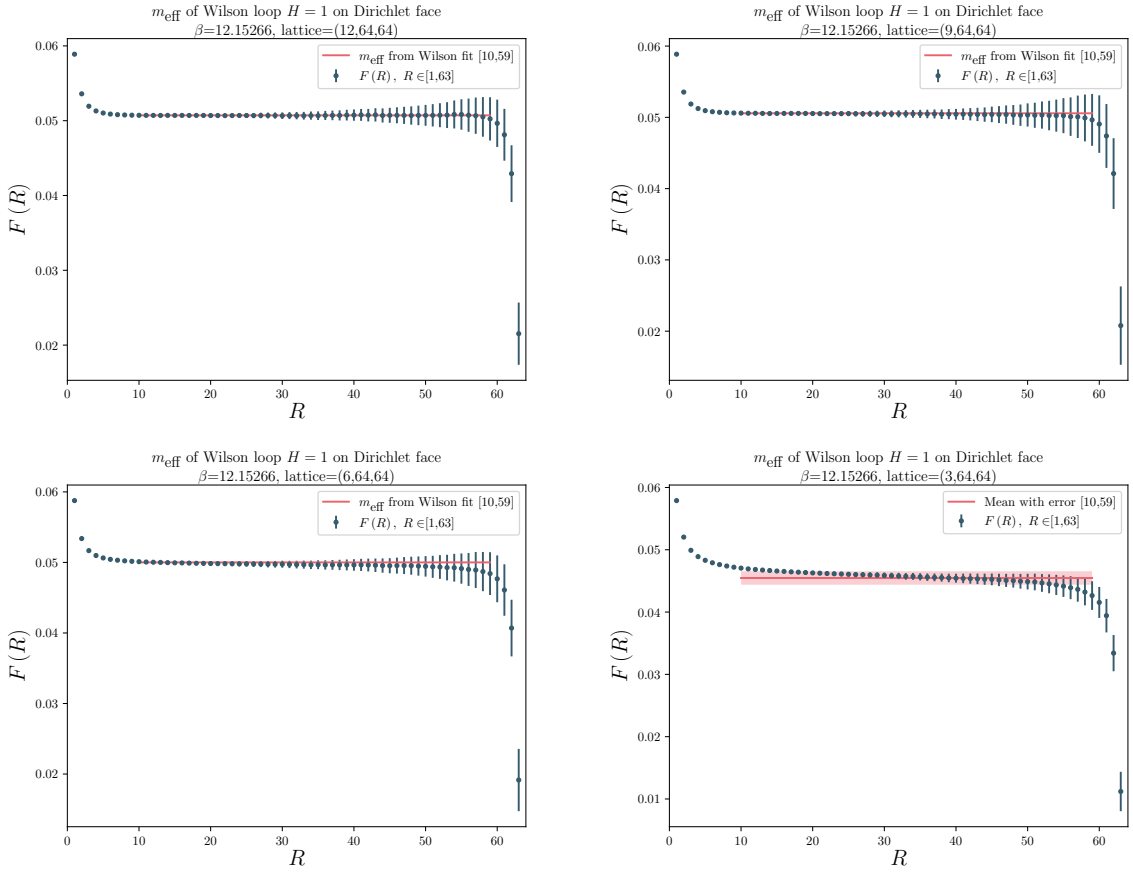


Figure B.8: Estimates of effective mass of Wilson loop on border for $N_{\text{t}} = 3, 6, 9, 12$. For $N_{\text{t}} = 3$ it is plotted the mean of selected data points, while for higher N_{t}s the free parameter from Wilson data fitted with(4.19).

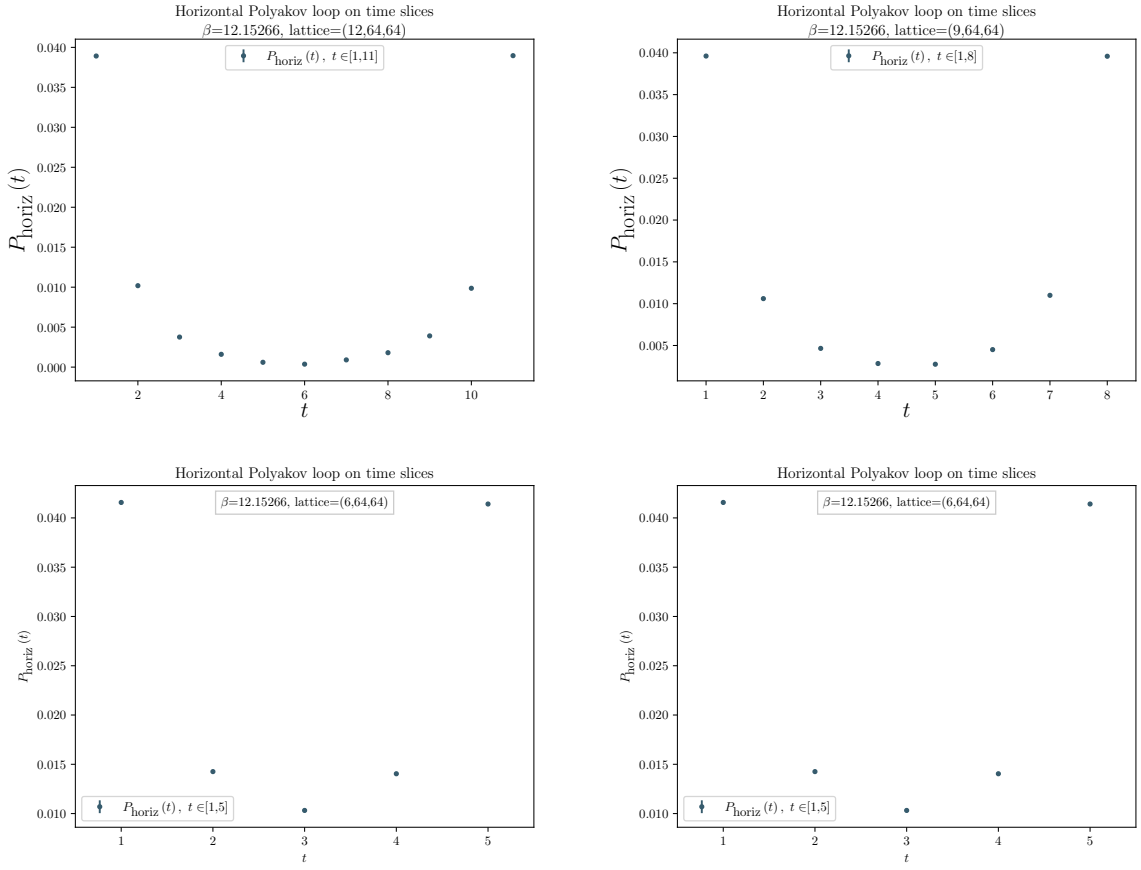


Figure B.9: Horizontal Polyakov loop on time slices for $N_{\text{t}} = 3, 6, 9, 12$.

Appendix C

Simulation Code

The code listed is from `https://github.com/marco-nebbia/yang-mills-leonardo`. It is based on Claudio Bonati's repository for simulating Yang-Mills theories on a lattice, available at `https://github.com/claudio-bonati/yang-mills`. In presenting the main components of the code, some lines are omitted, such as debug checks or memory allocation, to improve readability.

C.1 Lattice

The file `geometry.c` creates lattice structure to work on, stores it in `geo` of the defined class `Geometry`. Lattice sizes are stored in array `geo->d_size` and lattice points are identified with `r`, the lexicographic index (or single index). There are functions to switch between lexicographic index and cartesian coordinates `cartcoord` of any lattice site. `nnp[r][i]` and `nnm[r][i]` are the next neighbours points of `r` in direction `i`, respectively the next and the previous one.

```
// initialize geometry
void init_geometry(Geometry *geo, int insize[STDIM])
{
    int i, value, valuep, valuem, err;
    long r, rm, rp;
    int cartcoord[STDIM];

    for(i=0; i<STDIM; i++)
    {
        geo->d_size[i]=insize[i];
    }

    // derived constants
    geo->d_volume=1;
```

```
15      for(i=0; i<STDIM; i++)
16      {
17          (geo->d_volume)*=(geo->d_size[i]);
18      }
19
20      geo->d_space_vol=1;
21      for(i=1; i<STDIM; i++) // direction 0 is time
22      {
23          (geo->d_space_vol)*=(geo->d_size[i]);
24      }
25
26      geo->d_inv_vol=1.0/((double) geo->d_volume);
27      geo->d_inv_space_vol=1.0/((double) geo->d_space_vol);
28
29      // INITIALIZE
30      for(r=0; r<geo->d_volume; r++)
31      {
32          si_to_cart(cartcoord, r, geo);
33
34          for(i=0; i<STDIM; i++)
35          {
36              value=cartcoord[i];
37
38              valuep=value+1;
39              if(valuep >= geo->d_size[i])
40              {
41                  valuep-=geo->d_size[i];
42              }
43              cartcoord[i]=valuep;
44              rp=cart_to_si(cartcoord, geo);
45              geo->d_nnp[r][i]=rp;
46
47              valuem=value-1;
48              if(valuem<0)
49              {
50                  valuem+=geo->d_size[i];
51              }
52              cartcoord[i]=valuem;
53              rm=cart_to_si(cartcoord, geo);
54              geo->d_nnm[r][i]=rm;
55
56              cartcoord[i]=value;
57          }
58      } // end of loop on r
59
60      #ifdef DEBUG
61      test_geometry(geo);
62      #endif
63  }
```

Listing C.1: Function to initialize lattice sites based on input parameters.

```
1  // if site is on the border of direction dir
2  // and link is not orthogonal to the face, return 1 (true)
```

```
int check_link_on_border(Geometry const * const geo, long r, int dir, int dirlink)
{
    int cartcoord[STDIM];

    si_to_cart(cartcoord, r, geo);

    // if site is on the border of direction dir
    // and link is not orthogonal to the face, return 1 (true)
    if(cartcoord[dir]!=0 && cartcoord[dir]!= (geo->d_size[dir]-1)) return 0;
    else if(dirlink==dir) return 0;
    else return 1;
}
```

Listing C.2: Added function to check if a link is on lattice border.

C.2 Gauge Variables

The file `gauge_conf_def.c` initializes gauge variables.

```
void init_gauge_conf(Gauge_Conf *GC, Geometry const * const geo, GParam const * const
    param)
{
    long r, j;
    int err;

    // allocate the local lattice
    err=posix_memalign((void**) &(GC->lattice), (size_t) DOUBLE_ALIGN, (size_t) geo->
    d_volume * sizeof(GAUGE_GROUP *));
    if(err!=0)
    {
        fprintf(stderr, "Problems in allocating the lattice! (%s, %d)\n", __FILE__,
    __LINE__);
        exit(EXIT_FAILURE);
    }
    for(r=0; r<(geo->d_volume); r++)
    {
        err=posix_memalign((void**)&(GC->lattice[r]), (size_t) DOUBLE_ALIGN, (size_t )
    STDIM * sizeof(GAUGE_GROUP));
        if(err!=0)
        {
            fprintf(stderr, "Problems in allocating the lattice! (%s, %d)\n", __FILE__,
    __LINE__);
            exit(EXIT_FAILURE);
        }
    }

    #ifdef THETA_MODE
    alloc_clover_array(GC, geo);
    #endif

    // to check if we are on the face with Dirichlet boundary condition
```

```
28      // 0 <-> false, 1 <-> true
29      int check = 0;
30
31      // initialize lattice
32      if(param->d_start==0) // ordered start
33      {
34          GAUGE_GROUP aux1, aux2;
35          one(&aux1);
36
37          GC->update_index=0;
38
39          for(r=0; r<(geo->d_volume); r++)
40          {
41              for(j=0; j<STDIM; j++)
42              {
43                  #if DIRICHLET_MODE == 1
44                  int i;
45                  i=(int) j;
46
47                  if(i!=j)
48                  {
49                      fprintf(stderr, "Problems in converting long in int! (%s, %d)\n",
50                                      __FILE__, __LINE__);
51                      exit(EXIT_FAILURE);
52                  }
53                  // apply Dirichlet boundary conditions on time faces
54                  check=check_link_on_border(geo, r, 0, i);
55
56                  if(check==1)
57                  {
58                      one(&aux1);
59                      equal(&(GC->lattice[r][j]), &aux1);  // gauge variable is 1 on border
60                      continue;  // next loop iteration
61                  }
62                  #endif
63
64                  rand_matrix(&aux2);
65                  times_equal_real(&aux2, 0.001);
66                  plus_equal(&aux2, &aux1);
67                  unitarize(&aux2);
68                  equal_dag(&(GC->lattice[r][j]), &aux2);
69              }
70          }
71      }
72      if(param->d_start==1)  // random start
73      {
74          GAUGE_GROUP aux1;
75
76          GC->update_index=0;
77
78          for(r=0; r<(geo->d_volume); r++)
79          {
80              for(j=0; j<STDIM; j++)
81              {
```

```
                #if DIRICHLET_MODE == 1
                int i;
                i=(int) j;

                // apply Dirichlet boundary conditions on time faces
                check=check_link_on_border(geo, r, 0, i);

                if(check==1)
                {
                    one(&aux1);
                    equal(&(GC->lattice[r][j]), &aux1);  // gauge variable is 1 on border
                    continue;  // next loop iteration
                }
                #endif
                rand_matrix(&aux1);
                equal(&(GC->lattice[r][j]), &aux1);
            }
        }
    }

    if(param->d_start==2) // initialize from stored conf
    {
        read_gauge_conf(GC, geo, param);
    }
}
```

Listing C.3: Function to initialize gauge configuration.

The file `gauge_conf_upd.c` contains functions to update gauge variables at each Monte Carlo step.

```
// compute the staple in position r, direction i and save it in M
void calcstaples_wilson(Gauge_Conf const * const GC,
                        Geometry const * const geo,
                        long r,
                        int i,
                        GAUGE_GROUP *M)
{
    int j, l;
    long k;
    GAUGE_GROUP link1, link2, link3, link12, stap;

    #ifdef DEBUG
    if(r >= geo->d_volume)
    {
        fprintf(stderr, "r too large: %ld >= %ld (%s, %d)\n",
        r, geo->d_volume, __FILE__, __LINE__);
        exit(EXIT_FAILURE);
    }
    if(i >= STDIM)
    {
        fprintf(stderr, "i too large: i=%d >= %d (%s, %d)\n",
        i, STDIM, __FILE__, __LINE__);
        exit(EXIT_FAILURE);
```

```
24      }
25      #endif
26
27      zero(M); // M=0
28
29      for(l=i+1; l< i + STDIM; l++)
30      {
31          j = (l % STDIM);
32
33          //
34          //        i ^
35          //          |   (1)
36          //          +------>------+
37          //          |             |
38          //                        |
39          //          |             V (2)
40          //                        |
41          //          |             |
42          //          +------<------+-->   j
43          //        r      (3)
44          //
45
46          equal(&link1, &(GC->lattice[nnp(geo, r, i)][j]));  // link1 = (1)
47          equal(&link2, &(GC->lattice[nnp(geo, r, j)][i]));  // link2 = (2)
48          equal(&link3, &(GC->lattice[r][j]));               // link3 = (3)
49
50          times_dag2(&link12, &link1, &link2);  // link12=link1*link2^{dag}
51          times_dag2(&stap, &link12, &link3);   // stap=link12*link3^{dag}
52
53          plus_equal(M, &stap);
54
55          //
56          //        i ^
57          //          |   (1)
58          //          |----<------+
59          //          |           |
60          //          |
61          //      (2) V           |
62          //          |
63          //          |           |
64          //          +------>----+--->j
65          //         k      (3)    r
66          //
67
68          k=nnm(geo, r, j);
69
70          equal(&link1, &(GC->lattice[nnp(geo, k, i)][j]));  // link1 = (1)
71          equal(&link2, &(GC->lattice[k][i]));               // link2 = (2)
72          equal(&link3, &(GC->lattice[k][j]));               // link3 = (3)
73
74          times_dag12(&link12, &link1, &link2); // link12=link1^{dag}*link2^{dag}
75          times(&stap, &link12, &link3);        // stap=link12*link3
76
77          plus_equal(M, &stap);
```

```
78      }
79  }
```

Listing C.4: Function to compute staples around a link.

```
1   // perform an update with heatbath
2   void heatbath(Gauge_Conf *GC,
3                 Geometry const * const geo,
4                 GParam const * const param,
5                 long r,
6                 int i)
7   {
8       #ifdef DEBUG
9       if(r >= geo->d_volume)
10      {
11          fprintf(stderr, "r too large: %ld >= %ld (%s, %d)\n",
12                           r, geo->d_volume, __FILE__, __LINE__);
13          exit(EXIT_FAILURE);
14      }
15      if(i >= STDIM)
16      {
17          fprintf(stderr, "i too large: i=%d >= %d (%s, %d)\n",
18                          i, STDIM, __FILE__, __LINE__);
19          exit(EXIT_FAILURE);
20      }
21      #endif
22
23      GAUGE_GROUP stap;
24
25      #ifndef THETA_MODE
26      calcstaples_wilson(GC, geo, r, i, &stap);
27      #else
28      calcstaples_with_topo(GC, geo, param, r, i, &stap);
29      #endif
30
31      times_equal_real(&stap, param->d_beta);
32      single_heatbath(&(GC->lattice[r][i]), &stap);
33  }
```

Listing C.5: Function to perform a heat-bath on all lattice sites.

```
1   // perform an update with overrelaxation
2   void overrelaxation(Gauge_Conf *GC,
3                       Geometry const * const geo,
4                       GParam const * const param,
5                       long r,
6                       int i)
7   {
8       #ifdef DEBUG
9       if(r >= geo->d_volume)
10      {
11          fprintf(stderr, "r too large: %ld >= %ld (%s, %d)\n",
12                          r, geo->d_volume, __FILE__, __LINE__);
```

```
13          exit(EXIT_FAILURE);
14      }
15      if(i >= STDIM)
16      {
17          fprintf(stderr, "i too large: i=%d >= %d (%s, %d)\n",
18                          i, STDIM, __FILE__, __LINE__);
19          exit(EXIT_FAILURE);
20      }
21      #endif
22
23      GAUGE_GROUP stap;
24
25      #ifndef THETA_MODE
26      (void) param; // just to avoid warnings
27      calcstaples_wilson(GC, geo, r, i, &stap);
28      #else
29      calcstaples_with_topo(GC, geo, param, r, i, &stap);
30      #endif
31
32      single_overrelaxation(&(GC->lattice[r][i]), &stap);
33  }
```

Listing C.6: Function to perform overrelaxation on all lattice sites.

```
1  // perform a complete update
2  void update(Gauge_Conf * GC,
3  Geometry const * const geo,
4  GParam const * const param)
5  {
6      long r;
7      int j, dir;
8
9      // to check if we are on the face with Dirichlet boundary condition
10     // 0 <-> false, 1 <-> true
11     int check;
12
13     check=0;
14
15     // heatbath
16     for(dir=0; dir<STDIM; dir++)
17     {
18         #ifdef THETA_MODE
19         compute_clovers(GC, geo, dir);
20         #endif
21
22         #ifdef OPENMP_MODE
23         #pragma omp parallel for num_threads(NTHREADS) private(r)
24         #endif
25         for(r=0; r<(geo->d_volume)/2; r++)
26         {
27             #if DIRICHLET_MODE == 1
28             // don't update Dirichlet boundary conditions on time faces
29             check=check_link_on_border(geo, r, 0, dir);
30             if(check==1) continue;
```

```
            #endif

            heatbath(GC, geo, param, r, dir);
        }

        #ifdef OPENMP_MODE
        #pragma omp parallel for num_threads(NTHREADS) private(r)
        #endif
        for(r=(geo->d_volume)/2; r<(geo->d_volume); r++)
        {
            #if DIRICHLET_MODE == 1
            // don't update Dirichlet boundary conditions on time faces
            check=check_link_on_border(geo, r, 0, dir);
            if(check==1) continue;
            #endif

            heatbath(GC, geo, param, r, dir);
        }
    }

    // overrelax
    for(dir=0; dir<STDIM; dir++)
    {
        #ifdef THETA_MODE
        compute_clovers(GC, geo, dir);
        #endif

        for(j=0; j<param->d_overrelax; j++)
        {
            #ifdef OPENMP_MODE
            #pragma omp parallel for num_threads(NTHREADS) private(r)
            #endif
            for(r=0; r<(geo->d_volume)/2; r++)
            {
                #if DIRICHLET_MODE == 1
                // don't update Dirichlet boundary conditions on time faces
                check=check_link_on_border(geo, r, 0, dir);
                if(check==1) continue;
                #endif

                overrelaxation(GC, geo, param, r, dir);
            }

            #ifdef OPENMP_MODE
            #pragma omp parallel for num_threads(NTHREADS) private(r)
            #endif
            for(r=(geo->d_volume)/2; r<(geo->d_volume); r++)
            {
                #if DIRICHLET_MODE == 1
                // don't update Dirichlet boundary conditions on time faces
                check=check_link_on_border(geo, r, 0, dir);
                if(check==1) continue;
                #endif

```

```
85                  overrelaxation(GC, geo, param, r, dir);
86              }
87          }
88      }
89
90      // final unitarization
91      #ifdef OPENMP_MODE
92      #pragma omp parallel for num_threads(NTHREADS) private(r, dir)
93      #endif
94      for(r=0; r<(geo->d_volume); r++)
95      {
96          for(dir=0; dir<STDIM; dir++)
97          {
98              unitarize(&(GC->lattice[r][dir]));
99          }
100     }
101
102     GC->update_index++;
103 }
```

Listing C.7: Function to perform an update on all lattice sites, Dirichlet boundary conditions enforced.

The functions to perform single heat-bath and overrelaxation are contained in `su2_upd.c`.

```
1  // heatbath
2  //
3  // generate link according to the distibution \exp[-(1/N_c)ReTr(link*staple)]
4  // the coupling is inside the staple
5  //
6  void single_heatbath_Su2(Su2 *link, Su2 const * const staple)
7  {
8      double p, p0;
9      Su2 matrix1, matrix2;
10
11     equal_Su2(&matrix1, staple);                    // matrix1=staple
12
13     p=sqrtdet_Su2(&matrix1);
14     if(p>MIN_VALUE)
15     {
16         times_equal_real_Su2(&matrix1, 1.0/p);  // matrix1 *= 1.0/p
17         equal_dag_Su2(&matrix2, &matrix1);      // matrix2 = matrix1^{dag}
18
19         randheat_Su2(p, &p0);
20
21         rand_matrix_p0_Su2(p0, link);
22         times_equal_Su2(link, &matrix2);        // link*=matrix2
23     }
24     else
25     {
26         rand_matrix_Su2(link);
27     }
```

```
}
```

Listing C.8: Function to perform a heat-bath update on a single lattice site.

```
// overrelaxation
void single_overrelaxation_Su2(Su2 *link, Su2 const * const staple)
{
    double p;
    Su2 matrix1, matrix2;

    equal_Su2(&matrix1, staple); // matrix1=staple

    p=sqrtdet_Su2(&matrix1);
    if(p>MIN_VALUE)

    {
        times_equal_real_Su2(&matrix1, 1.0/p);     // matrix1*=(1/p)
        equal_dag_Su2(&matrix2, &matrix1);         // matrix2=matrix1^{dag}

        times_dag1_Su2(&matrix1, &matrix2, link); // matrix1=matrix2*link^{dag}
        times_Su2(link, &matrix1, &matrix2);       // link=matrix1*matrix2
    }
    else
    {
        rand_matrix_Su2(link);
    }
}
```

Listing C.9: Function to perform overrelaxation on a single lattice site.

C.3 Measures

Functions to compute observables are contained in `gauge_conf-meas.c`.

```
// computation of the plaquette (1/NCOLOR the trace of) in position r and positive
    directions i,j
double plaquettep(Gauge_Conf const * const GC,
                  Geometry const * const geo,
                  long r,
                  int i,
                  int j)
{
    GAUGE_GROUP matrix;

    #ifdef DEBUG
    if(r >= geo->d_volume)
    {
        fprintf(stderr, "r too large: %ld >= %ld (%s, %d)\n",
                        r, geo->d_volume, __FILE__, __LINE__);
        exit(EXIT_FAILURE);
    }
```

```
17      if(j >= STDIM || i >= STDIM)
18      {
19          fprintf(stderr, "i or j too large: (i=%d || j=%d) >= %d (%s, %d)\n",
20                          i, j, STDIM, __FILE__, __LINE__);
21          exit(EXIT_FAILURE);
22      }
23      #endif
24
25      //
26      //        ^ i
27      //        |   (2)
28      //        +---<---+
29      //        |       |
30      //   (3)  V       ^ (1)
31      //        |       |
32      //        +--->---+---> j
33      //        r   (4)
34      //
35
36      equal(&matrix, &(GC->lattice[nnp(geo, r, j)][i]));
37      times_equal_dag(&matrix, &(GC->lattice[nnp(geo, r, i)][j]));
38      times_equal_dag(&matrix, &(GC->lattice[r][i]));
39      times_equal(&matrix, &(GC->lattice[r][j]));
40
41      return retr(&matrix);
42  }
```

Listing C.10: Function to measure trace of a single plaquette.

```
1   // compute the mean plaquettes with normal vector
2   // parallel or orthogonal to time direction for a
3   // specific slice of the lattice
4   void plaquette_slice_time(Gauge_Conf const * const GC,
5                             Geometry const * const geo,
6                             int slice,
7                             double *plaqs,
8                             double *plaqt)
9   {
10      long r, rsp;
11      double ps, pt;
12
13      ps=0.;
14      pt=0.;
15
16      #ifdef DEBUG
17      // check if site is inside the lattice
18      if(slice>=(geo->d_size[0]))
19      {
20          fprintf(stderr, "Slice outside lattice: %d >= %d (%s, %d)\n",
21                          slice, geo->d_size[0], __FILE__, __LINE__);
22          exit(EXIT_FAILURE);
23      }
24      else if(slice<0)
25      {
```

```
        fprintf(stderr, "Slice outside lattice: %d < 0 (%s, %d)\n",
                        slice, __FILE__, __LINE__);
        exit(EXIT_FAILURE);
    }
    #endif

    int i,j;

    for(rsp=0; rsp<(geo->d_space_vol); rsp++)
    {
        r=sisp_and_t_to_si(geo, rsp, slice);

        i=0;
        for(j=1; j<STDIM; j++)
        {
            pt+=plaquettep(GC, geo, r, i, j);
        }

        for(i=1; i<STDIM; i++)
        {
            j=i+1;

            while(j<STDIM)
            {
                ps+=plaquettep(GC, geo, r, i, j);
                j++;
            }
        }
    }

    if(STDIM>2)
    {
        ps*=geo->d_inv_space_vol;
        ps/=((double) (STDIM-1)*(STDIM-2)/2);
    }
    else
    {
        ps=0.0;
    }

    pt*=geo->d_inv_space_vol;
    pt/=((double) STDIM-1);

    *plaqs=ps;
    *plaqt=pt;
}
```

Listing C.11: Added function to measure trace of spatial and temporal plaquettes on different time slices of the lattice.

```
// compute the Polyakov loop (the trace of) starting from the site r (its time column)
void polyakov_fixed_site(Gauge_Conf const * const GC,
                         Geometry const * const geo,
```

```
                        long r,
                        double *repoly,
                        double *impoly)
{
    GAUGE_GROUP matrix;

    one(&matrix);

    long rsp;
    int t;

    // rsp is single index for spatial coordinates,
    // t is time coordinate
    si_to_sisp_and_t(&rsp, &t, geo, r);

    // be sure to start from bottom of the lattice in time dimension
    r=sisp_and_t_to_si(geo, rsp, 0);

    int x0=0;

    // last link is outside lattice if Dirichlet BC are imposed
    // on both bottom and top faces, so it doesn't take part in
    // Polyakov loop, so if DIRICHLET_MODE == 1 then x0 stops at
    // geo->d_size[0]-1
    for(x0=0; x0<geo->d_size[0]; x0++)
    {
        #if DIRICHLET_MODE == 1
        if(x0 == geo->d_size[0]-1) break;
        #endif
        times_equal(&matrix, &(GC->lattice[r][0]));

        r=nnp(geo, r, 0);
    }

    if(x0!=geo->d_size[0]-DIRICHLET_MODE)
    {
        fprintf(stderr, "Links multiplied: %d (should be %d)\n",
        x0, geo->d_size[0] - DIRICHLET_MODE);
        exit(EXIT_FAILURE);
    }

    *repoly=retr(&matrix);
    *impoly=imtr(&matrix);
}
```

Listing C.12: Added function to measure horizontal Polyakov loop on a single site.

```
// compute the horizontal Polyakov loop (the trace of) starting from the site r
// in horizontal direction dir
void polyakov_horizontal_fixed_site(Gauge_Conf const * const GC,
                                    Geometry const * const geo,
                                    long r,
                                    int dir,
                                    double *repoly,
```

```
                                    double *impoly)
{
    #ifdef DEBUG
    if(dir>STDIM)
    {
        fprintf(stderr, "Direction of horizontal Polyakov loop
                        is greater than space-time dimensions! (%s, %d)\n",
                        __FILE__, __LINE__);
        exit(EXIT_FAILURE);
    }
    else if(dir==0)
    {
        fprintf(stderr, "Direction of horizontal Polyakov loop
                        is time direction! (%s, %d)\n",
                        __FILE__, __LINE__);
        exit(EXIT_FAILURE);
    }
    #endif

    // initialize gauge matrix and coordinates
    GAUGE_GROUP matrix;
    int cartcoord[STDIM];

    one(&matrix);
    si_to_cart(cartcoord, r, geo);

    // start from the 0th link in direction dir
    cartcoord[dir]=0;

    r=cart_to_si(cartcoord, geo);

    for(int x_dir=0; x_dir<(geo->d_size[dir]); x_dir++)
    {
        times_equal(&matrix, &(GC->lattice[r][dir]));
        r=nnp(geo, r, dir);
    }

    *repoly=retr(&matrix);
    *impoly=imtr(&matrix);
}
```

Listing C.13: Added function to measure horizontal Polyakov loop on a single site.

```
// correlator of two Polyakov loops along spatial direction dir
// averaged on all spatial sites
void polyakov_correlator_dir(Gauge_Conf const * const GC,
                             Geometry const * const geo,
                             int d,
                             int dir,
                             double *re,
                             double *im)
{
    double repoly1, repoly2;
    double impoly1, impoly2;
```

```
12      double reprod, improd;
13
14      int cartcoord[STDIM];
15      long r, rsp;
16
17      #ifdef DEBUG
18      if(dir==0)
19      {
20          fprintf(stderr, "Direction of correlator of Polyakov loops is time,
21                           should be a spatial dimension! (%s, %d)\n",
22                           __FILE__, __LINE__);
23          exit(EXIT_FAILURE);
24      }
25      else if(dir>=STDIM)
26      {
27          fprintf(stderr, "Direction of correlator of Polyakov loops is greater
28                           than space-time dimensions! %d>=%d (%s, %d)\n",
29                           dir, STDIM, __FILE__, __LINE__);
30          exit(EXIT_FAILURE);
31      }
32      if(d<=0)
33      {
34          fprintf(stderr, "Distance between Polyakov loops <=0! (%s, %d)\n",
35                           __FILE__, __LINE__);
36          exit(EXIT_FAILURE);
37      }
38      else if(d>geo->d_size[dir])
39      {
40          fprintf(stderr, "Distance between Polyakov loops
41                           greater than lattice size! %d>%d (%s, %d)\n",
42                           d, geo->d_size[dir], __FILE__, __LINE__);
43          exit(EXIT_FAILURE);
44      }
45      #endif
46
47      reprod=0.;
48      improd=0.;
49
50      for(rsp=0; rsp<(geo->d_space_vol); rsp++)
51      {
52          r=sisp_and_t_to_si(geo, rsp, 0);
53
54          polyakov_fixed_site(GC, geo, r, &repoly1, &impoly1);
55
56          si_to_cart(cartcoord, r, geo);
57          cartcoord[dir]+=d;
58
59          if(cartcoord[dir]>=(geo->d_size[dir])) cartcoord[dir]-=(geo->d_size[dir]);
60          r=cart_to_si(cartcoord, geo);
61
62          polyakov_fixed_site(GC, geo, r, &repoly2, &impoly2);
63
64          #if NCOLOR == 2
65          // if gauge group is SU(2), trace is real
```

```
        reprod+=repoly1*repoly2;
        improd=0.;
        #elif NCOLOR > 2
        // if gauge group is SU(N) with N!=2, trace is complex
        // so conjugate the 2nd loop to take it in downward sense
        // impoly2->-impoly2
        // and use complex multiplication rules
        reprod+=repoly1*repoly2+impoly1*impoly2;
        improd+=-repoly1*impoly2+repoly2*impoly1;
        #endif
    }

    *re=reprod*(geo->d_inv_space_vol);
    *im=improd*(geo->d_inv_space_vol);
}
```

Listing C.14: Added function to measure Polyakov loop correlator along a specific direction.

```
// correlator of two Polyakov loops across all spatial directions
// averaged on all spatial sites
void polyakov_correlator(Gauge_Conf const * const GC,
                         Geometry const * const geo,
                         int d,
                         double *re,
                         double *im)
{
    double reprod, improd;
    double rememo, immemo;

    #ifdef DEBUG
    if(d<=0)
    {
        fprintf(stderr, "Distance between Polyakov loops <=0! (%s, %d)\n",
                        __FILE__, __LINE__);
        exit(EXIT_FAILURE);
    }
    #endif

    reprod=0.;
    improd=0.;

    rememo=0.;
    immemo=0.;

    for(int dir=1; dir<STDIM; dir++)
    {
        polyakov_correlator_dir(GC, geo, d, dir, &reprod, &improd);

        rememo+=reprod;
        immemo+=improd;
    }

```

```
    *re=rememo/(STDIM-1);
    *im=immemo/(STDIM-1);
}
```

Listing C.15: Added function to measure the average Polyakov loop correlator.

```
// compute the Wilson loop of dimensions d_i*d_j on directions (i,j)
// starting from lattice site r
void wilson_fixed_site(Gauge_Conf const * const GC,
                       Geometry const * const geo,
                       long r,
                       int d_i,
                       int d_j,
                       int i,
                       int j,
                       double *re,
                       double *im)
{
    GAUGE_GROUP matrix;
    int x_i, x_j;

    #ifdef DEBUG
    if(i>=STDIM)
    {
        fprintf(stderr, "Spatial direction chosen for the Wilson loop
                         greater than STDIM! %d>=%d (%s, %d)\n",
                         i, STDIM, __FILE__, __LINE__);
        exit(EXIT_FAILURE);
    }
    if(j>=STDIM)
    {
        fprintf(stderr, "Spatial direction chosen for the Wilson loop
                         greater than STDIM! %d>=%d (%s, %d)\n",
                         j, STDIM, __FILE__, __LINE__);
        exit(EXIT_FAILURE);
    }
    if(d_i>geo->d_size[i])
    {
        fprintf(stderr, "Wilson loop size exceeds corresponding
                         lattice size! %d>%d (%s, %d)\n",
                         d_i, geo->d_size[i], __FILE__, __LINE__);
        exit(EXIT_FAILURE);
    }
    if(d_j>geo->d_size[j])
    {
        fprintf(stderr, "Wilson loop size exceeds corresponding
                         lattice size! %d>%d (%s, %d)\n",
                         d_j, geo->d_size[j], __FILE__, __LINE__);
        exit(EXIT_FAILURE);
    }
    #endif

    one(&matrix);

```

```
49      // first horizontal product
50      for(x_i=0; x_i<d_i; x_i++)
51      {
52          times_equal(&matrix, &(GC->lattice[r][i]));
53          r=nnp(geo, r, i);
54      }
55
56      // first vertical product
57      for(x_j=0; x_j<d_j; x_j++)
58      {
59          times_equal(&matrix, &(GC->lattice[r][j]));
60          r=nnp(geo, r, j);
61      }
62
63      // second horizontal product, reverse order so multiply for the adjoint
64      for(x_i=d_i-1; x_i>=0; x_i--)
65      {
66          // moving first this time because the SU(2) gauge variable U[r][-i]
67          // corresponds to the adjoint of the previous link U^{dag}[r-1][i]
68          r=nnm(geo, r, i);
69          times_equal_dag(&matrix, &(GC->lattice[r][i]));
70      }
71
72      // second vertical product, reverse order so multiply for the adjoint
73      for(x_j=d_j-1; x_j>=0; x_j--)
74      {
75          // moving first this time because the SU(2) gauge variable U[r][-j]
76          // corresponds to the adjoint of the previous link U^{dag}[r-1][j]
77          r=nnm(geo, r, j);
78          times_equal_dag(&matrix, &(GC->lattice[r][j]));
79      }
80
81      *re=retr(&matrix);
82      *im=imtr(&matrix);
83  }
```

Listing C.16: Added function to measure a single Wilson loop from a site.

```
1  // compute the mean Wilson loop of size R x T
2  // where R is a distance in direction dir
3  // and T is a distance in time direction
4  // on a whole slice in time dimension
5  void wilson_slice_time_dir(Gauge_Conf const * const GC,
6                             Geometry const * const geo,
7                             int R,
8                             int T,
9                             int slice,
10                            int dir,
11                            double *re,
12                            double *im)
13 {
14     long r, rsp;
15     double rewilson, imwilson, rewil_memo, imwil_memo;
16
```

```
17      #ifdef DEBUG
18      // check if direction is not greater than STDIM or null
19      if(dir>=STDIM)
20      {
21          fprintf(stderr, "Spatial direction chosen for the Wilson loop
22                           greater than STDIM! %d>=%d (%s, %d)\n",
23                           dir, STDIM, __FILE__, __LINE__);
24          exit(EXIT_FAILURE);
25      }
26      else if(dir==0)
27      {
28          fprintf(stderr, "Spatial direction chosen for the Wilson loop
29                           is actually time dimension! dir=%d (%s, %d)\n",
30                           dir, __FILE__, __LINE__);
31          exit(EXIT_FAILURE);
32      }
33      // check if slice to compute Wilson loop on is inside the lattice
34      if(slice>=(geo->d_size[0]))
35      {
36          fprintf(stderr, "Time slice to compute Wilson loop on greater
37                           than time size! %d>=%d (%s, %d)\n",
38                           slice, (geo->d_size[0]), __FILE__, __LINE__);
39          exit(EXIT_FAILURE);
40      }
41      // check if sizes of Wilson loop are smaller
42      // of the corresponding lattice sizes
43      if(R>(geo->d_size[dir]))
44      {
45          fprintf(stderr, "Spatial size R of Wilson loop greater than
46                           corresponding spatial lattice size! %d>%d (%s, %d)\n",
47                           R, (geo->d_size[dir]), __FILE__, __LINE__);
48          exit(EXIT_FAILURE);
49      }
50      if((slice+T)>(geo->d_size[0]))
51      {
52          fprintf(stderr, "Temporal size T of Wilson loop crossing
53                           the corresponding lattice border! %d+%d>%d (%s, %d)\n",
54                           T, slice, geo->d_size[0], __FILE__, __LINE__);
55          exit(EXIT_FAILURE);
56      }
57      #endif
58
59      rewilson=0.;
60      imwilson=0.;
61
62      rewil_memo=0.;
63      imwil_memo=0.;
64
65      for(rsp=0; rsp<(geo->d_space_vol); rsp++)
66      {
67          r=sisp_and_t_to_si(geo, rsp, slice);
68
69          wilson_fixed_site(GC, geo, r, R, T, dir, 0, &rewilson, &imwilson);
70
```

```
        rewil_memo+=rewilson;
        imwil_memo+=imwilson;
    }

    *re=rewil_memo*(geo->d_inv_space_vol);
    *im=imwil_memo*(geo->d_inv_space_vol);
}
```

Listing C.17: Added function to measure a single Wilson loop on a timeslice in a certain spatial direction.

```
// compute the mean Wilson loop of size R x T on every spatial direction
// where R is a distance in space and T is a distance in time
// on a whole slice in time dimension
void wilson_slice_time(Gauge_Conf const * const GC,
                       Geometry const * const geo,
                       int R,
                       int T,
                       int slice,
                       double *re,
                       double *im)
{
    double rewilson, imwilson, rewil_memo, imwil_memo;

    #ifdef DEBUG
    // check if slice to compute Wilson loop on is inside the lattice
    if(slice>=(geo->d_size[0]))
    {
        fprintf(stderr, "Time slice to compute Wilson loop on
                         is greater than time size! %d>=%d (%s, %d)\n",
                         slice, (geo->d_size[0]), __FILE__, __LINE__);
        exit(EXIT_FAILURE);
    }
    // check if sizes of Wilson loop are smaller
    // of the corresponding lattice sizes
    for(int dir=1; dir<STDIM; dir++)
    {
        if(R>(geo->d_size[dir]))
        {
            fprintf(stderr, "Spatial size R of Wilson loop greater than
                             corresponding spatial lattice size! %d>%d (%s, %d)\n",
                             R, (geo->d_size[dir]), __FILE__, __LINE__);
            exit(EXIT_FAILURE);
        }
    }
    if((slice+T)>(geo->d_size[0]))
    {
        fprintf(stderr, "Temporal size T of Wilson loop crossing
                         the corresponding lattice border! %d+%d>%d (%s, %d)\n",
                         T, slice, geo->d_size[0], __FILE__, __LINE__);
        exit(EXIT_FAILURE);
    }
    #endif
```

```
43
44      rewilson=0.;
45      imwilson=0.;
46
47      rewil_memo=0.;
48      imwil_memo=0.;
49
50      for(int i=1; i<STDIM; i++)
51      {
52          wilson_slice_time_dir(GC, geo, R, T, slice, i, &rewilson, &imwilson);
53
54          rewil_memo+=rewilson;
55          imwil_memo+=imwilson;
56      }
57
58      *re=(double) (rewil_memo/(STDIM-1));
59      *im=(double) (imwil_memo/(STDIM-1));
60  }
```

Listing C.18: Added function to measure the average Wilson loop on a timeslice.

```
1   void perform_measures_localobs(Gauge_Conf const * const GC,
2                                  Geometry const * const geo,
3                                  GParam const * const param,
4                                  FILE *datafilep,
5                                  FILE *monofilep)
6   {
7       double plaqs=0., plaqt=0.;
8       double polyre=0., polyim=0.;
9       double wilre=0., wilim=0.;
10      //double prod_polyre=0., prod_polyim=0.;
11      double linkre=0., linkim=0.;
12
13      int dist_max=param->d_dist_poly;
14
15      polyakov_dirich(GC, geo, &polyre, &polyim);
16
17      fprintf(datafilep, "%ld ", GC->update_index);
18      fprintf(datafilep, "%.12g %.12g ", polyre, polyim);
19
20      // to measure correlators between spins on bottom face
21      for(int dist=1; dist<=dist_max; dist++)
22      {
23          wilre=0.;
24          wilim=0.;
25
26          wilson_slice_time(GC, geo, dist, 1, 0, &wilre, &wilim);
27          fprintf(datafilep, "%d %.12g %.12g ", dist, wilre, wilim);
28      }
29
30      // to measure correlators between spins of PCM
31      for(int dist=1; dist<=dist_max; dist++)
32      {
33          wilre=0.;
```

```
34          wilim=0.;
35
36          wilson_slice_time(GC, geo, dist, 1, (geo->d_size[0])-1, &wilre, &wilim);
37          fprintf(datafilep, "%d %.12g %.12g ", dist, wilre, wilim);
38      }
39
40      // to measure average value of spins of PCM
41      link_vertical_timeslice(GC, geo, geo->d_size[0]-1, &linkre, &linkim);
42      fprintf(datafilep, "%.12g %.12g ", linkre, linkim);
43
44      // to measure correlators between Polyakov loops
45      for(int dist=1; dist<=dist_max; dist++)
46      {
47          polyre=0.;
48          polyim=0.;
49
50          polyakov_correlator(GC, geo, dist, &polyre, &polyim);
51          fprintf(datafilep, "%d %.12g %.12g ", dist,  polyre, polyim);
52      }
53
54      // to measure plaquettes (spatial and temporal) on time slices
55      for(int slice=0; slice<(geo->d_size[0]); slice++)
56      {
57          plaqs=0.;
58          plaqt=0.;
59
60          plaquette_slice_time(GC, geo, slice, &plaqs, &plaqt);
61          fprintf(datafilep, "%d %.12g %.12g ", slice,  plaqs, plaqt);
62      }
63
64      // to measure horizontal Polyakov loop on time slices
65      for(int slice=0; slice<(geo->d_size[0]); slice++)
66      {
67          polyre=0.;
68          polyim=0.;
69
70          polyakov_horizontal_timeslice(GC, geo, slice, &polyre, &polyim);
71          fprintf(datafilep, "%d %.12g %.12g ", slice,  polyre, polyim);
72      }
73  }
```

Listing C.19: Function to measure observables.

C.4 Main

File `yang_mills_local.c` creates an executable that performs the simulation based on the values in `template_input.in`.

```
1  void real_main(char *in_file)
2  {
3      Gauge_Conf GC;
```

```
    Geometry geo;
    GParam param;

    char name[STD_STRING_LENGTH], aux[STD_STRING_LENGTH];
    int count;
    FILE *datafilep, *monofilep;
    time_t time1, time2;

    // read input file
    readinput(in_file, &param);

    // initialize random generator
    initrand(param.d_randseed);

    // open data_file
    init_data_file(&datafilep, &param);

    // open mon_file
    if(param.d_mon_meas == 1)
    {
        init_mon_file(&monofilep, &param);
    }

    // initialize geometry
    init_geometry(&geo, param.d_sizeg);

    // initialize gauge configuration
    init_gauge_conf(&GC, &geo, &param);

    // montecarlo
    time(&time1);
    // count starts from 1 to avoid problems using %
    for(count=1; count < param.d_sample + 1; count++)
    {
        update(&GC, &geo, &param);

        if(count % param.d_measevery ==0 && count >= param.d_thermal)
        {
            perform_measures_localobs(&GC, &geo, &param, datafilep, monofilep);
        }

        // save configuration for backup
        if(param.d_saveconf_back_every!=0)
        {
            if(count % param.d_saveconf_back_every == 0 )
            {
                // simple
                write_conf_on_file(&GC, &geo, &param);

                // backup copy
                write_conf_on_file_back(&GC, &geo, &param);
            }
        }

```

```
58          // save configuration for offline analysis
59          if(param.d_saveconf_analysis_every!=0)
60          {
61              if(count % param.d_saveconf_analysis_every == 0 )
62              {
63                  strcpy(name, param.d_conf_file);
64                  sprintf(aux, "%ld", GC.update_index);
65                  strcat(name, aux);
66                  write_conf_on_file_with_name(&GC, &geo, name);
67              }
68          }
69      }
70      time(&time2);
71      // montecarlo end
72
73      // close data file
74      fclose(datafilep);
75
76      // close mon file
77      if(param.d_mon_meas == 1)
78      {
79          fclose(monofilep);
80      }
81
82      // save configuration
83      if(param.d_saveconf_back_every!=0)
84      {
85          write_conf_on_file(&GC, &geo, &param);
86      }
87
88      // print simulation details
89      print_parameters_local(&param, time1, time2);
90
91      // free gauge configuration
92      free_gauge_conf(&GC, &geo);
93
94      // free geometry
95      free_geometry(&geo);
96  }
```

Listing C.20: Function to run the simulation.

```
1   void print_template_input(void)
2   {
3       FILE *fp;
4
5       fp=fopen("template_input.in", "w");
6
7       if(fp==NULL)
8       {
9           fprintf(stderr, "Error in opening the file template_input.in (%s, %d)\n",
10                          __FILE__, __LINE__);
11          exit(EXIT_FAILURE);
12      }
```

```
13      else
14      {
15          fprintf(fp, "size 3 64 64\n");
16          fprintf(fp,"\n");
17          fprintf(fp, "beta 12.15266\n");
18          fprintf(fp, "theta 0.0\n");
19          fprintf(fp,"\n");
20          fprintf(fp, "sample    8000\n");
21          fprintf(fp, "thermal   0\n");
22          fprintf(fp, "overrelax 5\n");
23          fprintf(fp, "measevery 1\n");
24          fprintf(fp, "monomeas  0  # 1=monopoles measures are performed\n");
25          fprintf(fp, "dist_poly 64\n");
26          fprintf(fp,"\n");
27          fprintf(fp, "start                   1  # 0=ordered  1=random  2=from saved
    configuration\n");
28          fprintf(fp, "saveconf_back_every     5  # if 0 does not save, else save backup
    configurations every ... updates\n");
29          fprintf(fp, "saveconf_analysis_every 0  # if 0 does not save, else save
    configurations for analysis every ... updates\n");
30          fprintf(fp, "\n");
31          fprintf(fp, "coolsteps  3     # number of cooling steps to be used\n");
32          fprintf(fp, "coolrepeat 5     # number of times 'coolsteps' are repeated\n");
33          fprintf(fp,"\n");
34          fprintf(fp, "#output files\n");
35          fprintf(fp, "conf_file  conf.dat\n");
36          fprintf(fp, "data_file  dati.dat\n");
37          fprintf(fp, "mon_file   mon.dat\n");
38          fprintf(fp, "log_file   log.dat\n");
39          fprintf(fp, "\n");
40          fprintf(fp, "randseed RANDSEED    #(0=time, RANDSEED=takes random seed from SLURM)
    \n");
41          fclose(fp);
42      }
43  }
```

Listing C.21: Function to print the template with simulation parameters.

```
1   int main (int argc, char **argv)
2   {
3       char in_file[50];
4
5       if(argc != 2)
6       {
7           printf("\nPackage %s version %s\n", PACKAGE_NAME, PACKAGE_VERSION);
8           printf("Claudio Bonati %s\n", PACKAGE_BUGREPORT);
9           printf("Usage: %s input_file\n\n", argv[0]);
10
11          printf("Compilation details:\n");
12          printf("\tN_c (number of colors): %d\n", NCOLOR);
13          printf("\tST_dim (space-time dimensionality): %d\n", STDIM);
14          printf("\tNum_levels (number of levels): %d\n", NLEVELS);
15          printf("\tBoundary condition: ");
16          #if DIRICHLET_MODE == 1
```

```
17          printf("Dirichlet boundary condition\n");
18          #elif DIRICHLET_MODE == 0
19          printf("Periodic boundary condition\n");
20          #endif
21          printf("\n");
22          printf("\tINT_ALIGN: %s\n", QUOTEME(INT_ALIGN));
23          printf("\tDOUBLE_ALIGN: %s\n", QUOTEME(DOUBLE_ALIGN));
24
25          #ifdef DEBUG
26          printf("\n\tDEBUG mode\n");
27          #endif
28
29          printf("\n");
30
31          #ifdef __INTEL_COMPILER
32          printf("\tcompiled with icc\n");
33          #elif defined(__clang__)
34          printf("\tcompiled with clang\n");
35          #elif defined( __GNUC__ )
36          printf("\tcompiled with gcc version: %d.%d.%d\n",
37          __GNUC__, __GNUC_MINOR__, __GNUC_PATCHLEVEL__);
38          #endif
39
40          print_template_input();
41
42
43          return EXIT_SUCCESS;
44      }
45      else
46      {
47          if(strlen(argv[1]) >= STD_STRING_LENGTH)
48          {
49              fprintf(stderr, "File name too long. Increse STD_STRING_LENGTH in include/
    macro.h\n");
50          }
51          else
52          {
53              strcpy(in_file, argv[1]);
54          }
55      }
56
57      real_main(in_file);
58
59      return EXIT_SUCCESS;
60  }
```

Listing C.22: Main function, reads files and calls functions to run the simulation.

Bibliography

[1] R. P. Feynman. "Space-Time Approach to Non-Relativistic Quantum Mechanics". In: *Rev. Mod. Phys.* 20 (2 1948), pp. 367–387.

[2] J. Smit. *Introduction to Quantum Fields on a Lattice*. Cambridge Lecture Notes in Physics. Cambridge University Press, 2023.

[3] H. J. Rothe. *Lattice Gauge Theories : An Introduction (Fourth Edition)*. Vol. 43. World Scientific Publishing Company, 2012.

[4] K. G. Wilson. "Confinement of quarks". In: *Phys. Rev. D* 10 (8 1974), pp. 2445–2459.

[5] C. N. Yang and R. L. Mills. "Conservation of Isotopic Spin and Isotopic Gauge Invariance". In: *Phys. Rev.* 96 (1 1954), pp. 191–195.

[6] H. B. G. Casimir. "On the attraction between two perfectly conducting plates". In: *Indag. Math.* 10.4 (1948), pp. 261–263.

[7] F. Gursey. "On the Symmetries of strong and weak interactions". In: *Nuovo Cim.* 16 (1960), pp. 230–240.

[8] F. Green and S. Samuel. "Chiral Models: Their Implications for Gauge Theories and Large N". In: (1980).

[9] A. M. Polyakov. "Interaction of goldstone particles in two dimensions. Applications to ferromagnets and massive Yang-Mills fields". In: *Physics Letters B* 59.1 (1975), pp. 79–81.

[10] J. Balog et al. "Exact mass gap of the chiral SU(n)×SU(n) model". In: *Phys. Rev. Lett.* 69 (6 1992), pp. 873–876.

[11] M. Caselle, M. Hasenbusch, and M. Panero. "String effects in the 3d gauge Ising model". In: *Journal of High Energy Physics* 2003.01 (2003), p. 057. DOI: 10.1088/1126-6708/2003/01/057.

[12] C. Itzykson, M. E. Peskin, and J.-B. Zuber. "Roughening of Wilson's surface". In: *Physics Letters B* 95 (1980), pp. 259–264.

[13] A. Hasenfratz, E. Hasenfratz, and P. Hasenfratz. "Generalized roughening transition and its effect on the string tension". In: *Nuclear Physics* 180 (1981), pp. 353–367.

[14] I. Montvay and G. Münster. *Quantum Fields on a Lattice*. Cambridge Monographs on Mathematical Physics. Cambridge University Press, 1994.

[15] J. Von Neumann. "Various Techniques Used in Connection with Random Digits". In: *Appl. Math Ser* 12.36-38 (1951), p. 3.

[16] A. D. Kennedy and B. J. Pendleton. "Improved heatbath method for Monte Carlo calculations in lattice gauge theories". In: *Physics Letters B* 156.5 (1985), pp. 393–399.

[17] F. Caristo et al. "Fine corrections in the effective string describing SU(2) Yang-Mills theory in three dimensions". In: *Journal of High Energy Physics* 2022.3 (2022).

[18] S. R. Edwards and L. von Smekal. "SU(2) lattice gauge theory in 2+1 dimensions: Critical couplings from twisted boundary conditions and universality". In: *Physics Letters B* 681.5 (2009), 484–490.

[19] D. Karabali and V. P. Nair. "Casimir effect in (2+1)-dimensional Yang-Mills theory as a probe of the magnetic mass". In: *Physical Review D* 98.10 (2018).

[20] S. Coleman. "There are no Goldstone bosons in two dimensions". In: *Communications in Mathematical Physics* 31.4 (1973), pp. 259–264.

[21] N. D. Mermin and H. Wagner. "Absence of Ferromagnetism or Antiferromagnetism in One- or Two-Dimensional Isotropic Heisenberg Models". In: *Phys. Rev. Lett.* 17 (22 1966), pp. 1133–1136.

[22] P. C. Hohenberg. "Existence of long-range order in one and two dimensions". In: *Phys. Rev.* 158 (June 1967).